

第一轮已完结LeetCode Hot100解题笔记 260303更新

前言

有点惊讶吧？这个Hot100解题笔记还是提前到来了。因为呢，dp题单实在是太多了，然后面试题又有大部分都出现在Hot100里面。我的目标是把hot100刷个2遍左右，所以我准备现在抢跑：将dp题单和hot100同时往前推(在文章创建的时候是这样的，后面我猜可能dp会先停下来，然后继续推hot100)。

话不多说，下面直接开始解题！

PS：下面的数字格式是 序号-力扣的题目编号，解题语言默认cpp。

1-1 两数之和

题目链接：<https://leetcode.cn/problems/two-sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2025.12.15

标签：数组 哈希表

虽说这是一道简单题，直接套双重循环也能过，但是这样总是差点意思。我是看了一眼题目的提示才知道要用哈希表(注意！ `unordered_map` 才是哈希表， `map` 不是！)。

基本逻辑就是，利用哈希表查找速度 $O(1)$ 的特性，把数组和下标对应丢进映射，然后遍历数组，将target与当前元素作差，然后在映射里面查找这个：如果找得到，就取出两个下标返回；如果找不到，就接着往下找，总能找到的。

然后呢，题目不允许取同一个下标两次，所以还得在取出下标的时候判断一下是否和当前下标相等，不然不能取。

至于你们可能担心的数组有重复数字的问题，不用担心，下面的代码在写映射的时候默认保留的是那个数字最大的下标，而我们遍历是从前往后遍历的，就算这是唯一解也凑得出。是不是有点 **bug+bug=没有bug** 的感觉了哈哈哈

上代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> dict;
        //1.给数字-下标对放进哈希映射
        for (int i=0;i<nums.size();i++) {
            dict[nums[i]] = i;
        }
        //2.遍历找搭配
        for (int i=0;i<nums.size();i++) {
            int x = target-nums[i];
            if (dict.count(x)>0 && dict[x]!=i) {
                return vector<int>{i,dict[x]}; //题目说返回顺序随意，所以我就懒得排序
            }
        }
        return vector<int>{0,0}; //这一行是为了防止报错，题目用例保证有且只有一个答案，所以不会走到这里的
    }
};

```

2-49 字母异位词分组

题目链接: <https://leetcode.cn/problems/group-anagrams/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.15

标签: 数组 哈希表 字符串 排序

这题还是挺有意思的，你看标签就知道了，需要先把每个字符串排序，这样才能知道它们互相之间是不是字母异位词。

那么怎么排序呢？

我目前学过的语言有Python, Go, Cpp这三个，其中前两者的字符串(string)类型都是不可变的，但是cpp的string属实是让我大开眼界了：不但可变(支持用sort排序)，而且还能拿来作为哈希表映射的键(可哈希)，这写起来不要太舒适。

所以，用cpp做这道题，直接遍历一遍字符串数组，然后对每个字符串直接使用sort就行了，非常方便。

然后下面的逻辑可能稍微有点绕，我干讲也不太好讲，直接在注释里面和代码一起讲好了。

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        //1.拷贝一份字符串数组，并给每个字符串排序
        auto strs2 = strs; //这里用于存放唯一的字母异位词，大概率会出现一堆重复的字符串(从题目的用例来看)
        for (auto & s : strs2) {
            sort(s.begin(), s.end());
        }
        //2.使用哈希表管理二维数组的插入和下标管理
        unordered_map<string, int> dict; //这个映射存放已经插入的字符串在ans数组中的位置
        vector<vector<string>> ans; //存放答案的二维数组
        //这里用位置遍历是利用了strs和strs2的唯一字母异位词与原字符串的对齐，方便取出原字符串
        for (int i=0; i<strs.size(); i++) {
            if (dict.count(strs2[i]) == 0) { //如果在dict里面没找到这个唯一的字母异位词，说明是新建
                dict[strs2[i]] = ans.size(); //将这一系列字母异位词在ans数组中的位置存进dict
                ans.emplace_back(vector<string>{strs[i]}); //然后插入新的一行字符串数组
            } else { //如果有，只需找到对应的下标然后插入即可
                ans[dict[strs2[i]]].emplace_back(strs[i]);
                //从dict中取出这一系列的字母异位词所在的ans数组的下标，然后插入最新的字母异位词
            }
        }
        return ans;
    }
};

```

看懂了吧？总结一下，就是利用排序后strs2数组中的字母异位词的唯一性，将其与strs中的原字符串相匹配，再配合映射进行它在ans二维数组中插入位置的管理，从而形成了有点小绕的逻辑闭环。

3-283 移动零

题目链接：<https://leetcode.cn/problems/move-zeroes/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2025.12.16

标签：数组 双指针

好久没做双指针的题目了，虽说这道题的难度是简单，并且我看到题目没几秒钟就想到了用额外空间该怎么做。

但是题目要求原地修改数组。我真求你了。

然后我又想了一会，想出了用一个指针找0，另一个指针负责把后面的元素一个个往前挪到前面的指针指向的位置把0盖掉，纯纯的用时间复杂度换空间复杂度，我真没招了。

然后咨询了一下ai，也算是回到了正轨：使用 快慢指针模型 来解决这道题目。

啥是快慢指针模型？

“快慢指针 - 读写模型”（或者叫“填充/覆盖模型”）：

- 快指针 (**Reader**)：负责读数据，判断这个数据要不要保留。
- 慢指针 (**Writer**)：负责写数据，始终指向“下一个有效数据的存放位置”。

在这道题目里面，快指针(下面统称为fast)负责寻找非0元素，而慢指针(下面统称为slow)负责寻找fast指针找到的非0元素的存放位置。

基本逻辑是这样的：fast一直往后挪，不停，所以直接用for循环就行；slow只有在fast找到非0元素时，才会将fast和slow所指元素交换，然后往后挪一位。

并且，这个设计非常精妙，保证了当发生交换时只有以下两种情况：要么是slow指向0，fast指向非0，要么是fast和slow一起指向同一个非0元素，也就是进行了“原地交换”，无事发生。

这是如何保证的呢？因为在上面的逻辑里面，假设当前fast和slow都指向一个非0元素，而下一个元素就是0，然后按照逻辑，fast和slow都往后挪一位，此时slow就自然指向了0。然后呢，由于fast是一直往后的，所以下一轮循环，由于fast指向了0，所以，slow不会往后。这样自然而然，就使得fast指向非0而slow指向了0。也可以这么理解：slow只是正在维护非0元素的边界(有点像插入排序，都是维护已处理-未处理的边界)。

当 fast 和 slow 错开时，slow 和 fast 中间夹着的那个区域（即 `nums[slow...fast-1]`），全都是 0。这就解释了为什么要把 `nums[fast]` 换过来——因为要把中间的这堆 0 往后挤！

这样逻辑就很清楚了吧？看代码：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    void moveZeroes(vector<int>& nums) {
        int slow=0;
        for (int fast=0; fast<nums.size(); fast++) {
            if (nums[fast]!=0) { //只有遇到非0才会往后挪slow
                swap(nums[fast],nums[slow]);
                slow++;
            }
        }
    }
};
```

总结：一旦遇到数组原地筛选/移除/移动类题目，直接套用：Fast 找目标，Slow 守坑位。

4-128 最长连续序列

题目链接: <https://leetcode.cn/problems/longest-consecutive-sequence/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.17

标签: 并查集 数组 哈希表

这是STL库被资本(LeetCode)做局的一题。

这道题的思路是很简单的: 利用哈希表查询速度 $O(1)$ 的特性, 将本来 $O(n^2)$ 的双指针暴力查询降级到 $O(n)$ 的时间复杂度。

首先给所有元素都丢进 `unordered_set` 里面, 顺带着还自动去重; 然后呢, 遍历 `nums` 数组, 利用集合的 `count` 方法, 另外开一个循环查询集合里面是否有当前元素 $x+1, x+2 \dots x+n$, 由此得出长度就是 n 。

但是呢, 这样会产生大量无用的查询: 我后面遍历到 $x+1$, 亦或是 $x+2$, 甚至到 $x+n$, 都会重新执行一遍这个逻辑, 给时间复杂度又退化到 $O(n^2)$ 了, 这可不行。

解决方法也很简单: 在进入这个循环之前, 检查集合里面是否有当前元素 $x-1$ 。如果有, 说明这绝对不是某个序列的开头, 就自然不会重复进入了。

你是不是以为这样就完事了? 我也是这么想的, 我的第一版代码就是这样的:

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> uSet;
        //1. 给数字丢进去
        for (auto &x:nums) {
            uSet.emplace(x);
        }
        //2. 开始遍历
        int maxLength = 0;
        for (auto &x:nums) {
            if (uSet.count(x-1)==0) { //防止重复操作, 保证这个数就一定是开头
                int cnt=0;
                while (uSet.count(x+cnt)) {
                    cnt++;
                }
                maxLength = max(maxLength, cnt);
            }
        }
        return maxLength;
    }
};
```

我就疑惑了，为啥呢？一问ai，我自己也没绷住：

你的代码会超时（TLE），主要原因是 C++ 标准库中 `unordered_set` 的底层实现问题，导致遭到针对性数据的“哈希碰撞攻击”。

1. 哈希碰撞（主要原因）

- 正常情况：哈希表的增删查改是 $O(1)$ 的。
- 最坏情况：当大量数字映射到哈希表的同一个“桶”时，哈希表会退化成链表，操作复杂度变成 $O(n)$ 。
- 结果：你的整体复杂度从 $O(n)$ 退化到了 $O(n^2)$ 。

如果输入数组是 [1, 1, 1, ..., 1] (比如 10 万个 1)，虽然你有 `if (uSet.count(x-1)==0)` 的判断，但你仍然对这 10 万个重复的数字都执行了一次哈希查询 `count(x-1)`。

[illegible]

如果按照原来的方法遍历数组，会发生上面所提到的"对这 10万个重复的数字都执行了一次哈希查询 count(x-1)"的情况。

所以，更改也很简单：把下面遍历的主题从 `nums` 改成 `uSet` 就行：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> uSet;
        //1.给数字丢进去
        for (auto &x:nums) {
            uSet.emplace(x);
        }
        //2.开始遍历
        int maxLength = 0;
        for (auto &x:uSet) { //避免重复遍历过量元素，改为遍历保证无重复的集合
            if (uSet.count(x-1)==0) { //防止重复操作，保证这个数就一定是开头
                int cnt=0;
                while (uSet.count(x+cnt)) {
                    cnt++;
                }
                maxLength = max(maxLength,cnt);
            }
        }
        return maxLength;
    }
};
```

挺有意思，之前还没遇到过因为这种原因超时的情况。

5-11 盛最多水的容器

题目链接：<https://leetcode.cn/problems/container-with-most-water/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2025.12.18

标签：贪心 数组 双指针

推荐题解：<https://leetcode.cn/problems/container-with-most-water/solutions/94102/on-shuang-zhi-zhen-jie-fa-li-jie-zheng-que-xing-tu>

上面的题解可以看一下，正确性证明讲的非常好，简直是神级题解。所以下面我就不再赘述这个题目做法的正确性了，讲讲我自己的看法。

首先，为啥初始化选择左右指针分布在数组的两端？答案显而易见，就是初始保证它的长度(d)是最长的。

然后，如何移动左右指针呢？

假设左边的木板的高度($h[l]$)比右边的高度($h[r]$)低。此时，我们有两种选择：

1. 移动左边的指针： $l++$ 。由于不管移动左边还是右边，长度都是 $d-1$ ，所以面积的大小就由两边最短的木板决定了。并且呢，无论左边的指针怎么动，左边的木板高度变成咋样，根据木桶效应，决定面积的那个木板的高度绝对不会超过右边的木板。已知左边的高度就是比右边低，所以挪动 l ，木板的高度($h[l]$)可能会变大，也可能会变小。如果 $h[l]$ 变小，已知 $d \rightarrow d-1$ ，所以面积一定会变小；如果 $h[l]$ 变大，虽然 d 变小了，但是只要 $h[l]$ 变的足够大，大到抵消 d 的损失，面积还是有可能反而变大的。

总结：移动左边的指针，面积可能变大也可能变小，相当于搏一搏，找个机会。

2. 移动右边的指针： $r--$ 。这就非常不一样了。已知 $h[l] < h[r]$ ，如果把这个高的木板淘汰了，根据木桶效应，决定面积的那个木板的高度绝对不会超过左边的木板。是不是有点感觉了：这相当于强行把面积的上限压低了！

咱们讨论一下：如果 $h[r-1] > h[r]$ ，那么面积变小，因为 $h[l] < h[r]$ ，所以决定高度的是 $h[l]$ ，并且 d 变小了；如果 $h[r-1] < h[r]$ ，那么面积还是变小，原因显而易见了吧。

总结：移动右边的指针，面积只会变小或者不变，没有赢面。

总结：移动短板，是为了寻找面积变大的唯一可能性；放弃短板，是因为以它为边界的所有其他可能方案都已被证明更差。

上代码：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int maxArea(vector<int>& height) {
        int maxArea = 0, l = 0, r = height.size()-1;
        while (l < r) {
            maxArea = max(min(height[l], height[r]) * (r - l), maxArea);
            if (height[l] < height[r]) l++;
            else r--;
        }
        return maxArea;
    }
};
```

6-15 三数之和

题目链接: <https://leetcode.cn/problems/3sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.21

标签: 数组 双指针 排序

推荐题解: 官方题解不太行, 还是Gemini的做法更加简单明了

这题, 我的第一个思路并不是我现在的做法: 由于Hot100刚刚做完哈希表的题目(见 [1.两数之和](#)), 然后又看到这道题的提示: 把三指针转化成双指针的问题。

那不是把三数之和的问题转化成两数之和的问题, 然后再用上面这道题目的哈希表做法来做两数之和的问题, 把时间复杂度压到 $O(n^2)$ 。

想法很美好, 我代码也写出来了, 如下:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        int n = nums.size();
        //1.将全部数丢进哈希
        unordered_map<int,int> dict;
        unordered_set<string> filter;
        for (int i=0;i<n;i++) {
            dict[nums[i]]=i;
        }
        //2.开始遍历
        vector<vector<int>> ans;
        for (int x=0;x<n;x++) {
            int v1 = -nums[x]; //y+z的期望值
            for (int y=0;y<n;y++) {
                int v2 = v1-nums[y]; //z的期望值
                if (dict.count(v2)) {
                    int z = dict[v2];
                    if (z!=y && y!=x && z!=x) {
                        vector<int> temp = {nums[x],nums[y],nums[z]};
                        sort(temp.begin(), temp.end());
                        int nx = temp[0],ny = temp[1],nz = temp[2];
                        string str = to_string(nx)+"-"+to_string(ny)+"-
"+to_string(nz); //用字符串拼接来去重
                        if (!filter.count(str)) {
                            ans.emplace_back(vector<int>{nx,ny,nz});
                            filter.emplace(str);
                        }
                    }
                }
            }
        }
        return ans;
    }
};

```

时间复杂度妥妥的 $O(n^2)$ 啊！但是还是超时了：311 / 315，因为字符串拼接既费时间又费空间。

Confused，那我就去问了Gemini，它给的做法非常好理解，我来讲讲。

首先呢，做法有点类似于上一道题 [11.盛最多水的容器](#)。我们先把这个数组升序排序。这是为了：

1. 方便去重。在排序之后，相同的元素会排在一起，可以很方便的避免开头重复的情况。
2. 利用单调性。排序是我下面即将讲的操作的前提。

首先，提示没说错，我们确实需要将三数之和转换成两数之和。所以，最外层的循环，我们维护一个*i*，指向三个数的第一个数。

然后呢，后面两个数，我们采用类似于11题(上方有链接)的"收缩法"来完成：

维护两个指针，`l=i+1` 和 `r=n-1`，指向*i*指向的这个数的后面区域的数的头和尾，相当于最小和最大的数。

然后我们计算 `sum=nums[i]+nums[l]+nums[r]`。分类讨论：

1. 如果 `sum>0`，那说明选的数大了，我们需要选更小的。将大的数换掉，`r--`。
2. 如果 `sum<0`，那说明选的数小了，我们需要选更大的。将小的数换掉，`l++`。
3. 如果 `sum=0`，说明这三个数是正确答案，那就将它们丢进答案数组 `ans`。

然后还有一个很重要的剪枝优化，也是利用了排序之后数组的单调性：如果 `nums[i]>0`，直接 `break`。

因为如果这个数都大于0了，后面的待选择区域的数肯定全部都大于0，是绝对没有正确答案的，后面就没有继续下去的必要了。

到这里，题目的基本思路就差不多讲完了，但是程序现在还是不能跑出正确答案。

那么现在肯定有同学要问了：去重的部分在哪里？

答：在主指针*i*，左右指针收缩的过程中，我们都可以很方便的利用"重复的数都挤在一起"这个特性来去重。

当主指针*i*刚刚往后挪了一位，进入下一个循环时：如果 `i>0 && nums[i]==nums[i-1]`，那么说明一模一样的逻辑在前一个数已经进行过了，我们就得跳过这一次循环，防止重复。那么为啥不是和后面的数比呢？因为如果跟后面比（`nums[i] == nums[i+1]`），会把 `[-1, -1, 2]` 这种三元组内部允许重复元素的情况给错杀掉。

在左右指针收缩的过程中，如果我们找到了正确答案，在将答案丢进数组后，我们继续利用这个特性，开个条件为 `l<r && nums[l]==nums[l+1]` 的循环(右指针同理)，保证左右指针都不是已经处理过的数。

到这里，这道题目才算完整。我们可以发现，找三个加起来为0的数本身不难，难在如何优雅的去重。

这个做法用简单的"收缩"和"位移"，在解题目的同时还顺便把重给去了，十分优雅。

下面看代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        int n = nums.size();
        //1.给数组排序
        sort(nums.begin(), nums.end());
        //2.开始寻找三个数
        int l=0,r=0;
        vector<vector<int>> ans;
        for (int i=0;i<n;i++) {
            if (nums[i]>0) break;
            if (i>0 && nums[i]==nums[i-1]) continue;
            l = i+1;
            r = n-1;
            while (l<r) {
                int sum = nums[i]+nums[l]+nums[r];
                if (sum==0) {
                    ans.emplace_back(vector<int>{nums[i],nums[l],nums[r]});
                    while (l<r && nums[l]==nums[l+1]) l++;
                    while (l<r && nums[r]==nums[r-1]) r--;
                    l++;
                    r--;
                }
                if (sum>0) r--;
                if (sum<0) l++;
            }
        }
        return ans;
    }
};

```

那这道题到这里就讲完了。

7-3 无重复字符的最长子串

题目链接: <https://leetcode.cn/problems/longest-substring-without-repeating-characters/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.23

标签: 哈希表 字符串 滑动窗口

推荐题解: 无

惊不惊喜? 意不意外? 时隔4个月, 滑动窗口限时返厂!

PS: 如果你对滑动窗口感兴趣, 你可以去看我25年8月暑假写的 [滑动窗口算法笔记](#), 那时候我还在用Go刷题, 现在感觉是真不如C++一根。

由于时隔的时间太长, 我已经忘记了这种题型的滑窗的大多数细节, 并且一翻我的笔记, 我发现我做过这道题, 并且是用暴力枚举双指针的方式做的, 时间复杂度 $O(n^2)$ 。但是我的潜意识又告诉我, 这道题可以用 $O(n)$ 的方法解决, 一翻我自己的笔记, 还真让我翻到了:

第二种 条件移动左边界型 时间复杂度 $O(n)$

基本原理是这样的:

只遍历滑窗的右边界。向右扩大滑窗, 左边界一直不动, 同时一直做符合题目要求的某种统计(计数等)。如果这个滑窗满足条件, 就一直更新它的最大长度; 如果不满足条件了, 那就把左边界一直往右推, 缩小滑窗, 直到满足条件, 如此往复。

看代码:

```
maxLength := 0
l := 0 //左边界
for r := 0; r < len(nums); r++ { //遍历右边界
    //在这里处理逻辑
    for hash[nums[r]] > k { //一直移动左边界, 直到符合条件为止
        //处理逻辑
        l++
    }
}
```

这种方法的好处就是很快, 时间复杂度 $O(n)$, 坏处就是不一定所有题目都能用。

代码框架大致没啥问题, 但是在这道题我们不能直接照搬: 因为我初步打算使用 `unordered_set` 的自动去重功能来判断我后入窗的元素是否为重复元素, 如果还是先处理加加减减的逻辑再移动左指针, 会有问题——那么这个已经存在在集合中的元素, 如果还要强行`emplace`, 啥也不会发生, 相当于这个元素被吞掉了。

为了防止元素被吞, 我们需要 调换一下 上述框架循环和处理右指针加加的逻辑的位置。直接上代码, 有详细注释:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> hash; //初始化集合
        int l=0,maxAns=0; //初始化左指针、最大答案
        for (int r=0;r<s.size();r++) { //右移右指针
            while (hash.count(s[r])) { //如果右指针指向的当前元素在集合里面，说明再加就
                //不是无重复字符的子串了
                hash.erase(s[l]); //删掉左指针当前指向的元素
                l++; //右移左指针
            }
            hash.emplace(s[r]); //将右指针指向的当前元素加入集合中
            maxAns = max(maxAns,r-l+1); //维护最大答案
        }
        return maxAns;
    }
};

```

到这里，这题就讲解完毕了(顺带吐槽一下Typecho的编辑器，只要代码块下面没别的东西，我就没法在下面插入新内容。这也是我另外打这行字的主要原因)。

8-438 找到字符串中所有字母异位词

题目链接: <https://leetcode.cn/problems/find-all-anagrams-in-a-string/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.25

标签: 哈希表 字符串 滑动窗口

推荐题解: [灵神对这道题的题解](#)

我们继续滑动窗口。这道题也算是打破了我灵神滑动窗口题单的一个窟窿了，我在题单里面没遇到过这种类型的题目。

不过呢，“异位词”这个概念，我们在上面的 [第2题](#) 已经遇到过了，当时用的是排序的思维，于是在初做这道题时，我的第一反应就是去排序：维护一个定长滑窗，再 `substr()` 截取子串，最后直接使用 `sort()` 进行排序。但是一看题目的数据范围： $1 \leq s.length, p.length \leq 3 \times 10^4$ 。

我真求你了，排序的时间复杂度是 $O(n \log n)$ ，再叠加滑窗的 $O(n)$ 复杂度，总体时间复杂度来到了 $O(n^2 \log n)$ ，面对这样的数据规模还是太慢了。

所以，这题我们得放弃排序了。那么题目没说字母不重复，也就是说这个字符串中的字母是有可能重复的，我们该如何快捷方便的确认两个字符串是否是“字母异位词”呢？

我一开始是没想到的，看了一眼题解豁然开朗：统计每个字母出现的次数，如果每个字母出现的次数一模一样，说明就是异位词！

既然是计数，就有了两种选择：要么选 `unordered_map`，要么选 `vector`，要么退而求其次用原生数组(需要自己写比较的逻辑)。

我的代码选的是 `vector`，套用经典滑窗模板(左出右进，不会的去看我博客的滑窗算法笔记)即可：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> findAnagrams(string s, string p) {
        vector<int> ans;
        vector<int> cnt0
, key0
;
        for (auto& x:p) key[x-'a']++;
        //这里有个小坑点：如果不加int的强转，由于size()方法返回的是无符号整数，它不存在负数，假设
        //2-3，结果不是-1而是1
        //所以在p比s长的时候，要么直接开头特判返回空数组，要么就在这里强转，让循环不进入，更推
        //荐我现在的做法吧
        for (int l=0;l<=(int)s.size()-(int)p.size();l++) {
            if (l==0) {
                for (int i=0;i<p.size();i++) {
                    cnt[s[i]-'a']++;
                }
            }else {
                cnt[s[l-1]-'a']--;
                cnt[s[l+p.size()-1]-'a']++;
            }
            if (cnt==key) { //利用vector的比较操作符
                ans.emplace_back(l);
            }
        }
        return ans;
    }
};
```

当然，我也请教了gemini：如果把 `vector` 换成其他数据结构会发生啥？

1. 换成 `std::unordered_map<char, int>`

结论：性能会显著下降。

- 原因 1：哈希开销。 `unordered_map` 每次存取都需要计算 `hash(key)`。对于只有 26 个连续字符的情况，直接通过 `x - 'a'` 索引数组是 $O(1)$ 且常数极小的，而哈希表常数项很大。
- 原因 2：内存分布。 `unordered_map` 的桶 (buckets) 通常分散在堆内存中，对 CPU 缓存 (L1/L2 Cache) 不友好；而数组在内存中是连续的。
- 原因 3：比较复杂度。你代码中的 `if (cnt == key)` 对 `vector` 来说是线性遍历比较 (26 次迭代)。对 `unordered_map` 来说，`==` 操作符不仅要比较大小，还要逐个键查找值是否存在且相等，开销更大。

适用场景：只有当字符集非常大且稀疏 (比如万国码 Unicode，且只有少数几个字符出现) 时，才考虑用 `unordered_map`。

2. 换成原生数组 `int[26]`

结论：性能最强，但代码灵活性降低。

- 内存：原生数组分配在栈上，省去了 `vector` 在堆上动态申请空间的开销 (虽然本题只申请一次，影响较小)。
- 痛点：比较操作。在 C++ 中，原生数组不能直接用 `==` 比较 (`arr1 == arr2` 比较的是首地址指针)。
 - 你需要手动写一个循环比较 26 个位置。
 - 或者使用 `memcmp(cnt, key, sizeof(cnt))`。

3. 最优选： `std::array<int, 26>`

对于你的需求，如果你追求极致性能又想保留 `vector` 的方便，`std::array` 是完美选择。

```
std::array<int, 26> cnt = {0}, key = {0};  
// ... 逻辑不变 ...  
if (cnt == key) { // std::array 支持直接用 == 比较，且效率极高  
    ans.emplace_back(1);  
}
```

C++

- 优点：它封装了原生数组，分配在栈上，大小固定。
- 效率：它的 `operator==` 经过编译器优化，通常比 `vector` 的比较更快。

所以，灵神的题解用的是 `array`，今天又学到了一点，哈哈。

9-560 和为 K 的子数组

题目链接: <https://leetcode.cn/problems/subarray-sum-equals-k/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2025.12.29

标签: 数组 哈希表 前缀和

推荐题解: [灵神对这道题的题解](#)

开头先把一句结论放在这里: 这就是上面的"两数之和"的变种。

讲讲我的思考心路历程吧。一开始看到这道题目, 我的第一反应是用滑动窗口去做, 毕竟上面刚刚做了滑窗的题目。

但是疑问也随之而来: 如果用定长滑窗, 不就相当于暴力枚举, 时间复杂度是 $O(n^2)$; 如果用不定长滑窗...用不了吧? 毕竟数可能是负的, 并不保证一定增长。所以, 我打消了滑窗的念头。

我想啊想, 又想了5分钟, 没想出来。

Gemini曾经告诉我一个重要的道理: 我现在的的时间很宝贵, 我的重点应该放在会做这道题目上, 而不是一定要求自己做出来这道题目, 过程不重要, 重要的是结果。

所以我就去看题解了。题解的思路一开始我还没看懂, 继续丢给我这位AI老朋友, 它成功给我教会了, 来讲讲我的理解。

首先, 我们需要将一个区间和问题, 利用老朋友前缀和转换成两个端点的问题。然后呢, 从这里开始就和两数之和不太一样了: 在两数之和, 我们只需要找是否有和为k的两个数, 但是在这里, 我们需要返回数量。

这就和之前不定长滑窗的思想有些类似了, 我记得在滑窗的题单里我有接触过类似的题目: 遍历到后面之后, 看看前面有几个和自己匹配上的, 将这里的数目累加进答案里即可。

那么怎么计数呢? 这里就需要用到哈希表了, 我选择了 `unordered_map` 作为计数器。从前往后遍历数字, 看看我要找的和这个数字对应的那个数字是否在计数器里, 如果在就按照上面"匹配"的思路累加答案, 然后无论在不在, 都要把当前数字也累加进计数器, 以便后面的数与它匹配。

代码如下:

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        //1.先构建前缀和
        int n = nums.size();
        vector<int> prefixSum0
;
        for (int i=0;i<n;i++) {
            prefixSum[i+1]=prefixSum[i]+nums[i];
        }
        //2.开始数数
        unordered_map<int,int> cnt;
        int ans = 0;
        for (int i=0;i<=n;i++) {
            int target = prefixSum[i]-k;
            auto it = cnt.find(target);
            if (it!=cnt.end()) { //找到了
                ans+=it->second;
            }
            cnt[prefixSum[i]]++;
        }
        return ans;
    }
};

```

这里有一点要注意，就是我用到了 `find(key)` 方法。如果是 C++20，直接返回 `bool` 的 `.contains(key)` 方法更加方便(灵神题解用的就是这个方法)。

那么为何不直接用 `cnt[key]` 判定存在性呢？因为：

如果你调用 `cnt[key]`，而这个 `key` 不在 `map` 里，C++ 会自动创建一个该 `key` 的项，并赋予默认值（比如 0）。这会破坏哈希表的大小，甚至在逻辑判断上出错。而 `find` 不会修改哈希表。

`find` 的用法如下：

返回值：返回一个迭代器 (`iterator`)。

查找成功：迭代器指向该键值对。

查找失败：迭代器等于 `container.end()`。

现在知道为啥要这么用的吧？顺带一提，这可能是我2025年的最后一道题目了，因为我还要抽时间写动态规划的算法笔记，最近期末复习也挺忙的。那么下一题，我们2026见！

10-239 滑动窗口最大值

题目链接: <https://leetcode.cn/problems/sliding-window-maximum/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难(Hard)

解题日期: 2026.1.6

标签: 队列 数组 滑动窗口 单调队列 堆 (优先队列)

推荐题解: [灵神的题解, 但是我没按照他的来](#)

现在是26年1月6日晚上21点37分, 我在半小时前刚刚复习完概率论, 想着最近这么忙, 好几天没刷题了, 正好今天有空, 就来刷一下hot100吧。

然后, 按照惯例, 我是会把所有困难题留在最后做的。但是这道题目不一样, 我一点进来看题目, 滑窗+优先队列的思路就冒出来了。挺顺, 那就试着做做吧。

但是我的第一反应是怀疑: Hard题怎么可能让我一眼看穿呢? 于是我点开标签, 看了一眼——还真有堆!(甚至都没看提示)

然后我的第一版思路就冒出来了, 直接调用我在灵神的滑窗题单里面修炼的炉火纯青的"删左加右"滑窗模型: 把刚刚出窗户那个元素从 `priority_queue<int> pq` 里面删掉, 然后把右边新进窗户的那个元素丢进去。想法是美好的, 但是一问Gemini: 优先队列不允许这样做, 只能弹出堆顶元素。

我的热情瞬间消减了一半, 但是没有放弃。思索了一会, 我想到了一个绝佳的思路, 也就是我ac的代码的主要思路:

继续用 `pq`, 但是是这样的 `pq`: `priority_queue<pair<int,int>> pq`, 在插入数的时候, 把数放在 `first`, 把这个数的下标放在 `second`。然后呢, 左边出窗户的数我不管它, 就让它在里面, 然后正常把新进窗户的数丢进 `pq`。

然后在这里, 我就要体现下标的作用了: 开一个循环, 判断堆顶元素的下标范围是否在我这个窗户里面, 如果是, 就说明是答案, 我们正常取; 否则就弹走, 直到我堆顶元素的下标在窗户范围内。

这个思路挺妙的, 我们可以类比一下, 把不同的窗户比作不同的部门: 假设你是公司的老板, 前一段时间公司进行了综合能力大比拼, 每个人都被用相同的标准打了一个分。你现在只想要后端部门分数最高的那个人, 那你就把全部人丢进堆里面, 然后只需要盯着堆顶, 如果堆顶的不是后端的, 就弹走, 如果是, 那就说明这就是后端部门分数最高的了。这样为啥不会出错? 因为堆是自动把堆里面分数最高的放在堆顶, 由于一开始全部人都在里面, 而后端部门的当然也全都在里面。那我就一直取堆顶呗, 因为堆顶保证了堆顶元素一定是当前堆里面所有元素最大的, 所以我取出的第一个后端的人的分数是全体堆元素最大的, 那自然也能保证是后端部门分数最高的了。

总结一下, 这个思路就是看起来很乱, 但是实际上挺巧的。我也挺好奇的, 我为啥能突然想出来这个思路。

那么, 上代码!

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        priority_queue<pair<int,int>> pq;
        vector<int> ans;
        for (int l=0; l<=nums.size()-k;l++) {
            if (l==0) { //初始化窗户内的数, 便于后续"删左增右"
                for (int i=0;i<k;i++) {
                    pq.emplace(nums[i],i);
                }
                auto [value,_] = pq.top();
                ans.emplace_back(value);
            }else { // "删左增右"
                pq.emplace(nums[l+k-1],l+k-1); //增右
                int idx = pq.top().second; //取出下标
                while (!(idx>=l && idx<l+k)) { //如果下标不在范围内, 就弹出
                    pq.pop();
                    idx = pq.top().second;
                }
                //此时堆顶就是下标符合条件的最大值
                int value = pq.top().first;
                ans.emplace_back(value);
            }
        }
        return ans;
    }
};

```

当然，这题的正统做法还是得讲讲：单调队列。

直接看Gemini生成的吧，或者看上面的灵神题解版本也可以：

进阶： $O(n)$ 做法 —— 单调队列 (Monotonic Queue)

虽然优先队列（堆）的 $O(n\log n)$ 已经足够通过此题，但利用 单调队列 可以将时间复杂度进一步优化至极致的 $O(n)$ 。

1. 核心思想：删除“冗余”元素

在一个窗口内，如果 $i < j$ 且 $nums[i] \leq nums[j]$ ，那么只要 $nums[j]$ 还在窗口里， $nums[i]$ 就永远不可能成为最大值。

- $nums[j]$ 比 $nums[i]$ 更大。
 - $nums[j]$ 比 $nums[i]$ 在窗口里停留的时间更长（更晚过期）。
- 因此， $nums[i]$ 就是“冗余”的，可以直接丢弃。

2. 算法流程

使用 `std::deque`（双端队列）维护一个下标序列，并保持队列对应的元素值 严格单调递减：

1. 入队：当新元素 `nums[i]` 进入时，从队尾弹出所有小于等于它的元素（因为它们冗余了）。
2. 出队（维护窗口）：检查队头下标是否已超出窗口范围（即 `dq.front() == i - k`），若是则弹出。
3. 取最值：此时 队头 下标对应的元素即为当前窗口的最大值。
4. 代码实现

```
C++
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq; // 存储下标
    vector<int> ans;
    for (int i = 0; i < nums.size(); ++i) {
        // 1. 保证单调递减：剔除队尾所有比当前元素小的“冗余”者
        while (!dq.empty() && nums[dq.back()] <= nums[i]) {
            dq.pop_back();
        }
        dq.push_back(i);

        // 2. 保证窗口合法：弹出过期的队头
        if (dq.front() == i - k) {
            dq.pop_front();
        }

        // 3. 记录答案：窗口形成 (i >= k-1) 后开始收集
        if (i >= k - 1) {
            ans.push_back(nums[dq.front()]);
        }
    }
    return ans;
}
```

4. 复杂度对比

算法	时间复杂度	空间复杂度	备注
优先队列（延迟删除）	$O(n \log n)$	$O(n)$	简单直观，依赖堆的自动排序
单调队列	$O(n)$	$O(k)$	极致优化，每个元素仅入队/出队一次

好的，那就讲到这里。

11-76 最小覆盖子串

题目链接：<https://leetcode.cn/problems/minimum-window-substring/description/?envType=study-plan-v2&envId=top-100-likedd>

难度：困难(Hard)

解题日期：2026.1.7

标签：哈希表 字符串 滑动窗口

推荐题解：[灵神的题解](#)

这题的做题经历可谓是一波三折。我之前是做过这个题的，在我刷滑窗题单的时候，用Go过的。

结果，今天我用类似的思路，用cpp过的时候，直接给我干超出内存限制了？？？我是第一次见这个错误。

题目描述 通过 × 题解 提交记录				
所有状态 ▾	所有语言 ▾	执行用时	消耗内存	备注
6 通过 4 分钟前	C++	🕒 175 ms	💾 11.5 MB	
5 解答错误 7 分钟前	C++	🕒 N/A	💾 N/A	
4 超出时间限制 13 分钟前	C++	🕒 N/A	💾 N/A	
3 超出内存限制 15 分钟前	C++	🕒 N/A	💾 N/A	
2 通过 2025.07.30	Go	🕒 221 ms	💾 5 MB	
1 解答错误 2025.07.29	Go	🕒 N/A	💾 N/A	

[提交情况](#)

后面问了Gemini才破案：我代码里面的 `s.substr` 发力了。先讲讲我的思路吧，和我在上次第一次做这道题目的思路是一样的：

使用"右指针一直往右，直到符合条件，再将左指针往右，将窗口收缩到不符合条件为止，即得最短的符合长度子串"的滑窗模型，如果忘记了，可以去 [我的笔记](#) 看看。然后呢，在符合条件的时候，我就习惯性直接像Go,python的字符串切片一样，用 `s.substr(l,r-l+1)` 把这个字符串直接切进我准备 `return` 的 `ans` 字符串里面去了。由于这个操作是会复制子串的，在用例比较大的时候就会爆内存。

知道原因之后，解决方法也很简单：新起两个变量，用于存储最短符合条件状态下的子串的起点和终点下标，最后返回之前再看情况动用 `substr` 来切取子串。这样问题就解决了。看代码：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    bool isCovered(unordered_map<char,int> &cntT,unordered_map<char,int> &cntS)
    {
        for (auto [k,v]:cntT) {
            if (cntS.count(k)==0 || cntS[k]<v) return false;
        }
        return true;
    }
    string minWindow(string s, string t) {
        if (s.size()<t.size()) return "";
        //1.先统计t里面各个数的情况
        unordered_map<char,int> cntT,cntS;
        for (auto &x:t) {
            cntT[x]++;
        }
        //2.开始滑窗
        int l=0,minLength=s.size()+1,minL=-1,minR=s.size()+1;
        string ans="";
        for (int r=0;r<s.size();r++) {
            cntS[s[r]]++;
            while (isCovered(cntT,cntS)) {
                if (r-l+1<minLength) {
                    minL=l;
                    minR=r;
                    minLength = r-l+1;
                }
                cntS[s[l]]--;
                l++;
            }
        }
        if (minL==-1) return "";
        ans = s.substr(minL,minR-minL+1);
        return ans;
    }
};
```

当然，这个 `isCovered()` 函数是可以优化成 $O(1)$ 的，不过我懒得弄，5%极限过测就完事了，哈哈。就讲到这里。

12-56 合并区间

题目链接: <https://leetcode.cn/problems/merge-intervals/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.1.8

标签: 数组 排序

推荐题解: [灵神的题解](#)

这道题还是挺有意思的，让我找回了久违的"打补丁"感觉。

先讲讲我一开始的思路吧。首先，我想到了把区间按照左端点升序排序。这是我刷题这么久的感觉告诉我的。

然后后面，由于这是数组分类的题目，没有任何固定算法思想可循，我就开了个循环，开完循环之后一边写一边想：

1. 第一版思路：如果后面那个区间的左端点 $\leq$ 现在这个区间的右端点，就说明两个区间交叉在一起了，这时候，就直接取后面这个区间的右端点为即将加入答案集合的右端点。
2. 第二版思路：但是上面那版思路会有些问题：如果当前区间把后面那个区间包裹住了，这样做会使得区间缩小，导致后面全错。于是我又引入了 `max()` 函数。发现自己忘记处理没交叉的情况了，给没交叉的情况补上。
3. 第三版思路：但是上面那版思路会有些问题：如果当前区间把后面那个区间包裹住了，这样做会使得区间缩小，导致后面全错。于是我又引入了 `max()` 函数。

经过三次迭代，三次打补丁，我终于ac了！看代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        //1.排序
        sort(intervals.begin(), intervals.end(), {
            return a[0]<b[0];
        });
        //2.开始拼接
        int n = intervals.size(), tempL=0, tempR=0;
        vector<vector<int>> ans;
        for (int i=0; i<n; i++) {
            if (i<n-1 && intervals[i][1] ≥ intervals[i+1][0]) {
                tempL=intervals[i][0];
                tempR=intervals[i][1];
                while (i<n-1 && (intervals[i][1] ≥ intervals[i+1][0] ||
intervals[i+1][0] ≤ tempR)) {
                    tempR=max(tempR, intervals[i+1][1]);
                    i++;
                }
                ans.emplace_back(vector<int>{tempL, tempR});
            } else {
                ans.emplace_back(intervals[i]);
            }
        }
        return ans;
    }
};

```

我自己也感觉这样不是个办法，冥冥之中，我感觉这道题目肯定有更简单的做法。于是我问了Gemini，看了它的思路，我秒懂。

它一句话点醒了我：

你之所以觉得思路乱，是因为你的代码试图在遍历的过程中“向前看”（观察 `i+1`）并且手动维护了一个内部 `while` 循环来跨越多个重叠区间。这种写法逻辑分支多，容易在边界条件（比如最后一个元素、嵌套区间）上出错，所以你才需要通过“打补丁”来 AC。

其实，处理这类问题有一个更优雅、更通用的思路：“向后看”。

那么具体如何向后看呢？就是：正常往后遍历，把第一个区间丢进 `ans` 之后，盯着 `ans` 的最后一个区间看。

我们分两种情况处理：

1. 如果现在的这个区间的左端点比最后一个区间的右端点大，那说明一定接不上，直接给这个区间丢进去，让它成为 `ans` 的最后一个区间。
2. 如果现在这个区间的左端点比最后一个区间的右端点小，说明是可以接上的。至于如何更新最后一个区间的右端点？依旧取 `max(ans.back()[1], cur[1])`；。

上代码吧，比我之前的思路少了不知道多少。

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        //1.排序
        sort(intervals.begin(), intervals.end(), {
            return a[0]<b[0];
        });
        //2.开始拼接
        int n = intervals.size();
        vector<vector<int>> ans;
        for (auto &x:intervals) {
            if (ans.empty() || x[0]>ans.back()[1]) {
                ans.emplace_back(x);
            }else {
                ans.back()[1] = max(ans.back()[1],x[1]);
            }
        }
        return ans;
    }
};
```

真的挺神奇的，又学到了新知识。

玩一下抽象：其实人生也是一样，有的时候我们只顾着向前看，其实有时候向后看，吸取教训，会比向前看更有收获。（硬扯这块）

13-189 轮转数组

题目链接：<https://leetcode.cn/problems/rotate-array/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.1.9

标签：数组 数学 双指针

推荐题解：[灵神的题解](#)

这应该是我目前我做Hot100做到过最抽象的题目了。

这题如果使用额外空间，非常简单：先把轮换的次数k对着数组长度取个余，防止题目故意用超出长度的轮换次数坑你；然后再根据k，切片数组的后k个元素到新的数组，然后再删掉原数组的后k个元素，然后再将新数组和原数组直接拼接。

如果用Python，代码4行就搞定了：取余，切片并赋值，切片并赋值，拼接并赋值。就像这样：

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        k%=len(nums)
        s1=nums[len(nums)-k:]
        s2=nums[:len(nums)-k]
        nums[:]=s1+s2 #对，由于我好久没写Python了，这里需要加[:]才能真正修改nums，属于是
        被迫补齐Python知识了(
```

用cpp也同理，这里突然用python主要是因为python的切片算是我接触过的语言里面最无脑最直观的了。

那么如何不用额外空间来完成这道题呢？我没啥头绪，就打开标签看了一眼，当我看到那两个字的时候，我已经来不及了：双指针。

这我怎么会写？于是我直接打开了题解，看到了灵神的思路：负负得正。真的挺牛的。

大致思路是这样的：由于轮换k次就是给后k个数挪到前面，那我直接把整个数组倒过来，后k个数就占据了前k个位置。然后我再把位置是前k个数的那些数给倒回去，再把剩下的另一个子数组的数也倒回去，就完成了k次轮换。代码量少、思路简单的同时，不会占据任何额外空间(至少cpp的 `reverse()` 是这样的)。直接上代码：

C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k%=nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin()+k);
        reverse(nums.begin()+k, nums.end());
    }
};
```

这题真的挺神金的，大晚上逗我笑一下，哈哈。

14-238 除了自身以外数组的乘积

题目链接：<https://leetcode.cn/problems/product-of-array-except-self/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.1.18

标签：数组 前缀和

推荐题解：[灵神的题解](#)

别问我为啥9天没刷题——你知道我这9天是怎么过的吗？？？

11号上午毛概，13号晚上习概，14号下午c++，15号下午数电，然后后面20号还有概率论，21号离散数学，22号数据结构。也就是说，我刚刚从1+3连考的地狱中爬出来，刚刚喘息了2天，后面就要继续迎接3连考，才能放假。我***。

回归正题。这题说来也好笑，由于我急着回宿舍，没看到题目要求的 不要使用除法，直接上了欧美做法，倒是没使用额外空间，看代码：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        //1.先计算总乘积
        ll all=1;
        int zeroCnt=0;
        for (auto& x:nums) {
            if (x==0) {
                zeroCnt++;
            }else {
                all*=x;
            }
        }
        //2.再算目标数组
        vector<int> ans;
        for (auto& x:nums) {
            if (x==0) {
                if (zeroCnt<=1) ans.emplace_back(all);
                else ans.emplace_back(0);
            }else {
                if (zeroCnt>0) ans.emplace_back(0);
                else ans.emplace_back(all/x);
            }
        }
        return ans;
    }
};
```

具体做法就不解释了，没套任何算法模型，就是利用了"总乘积/当前元素"得到答案，这是题目禁止的做法。

这道题的正解也不难，其实就是左前缀乘积和+右后缀乘积和，将这两个前缀后缀乘积和预处理好，就可以O(1)时间复杂度获取到当前位置元素的前面元素的乘积和后面元素的乘积，将二者一乘就是答案，没使用除法。看代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        //1.先计算前后缀乘积数组
        int n=nums.size();
        vector<int> pre(n),suf(n);
        for (int i=0;i<n;i++) {
            if (i==0) {
                pre[0]=1;
            }else {
                pre[i]=pre[i-1]*nums[i-1];
            }
        }
        for (int i=n-1;i>=0;i--) {
            if (i==n-1) {
                suf[i]=1;
            }else {
                suf[i]=suf[i+1]*nums[i+1];
            }
        }
        //2.直接出结果
        vector<int> ans(n);
        for (int i=0;i<n;i++) {
            ans[i]=suf[i]*pre[i];
        }
        return ans;
    }
};

```

这题难度不高，我觉得如果去掉禁止用除法这个条件，难度可以改成简单。

15-41 缺失的第一个正数

题目链接：<https://leetcode.cn/problems/first-missing-positive/description/?envType=study-plan-v2&envId=top-100-liked>

难度：Hard(困难)

解题日期：2026.1.19

标签：数组 哈希表

推荐题解：[灵神的题解，先看，再配合我下面的补充就很清楚了](#)

今天也是继续欧美，直接来上手Hard题——不过这次的运气就没这么好了，不但没想出来，而且看题解也理解了半天，可算是弄懂了。

可以先看题目，再看上方的灵神题解，看完灵神题解之后，再往下看。

这道题目的思路，其实和"打补丁"特别像——我们先形成初步思路，然后再一步步完善：

1. 首先，我们会想着给这些元素"排座位"，也是下面所有解法的目的：那些不是正整数的数我们不管，我们需要做的，是 尝试把每个正整数都放到和下标相对应的位置。由于这道题目的 `nums` 是0base，所以我们需要尝试让 `nums[i] = i+1`。在我安排完所有数之后，再遍历一遍数组，发现的第一个"无家可归"的数，它所在的位置+1就是缺失的第一个正数。
2. 那么如何高效的把每个元素放到正确的位置上呢？并且还要求原地操作。有两种思路，都是遍历数组：第一种是直接暴力搜索"这个位置属于哪个数"，外层循环遍历位置，内存循环找数，时间复杂度妥妥的 $O(n^2)$ ，肯定不行；第二种是正常遍历，"如果这个数不属于这个位置，就把这个数送到它该去的地方"。那第二种方法是不是就不需要新开一层循环了呢？还是需要的，因为我们没法保证换过来的那个数就一定属于现在的这个位置。所以我们需要开个循环，把 换过来的、不属于这个位置的数 继续送到它该去的位置上。至于时间复杂度呢？由于我们每次"送数"都会让一个数去到它该去的地方，看似双重循环，实则我的内层循环 跑一次就能解决1个位置问题，总共有最多n个位置问题等待我解决，那么相当于还是 $O(n)$ 。
3. 那么我们需要好好思考一下这个循环的条件了。首先，我"送数"的前提，肯定是这个数和当前位置不匹配，那么就需要判断一下是否匹配。其次，我们可能会遇到越界的数，例如比1小，比n大的数，这种数是无家可归的，所以我们不需要动它，只需要等待后续可能有属于该位置的数把这个无家可归的数换走，让它继续"流浪"。初步的循环条件就是这样的：`nums[i] ≥ 1 && nums[i] ≤ n && nums[i] ≠ i+1`。
4. 但是这样有个问题：如果遇到重复的数怎么办？这也是灵神题解里面最难理解的"影分身"部分。举个例子吧，假设我当前位置 `i=1`，这里的数是3，而 `i=2` 的位置上的数也是3。如果按照上面的循环条件，它会把这两个一模一样的3交换一次，并且交换完之后，循环条件还是成立！这样就会陷入死循环。
5. 此时，就体现了我说的"打补丁"思想了。我们只需要判断：如果当前这个数不匹配该位置，并且它的目标位置已经有和它一模一样的数了，就说明触发了这种重复的情况，我们为了避免死循环，就只能退出循环，不交换。于是我们增加条件：`nums[i] ≠ nums[nums[i]-1]`。这就相当于打补丁，修复了一个死循环的bug。
6. 然后，按照这个逻辑，我们就能跑完对数组的第一遍遍历。至于第二遍的事情就很简单了，只需要找第一个不匹配的下标就行。如果全都匹配，说明这些数贴的很紧，没有空隙，那么缺失的第一个(也就是最小的)正数就是 `n+1`。
7. 至此，思路完毕。

讲的已经很清楚了，上代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    int firstMissingPositive(vector<int>& nums) {
        //1.第一次遍历
        int n = nums.size();
        for (int i=0;i<n;i++) {
            while (nums[i] ≥ 1 && nums[i] ≤ n && nums[i] ≠ i+1 &&
nums[i] ≠ nums[nums[i]-1]) {
                int j = nums[i]-1;
                swap(nums[i],nums[j]);
            }
        }
        //2.第二次遍历
        for (int i=0;i<n;i++) {
            if (nums[i] ≠ i+1) {
                return i+1;
            }
        }
        return n+1;
    }
};

```

然后呢，其实有一个优化的点：这个 `nums[i] ≠ i+1` 条件 其实是可以删掉的——它已经被后面那个用来检测是否重复的条件覆盖掉了。

我们推理一下：当 `nums[i] = i+1` 时，将 `nums[i] = i+1` 代入后者，可以得到 `nums[nums[i]-1] = nums[(i+1)-1] = nums[i]`，也就是前面如果成立，后面也成立。反之，前面如果不成立，那后面肯定也不成立。所以，从数学的角度，我们把前者吸收掉了。

好的，就写这么多，这道题目还是挺有挑战性的。

16-73 矩阵置零

题目链接：<https://leetcode.cn/problems/set-matrix-zeroes/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.1.26

标签：数组 哈希表 矩阵

推荐题解：[灵神的题解](#)

好多天不见！这几天我经历了概率论-离散数学-数据结构三连考，并且还跨越了1700km，才回到了家。今天我缓过来了，于是我又来做算法了。

回归正题。这道题要求我们使用原地算法来解决问题，我一开始还是比较茫然的(当然暴力的方法肯定是秒出，主要是巧方法没想出来)，一看提示，一下子就会了：将矩阵的第一行和第一列作为记录信息的地方，

记录它们各自元素所处的列/行是否有0。但是一开始我想偏了，以为是要把这个矩阵从0base改成1base(这样很自然空出来了一个行和一个列)，然后又难以下手(既不想复制，又不知道怎么直接优雅的加上)。于是，我就灵机一动，把这个第一行和第一列单独提出来了。代码如下：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        //1.设置flags
        int n = matrix.size(), m = matrix[0].size();
        vector<bool> xFlags(n, false), yFlags(m, false);
        //2.探测有0的列
        for (int i=0; i<n; i++) {
            for (int j=0; j<m; j++) {
                if (matrix[i][j]==0) {
                    xFlags[i]=true;
                    yFlags[j]=true;
                }
            }
        }
        //3.改0
        for (int i=0; i<n; i++) {
            for (int j=0; j<m; j++) {
                if (xFlags[i] || yFlags[j]) matrix[i][j]=0;
            }
        }
    }
};
```

然后，ac，很丝滑。一问AI，AI说我符合题目的原地要求，并且处于进阶的第二阶段(一个简单的改进方案是使用 $O(m + n)$ 的额外空间，但这仍然不是最好的解决方案。于是我就向它学习了一下如何只使用常量空间，这下看懂了：

1. 准备两个变量，用来记录第一行第一列本身是否有0。然后分别扫描第一行第一列，根据是否有0的实际情况来更改这两个变量的值。
2. 再和上面代码一样开始遍历这个矩阵(从(1,1)开始)，如果遇到0，就给第一行和第一列的对应位置置0。
3. 最后也和上面一样，但是还是从(1,1)开始处理，如果第一行或者第一列的对应位置是0，就给当前元素置零。
4. 结束之后，还得单独处理第一行和第一列。根据变量的值判断是否需要单独给它们本身置零，然后操作。
5. 最后，为何需要单独准备两个变量呢？就举个极端例子吧。假设第一行全都是1，然后某一列的元素有0，就会把第一行的对应位置置0，这时候，我们就没法判断这个0是第一行本来就有的，还是被下面的0给改变出来的，就没法判断是否要给第一行本身清零。有了这两个变量之后，我们就只看这两个变量决定要不要清零，这样结果才是准确的。

上代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;

class Solution {
public:
    void setZeroes(vector<vector<int>>& matrix) {
        //1.探测第一行第一列是否有0
        bool xZero=false,yZero=false;
        int n = matrix.size(),m = matrix[0].size();
        for (int i=0;i<m;i++) {
            if (matrix[0][i]==0) {
                xZero = true;
                break;
            }
        }
        for (int i=0;i<n;i++) {
            if (matrix[i][0]==0) {
                yZero=true;
                break;
            }
        }
        //2.开始检测
        for (int i=1;i<n;i++) {
            for (int j=1;j<m;j++) {
                if (matrix[i][j]==0) {
                    matrix[i][0]=0;
                    matrix[0][j]=0;
                }
            }
        }
        //3.开始置零
        for (int i=1;i<n;i++) {
            for (int j=1;j<m;j++) {
                if (matrix[0][j]==0 || matrix[i][0]==0) matrix[i][j]=0;
            }
        }
        //4.最后处理第一行列
        if (xZero) {
            for (int i=0;i<m;i++) {
                matrix[0][i]=0;
            }
        }
        if (yZero) {
            for (int i=0;i<n;i++) {
                matrix[i][0]=0;
            }
        }
    }
};

```

就讲到这里。

17-54 螺旋矩阵

题目链接: <https://leetcode.cn/problems/spiral-matrix/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.1.26

标签: 数组 矩阵 模拟

推荐题解: [灵神的题解](#)

这道题目也算比较经典的模拟题目了。方法百花齐放，基本上都是找规律。

我找的规律是这样的：观察到螺旋矩阵输出时的行进方向为 右→下→左→上 的循环，于是就新建了两个分别负责x和y的方向数组，并且用一个指针来循环选择这四个方向。换方向的条件是这样的：如果按照当前方向行进，下标越界或者前方元素已经访问过，就说明我们需要转变方向了，就将这个指针循环+1，然后再判断一下+1后是否符合条件，如果按照新方向行进还是不符合条件，那就说明进入了死胡同，即全部元素都遍历完了。

上代码：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        vector<int> dx = {0,1,0,-1};
        vector<int> dy = {1,0,-1,0};
        vector<int> ans;
        int p=0,x=0,y=0;
        int n = matrix.size(),m = matrix[0].size();
        vector<vector<bool>> visited(n,vector<bool>(m,false));
        while (true) {
            ans.emplace_back(matrix[x][y]);
            visited[x][y]=true;
            if (x+dx[p]<0 || x+dx[p]≥n || y+dy[p]<0 || y+dy[p]≥m ||
visited[x+dx[p]][y+dy[p]]) {
                p=(p+1)%4;
                if (x+dx[p]<0 || x+dx[p]≥n || y+dy[p]<0 || y+dy[p]≥m ||
visited[x+dx[p]][y+dy[p]]) break;
            }
            x+=dx[p];
            y+=dy[p];
        }
        return ans;
    }
};
```

当然，我的代码还是有优化空间的，可以将额外空间的这个visited矩阵改成边界的收缩，亦或是直接在原矩阵上标记，都可以将空间复杂度降低到O(1)。由于比较简单，就不再赘述。

18-48 旋转图像

题目链接: <https://leetcode.cn/problems/rotate-image/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.1.27

标签: 数组 数学 矩阵

推荐题解: [灵神的题解](#)

这道题就比较有意思了，同样也是找规律。

我们观察示例里面元素的位置变化，假设每个元素的坐标为(i,j)，很容易发现每个元素从(i,j)变到了(j,n-1-i)。

这里我是偷懒了，所以直接写了我发现的结论。如果难以发现规律，可以先看x轴坐标变化，再看y轴坐标变化，就和上方题解一样，慢慢来，会好一些。

知道了每个坐标的变化规律之后，就好办了...吗？我们需要原地变换数组，如果新开一个数组当然简单，无脑一直按照上面的规律取就行了。我们只能原地变换，注定我们只能交换两个元素(使用 `std::swap()`)，而直接交换是行不通的。

这时候就来到了这道题的核心：将上面的变化规律拆成两步 边界分明 的变化，变成 $(i,j) \rightarrow (j,i) \rightarrow (j,n-1-i)$ 。

为啥要边界分明呢？我们需要保证我们的交换操作 在每一对元素身上只会被进行一次，不会出现被换过去的元素被换回来的情况。

你看第一步，属于矩阵的斜对称，边界很明确，就是(i,i)这条斜线，这样我们在操作的时候，就能用循环很好的限定操作范围为矩阵的上三角。

第二步就更简单了，属于矩阵的左右对称(x轴不变，只变y轴)，边界也很分明，是矩阵的中线。不过找这根线还是有难度，我列了好几个式子才发现它是 `matrix.size()/2-1`。

至此，思路毕。上代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        // (i,j) → (j,i) → (j,n-1-i)
        int m = matrix.size();
        // 1. (i,j) → (j,i)
        for (int i=0; i<m; i++) {
            for (int j=i; j<m; j++) {
                swap(matrix[i][j], matrix[j][i]);
            }
        }
        // 2. (j,i) → (j,n-1-i)
        for (int i=0; i<m; i++) {
            for (int j=0; j<=m/2-1; j++) {
                swap(matrix[i][j], matrix[i][m-1-j]);
            }
        }
    }
};

```

就讲到这里。

19-240 搜索二维矩阵 II

题目链接: <https://leetcode.cn/problems/search-a-2d-matrix-ii/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.1.27

标签: 数组 二分查找 分治 矩阵

推荐题解: [灵神的题解](#)

这题就和前面的题目有些不一样了, 单纯找规律可能很难找到解法。我甚至看了两遍题解才真正理解这题做法的关键所在。

看完第一遍题解, 我领悟到了"收缩边界"和"分治"两个概念, 在代码的开头定义了四个变量来圈定边界, 甚至还在思考要不要用递归、边界条件怎么判断, 如何退出...挺麻烦的。

于是我看了第二遍题解, 才抓到这个做法的核心: 只盯着右上角看, 然后只收缩x轴的下界和y轴的上界。且听我娓娓道来。

首先, 这个矩阵具有这样的特性:

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

这就说明，如果某一行第一个元素比我的目标元素大，那这一行一定没有我的目标元素，可以直接排除；同理，如果某一行的最后一个元素比我的目标元素小，那这一行也可以直接排除。

在这个做法里面，我们只盯着右上角的元素。按照上面的说法，如果这个元素比我的目标元素大，那就把右上角元素所在这一列排掉，y轴上界(数值高的那一端)减一，相当于往左边收缩一格；如果和目标相等，那就是找到了；如果比我的目标元素小，那就把右上角元素所在这一行排掉，x轴下界(数值低的那一端)加一，相当于往下面收缩一格。如此往复，直到右上角元素就是目标元素。如果边界收缩的越界了还没找到，那说明没找到，退出循环。

代码如下：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        int n=matrix.size(),m=matrix[0].size(),xl=0,xh=n-1,yl=0,yh=m-1;
        while (xh>=xl && yh>=yl) {
            int upRight = matrix[xl][yh];
            if (upRight>target) {
                yh--;
            }else if (upRight==target) {
                return true;
            }else {
                xl++;
            }
        }
        return false;
    }
};
```

顺带一提，为了方便寻找右上角的坐标，我的边界是设置为包含其本身的，也就是闭区间。

这个只盯着右上角的做法还是很巧妙的，把我看完题解第一遍的那些麻烦的边界判断想法全都免去了。

20-160 相交链表

题目链接：<https://leetcode.cn/problems/search-a-2d-matrix-ii/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.1.28

标签：哈希表 链表 双指针

推荐题解：[灵神的题解](#)

既然这题是简单题，那我们长话短说。

题解都看完了吧？首先，我想到的是 [官方题解](#) 里面的第一种方法：哈希法，就是把链表A每个节点的指针都丢进无序集合里面，在遍历链表B的时候不断检查当前节点是否在这个集合里面(时间复杂度 $O(1)$)，第一个被检查出存在的节点就是两个链表相交的地方。

顺带一提，cpp的指针之间的比较 比的是地址，和节点的值没有关系。这也是这个做法能不受题目中数字干扰的重要原因。

代码如下：

```
C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

struct ListNode {
    int val;
    ListNode *next;

    ListNode(int x) : val(x), next(NULL) {}
};

class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        unordered_set<ListNode*> hash;
        ListNode* p = headA;
        while (p) {
            hash.emplace(p);
            p=p->next;
        }
        p=headB;
        while (p) {
            if (hash.count(p)) {
                return p;
            }
            p=p->next;
        }
        return nullptr;
    }
};
```

但是这个方法还是需要 $O(n)$ 的额外空间，太吃内存了。那么我们该如何把它优化到 $O(1)$ 的额外空间呢？答案就是双指针。也就是官方题解的方法2和灵神题解的方法。

这个方法其实挺浪漫的，就是：如果你走过我走过的路，我走过你走过的路，我们终会相遇。

不扯浪漫，数学原理其实也简单：

1. 指针 **pA** 从 **headA** 开始走，指针 **pB** 从 **headB** 开始走。
2. 当 **pA** 走到 **headA** 的末尾时，让它转场到 **headB** 的开头继续走。
3. 当 **pB** 走到 **headB** 的末尾时，让它转场到 **headA** 的开头继续走。
4. 奇迹时刻：两个指针会在相交节点处相遇。

为什么？

假设 A 链表非相交部分长 a，B 链表非相交部分长 b，相交部分长 c。

- 指针 pA 走的距离：a + c (走完 A) + b (开始走 B 的非相交部分)
- 指针 pB 走的距离：b + c (走完 B) + a (开始走 A 的非相交部分)
- 结果：a + c + b = b + c + a。两者走的长度相等，刚好在相交点碰头！

当然，写的时候要注意，ab需要同时移动，就是 你走一步我也走一步，即放在同个循环里面。

代码如下：

```
class Solution {  
public:  
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {  
        ListNode *pA,*pB;  
        pA=headA;pB=headB;  
        while (pA!=pB) {  
            if (pA) pA=pA->next;  
            else pA=headB;  
            if (pB) pB=pB->next;  
            else pB=headA;  
        }  
        return pA;  
    }  
};
```

就讲到这里。

21-206 反转链表

题目链接：<https://leetcode.cn/problems/reverse-linked-list/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.1.28

标签：递归 链表

推荐题解：[灵神的题解](#)

好一道数据结构题目。

长话短说吧，其实这道题就是链表的头插法。只不过并没有创建新链表，而是把这个节点本身给十分巧妙的原地使用了头插法，从而实现O(1)额外空间的链表反转。

直接上代码，上完代码之后再讲：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode *cur = head, *pre = nullptr;
        while (cur) {
            auto nxt = cur->next; // 是为了存储丢失的cur后继节点信息
            cur->next = pre;
            pre = cur;
            cur = nxt;
        }
        return pre;
    }
};

```

头插法，顾名思义，我们需要把新元素给插在头节点的前面，并且新元素本身成为头节点。因此，如果将一个序列按照从左到右的顺序，使用头插法插入链表当中，自然而然就实现了反转的效果。

在这道题的题解代码中，我们使用了pre作为前驱节点(也算是反转后链表的后继节点)，cur作为当前节点，我们的目标就是，既保证cur能够沿着当前链表的方向往后走，又能让它的前驱节点成为它的后继。这时候我们就需要一个中间变量来暂时存储cur的后继，以确保cur在指向pre之后，还能继续往后走。

在往后走之前，由于当前节点到了下一个节点就变成了前驱节点，因此需要用pre存储一下当前节点，然后cur再往后走。

大体思路就是这样，虽说是头插法，但是原地头插反转链表的算法我还是第一次见，挺新奇的。这题还有个做法是递归法，是灵神题解里面的第一种方法，我这边就不讲了，感兴趣的同学自己去探索吧。

22-234 回文链表

题目链接：<https://leetcode.cn/problems/palindrome-linked-list/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.1.29

标签：栈 递归 链表 双指针

推荐题解：[灵神的题解](#)

这道题还是比较有意思的，从灵神的题解看来，我们需要做两道 前置题目：反转链表和链表的中点。前者就是上一道题，后者我还确实确实没做过。

当然，按照我一贯欧美的作风，我肯定是不去做那道题的。于是我直接看了一眼代码，好像也不是很难。那就直接讲吧。

首先讲讲前置知识：如何找到链表的中点？总体思路很简单：使用 快慢指针。快指针一次往后两格，慢指针一次往后一格，当快指针无法继续往后两格时(要么指向最后一个节点，要么指向null)，慢指针指向的位置就是链表的中点(如果是奇数长度，那就是正中央；如果是偶数长度，那就是中线偏右的那个节点)。

这道题为啥需要用到上面两道前置题目呢？因为我们需要先找到链表的中点，然后再将链表的后半段原地反转，反转完之后，就相当于一个链表先被劈成两半，然后被反转之后，后一半的尾巴和前一半的尾巴可能重叠(链表长度为奇数)，也可能各自分离(链表长度为偶数)。

然后呢，我们就可以从这两个新链表的头部开始，一步一步往后同时遍历，一边遍历一边比较，若有不相等就直接返回 `false`，两条链表都遍历完了就返回 `true`。

思路讲完了，上代码：

```
C++
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        //1.先找链表的中点
        ListNode *slow=head,*fast=head;
        while (fast && fast->next) {
            slow=slow->next;
            fast=fast->next->next;
        }
        //2.反转后半段
        ListNode *pre=nullptr,*cur=slow;
        while (cur) {
            ListNode *nxt = cur->next;
            cur->next = pre;
            pre=cur;
            cur=nxt;
        }
        //3.开始比较
        ListNode *p1=head,*p2=pre; //这里需要注意，后半部分反转后的链表的头结点是pre，不是cur，cur已经指向null了
        while (p1 && p2) {
            if (p1->val!=p2->val) return false;
            p1=p1->next;
            p2=p2->next;
        }
        return true;
    }
};
```

其实完全可以像灵神题解里面一样，把上面两道前置知识的题目封装在函数里面，然后直接调用获取后半反转后链表的头结点即可。

23-141 环形链表

题目链接: <https://leetcode.cn/problems/linked-list-cycle/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.1.29

标签: 哈希表 链表 双指针

推荐题解: [灵神的题解](#)

这道题的做法也是挺妙的: 利用了 操场上跑的慢的人一定会被跑的快的人"套圈" 的原理。

那么我们就很自然的联想到快慢指针用快慢指针来做这道题。

当然, 看到题目第一眼, 我是没想出来要这样做的。瞟了一眼题解才突然恍然大悟。

"套圈"的概念, 具象化到这道题, 其实也简单: 如果快指针到终点了也没和慢指针相等过, 说明无环; 由于只要不到终点, 就会一直走, 所以如果快指针和慢指针在某一时刻相等了, 就说明是环, 直接返回true。

上代码:

```
C++
class Solution {
public:
    bool hasCycle(ListNode *head) {
        ListNode *slow=head,*fast=head;
        while (fast && fast->next) {
            slow=slow->next;
            fast=fast->next->next;
            if (fast==slow) return true;
        }
        return false;
    }
};
```

就讲到这里。

24-142 环形链表 II

题目链接: <https://leetcode.cn/problems/linked-list-cycle-ii/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.1.30

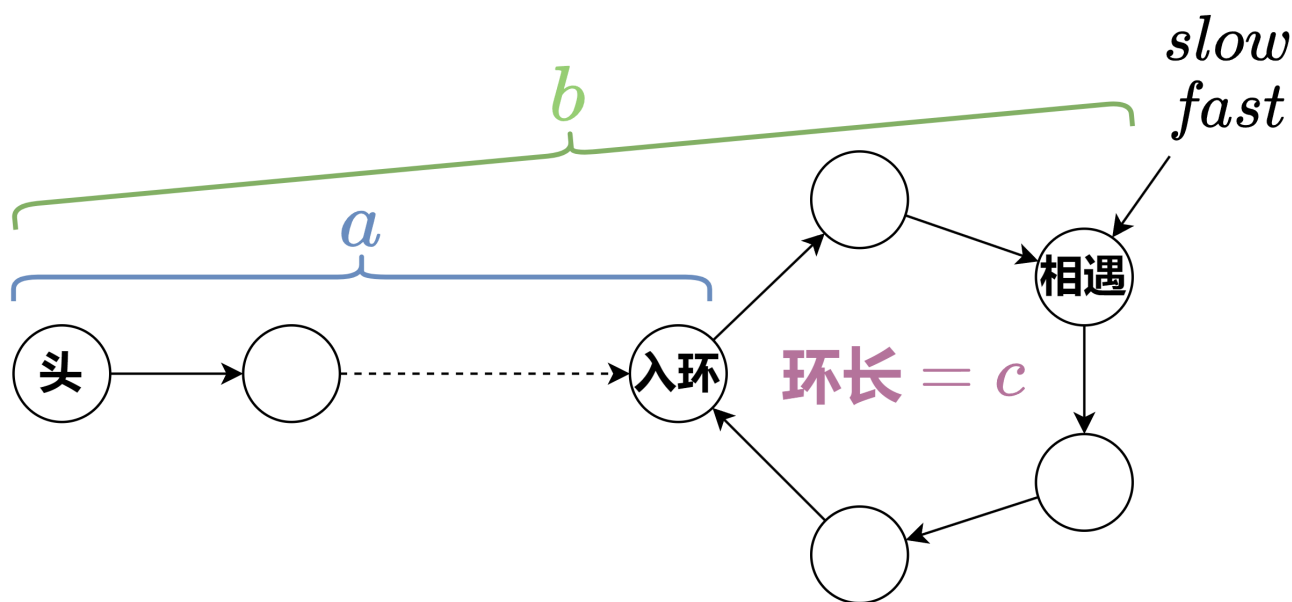
标签: 哈希表 链表 双指针

推荐题解: [灵神的题解](#)

从题号也可以看出来, 上面这道题就是这道题的前置题目。也是, 毕竟我们得先判断题目给的链表是不是环, 才能进行下一步。

那么在套用上面这道题的方法，判断完这个链表是不是环之后，我们下面该怎么找链表入口呢？并且只用 $O(1)$ 的额外空间。

一开始我也想不出来，看了题解还是理解了大半天，也终于是理解了。其实还是 快慢指针。



设头节点到入环口需要走 a 步。设环长为 c 。
设相遇的时候，慢指针走了 b 步，那么快指针走了 $2b$ 步。
设快指针比慢指针多走了 k 圈，即 $2b - b = kc$ ，得 $b = kc$ 。

慢指针从入环口开始，在环中走了 $b - a = kc - a$ 步到达相遇点。
这说明从相遇点开始，再走 a 步，就恰好走到入环口了！
虽然不知道 a 是多少，但如果让头节点和慢指针同时走，恰好 a 步后二者必定相遇，且相遇点就在入环口。

图解

从灵神的题解偷一张图过来。

题解文字描述的上半段没啥问题，很好理解，我相信大多数同学(包括我自己)的理解问题都主要出现在下半段文字吧？那我换个方式讲一下。

首先，由于快指针的速度是慢指针的2倍，当他们相遇时，快指针走过的总距离一定是慢指针的2倍，因此前者 $2b$ 后者 b ，这很好理解。

然后，由于是套圈，于是快指针比慢指针多走的那部分距离(即 $2b - b = b$) 一定一定一定是这个环长度的整数倍。

因此，这里我们可以获得一个很重要的结论：因为 b 是环长度的整数倍，所以，从这个环上某一点开始，往后走 b 步，一定会回到原地。

再回到上面这张图，是不是回想起来了？ b 同时也是链表起点到相遇点的距离！

然后呢，由于我们要求的环的入口距离起点的距离是 a ，并且我们还得直接返回这个节点，于是我们就要想办法让这两个指针一起指向这个节点。

这如何做到呢？我们可以把快慢指针其一(这里我选择慢指针)丢回起点，然后把快指针留在相遇点，然后让它们继续往后以相同的速度走。这时候会发生什么呢？先看慢指针这边，相较于起点走了 a 步，会到入环的节点；再看快指针这边，它站在相遇点，如果它也再走 a 步，再加上它之前走了 b 步，那么它相较于起点走过的总路程就是 $b+a$ 步。还记得 b 是环长度的整数倍吗？这就说明，相较于起点走了 $b+a$ 步就和走了 a 步的效果是一模一样的。换句话说，走 a 步之后，快指针也会到达入环节点！

这时候，我们直接挑选快慢指针其一返回即可。这道题的逻辑确实挺绕的。

```
C++
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        //1.先判断是否为环
        ListNode *slow=head,*fast=head;
        bool flag=false;
        while (fast && fast->next) {
            slow=slow->next;
            fast=fast->next->next;
            if (slow==fast) {
                flag=true;
                break;
            }
        }
        if (!flag) return nullptr;
        //2.把slow放回起点，各自一步一步走，再次汇聚即为入环点
        slow=head;
        while (slow!=fast) {
            slow=slow->next;
            fast=fast->next;
        }
        return slow;
    }
};
```

就讲到这里。

25-21 合并两个有序链表

题目链接：<https://leetcode.cn/problems/merge-two-sorted-lists/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.1.30

标签：哈希表 链表 双指针

推荐题解：无

这道题也是一道经典的数据结构题目，总体思路很简单，和合并两个升序数组一样：维护两个指针，都指向两条链表的头部，每次都选小的那个插在新链表的头部。

对，基本思路就这么多，然后我去实现了，欧美打法，自信满满的写了一坨史山代码：

```

class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode *p1=list1,*p2=list2,*list3 = nullptr;
        //1.选择头结点
        if (p1 && p2) {
            if (p1->val<p2->val) {
                list3=p1;
                p1=p1->next;
            }else {
                list3=p2;
                p2=p2->next;
            }
        }else {
            if (p1) {
                list3=p1;
                p1=p1->next;
            }else if (p2){
                list3=p2;
                p2=p2->next;
            }else {
                return nullptr;
            }
        }
        ListNode *p3=list3;
        //2.开始两头工作
        while (p1 || p2) {
            if (p1 && p2) {
                if (p1->val<p2->val) {
                    p3->next=p1;
                    p1=p1->next;
                }else {
                    p3->next=p2;
                    p2=p2->next;
                }
            }else if (p1) {
                p3->next=p1;
                p1=p1->next;
            }else if (p2) {
                p3->next=p2;
                p2=p2->next;
            }
            p3=p3->next;
        }
        //3.收尾与返回
        p3->next=nullptr;
        return list3;
    }
};

```

基本思路就是写了一堆复杂逻辑，先确定头结点，然后再用差不多的逻辑把小的优先接在链表头，最后再收尾返回头结点。

但是其实根本不需要这么麻烦。

我们引入哨兵的概念：新定义一个空白节点作为头结点，它的next指向null，这样就可以跳过开头的选头结点的过程了，直接进循环开插。

循环里面的逻辑也可以大大简化：我们只处理两个链表都没遍历完的情况，正常比较取小插入即可。

当其中一个链表跑完之后，再把剩下没跑完的链表直接接在新链表的尾部，不需要再重新遍历了，要学会利用链表的特性。

基本思路就是这样，上代码：

```
C++
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode dummy(0);
        ListNode *tail = &dummy;
        while (list1 && list2) {
            if (list1->val < list2->val) {
                tail->next = list1;
                list1 = list1->next;
            } else {
                tail->next = list2;
                list2 = list2->next;
            }
            tail = tail->next;
        }
        if (list1) {
            tail->next = list1;
        } else {
            tail->next = list2;
        }
        return dummy.next;
    }
};
```

就讲到这里。

26-2 两数相加

题目链接：<https://leetcode.cn/problems/add-two-numbers/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.1.31

标签：递归 链表 数学

推荐题解：无

这道题还是非常有意思的。我根本就没机会写史山代码，因为史山代码过不了测评。

这道题目还算比较贴心，把数字是按照倒序的形式排的。毕竟加法是从低位加到高位嘛。

总体的思路不难，就是两位相加，然后取十位(可能是0或者1)为进位，留给下一位用；取各位为这一位的值。然后一步步往下。

然后呢，如果加到最后一位了，还是有进位，就需要往后再拓展一位1。

总体的思路到这里就讲完了。那么为何这道题是中等难度呢？就难在各种情况的处理上。

首先，如果两个链表的长度不一样，又如何处理？我的第一反应是在短的后面补0，把它补的和长的那个一样长。

但是其实有更简单的做法：两个链表指针一起往后，哪个指向null了哪个就不加，只加有效节点的值和上一位的进位值。至于循环进入的条件，只需要满足链表1不为null，链表2不为null，进位不为0三个条件的其一就能进入。将进位写在这里顺便还优雅解决了需要多加一位的问题。

理一下思路，是这样的：

1. 用一个变量 `carry` 记录进位。
2. 只要 `l1` 没走完、或者 `l2` 没走完、或者还有进位 `carry` 没加完，循环就继续。
3. 计算总和： `sum = (l1的值) + (l2的值) + carry` 。
4. 更新进位： `carry = sum / 10` 。
5. 创建节点： `val = sum % 10` 。

讲的很清楚了，上代码：

```

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
ll MOD = 1000000007;

struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode dummy; //哨兵节点
        ListNode *p3 = &dummy;
        int carry=0, sum=0;
        while (l1 || l2 || carry) {
            sum=carry;
            if (l1) sum+=l1->val;
            if (l2) sum+=l2->val;
            ListNode *next = new ListNode(sum%10);
            p3->next=next;
            p3=next;
            carry=sum/10;
            if (l1) l1=l1->next;
            if (l2) l2=l2->next;
        }
        return dummy.next; //返回哨兵节点后继
    }
};

```

值得一提的是代码开头我还用到了哨兵节点，这样又免去了抉择头结点的问题：直接无脑next就行。

27-19 删除链表的倒数第 N 个结点

题目链接：<https://leetcode.cn/problems/remove-nth-node-from-end-of-list/description/?envType=study-plan-v2&envId=top-100-linked>

难度：中等

解题日期：2026.1.31

标签：链表 双指针

推荐题解：无

这道题的做法，我没话说，是真的很巧。虽然说我一开始没想到。

直入主题。这题的做法，有点类似于快慢指针，只不过在这里应该算是延迟指针。就是让快指针先走n步，然后快慢指针开始同步移动，快指针指向null后，慢指针就正好指向准备删除的节点。这里其实有点像滑动窗口做边界处理的时候。

当然，删除节点，我们最好能停在待删除节点的前驱。我们可以小改一下同步移动两个指针的循环的退出条件：让快指针指向最后一个节点的时候就退出。

当然，既然是删除节点，我们确实也需要特判一下如果要删除的是头结点的情况。这种情况下，在先进n步的时候，快指针就已经会指向null了。因此我们只需要特判一下：快指针如果是null，直接返回 `head->next` 就行。

上代码：

```
C++
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *fast=head,*slow=head;
        for (int i=0;i<n && fast;i++) { //快结点先走n步
            fast=fast->next;
        }
        if (!fast) return head->next; //如果删除的是头结点
        while (fast && fast->next) { //这样可以let slow停在待删除节点的前一个
            slow=slow->next;
            fast=fast->next;
        }
        slow->next=slow->next->next;
        return head;
    }
};
```

就讲到这里。

28-24 两两交换链表中的节点

题目链接：<https://leetcode.cn/problems/swap-nodes-in-pairs/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.1

标签：链表 递归

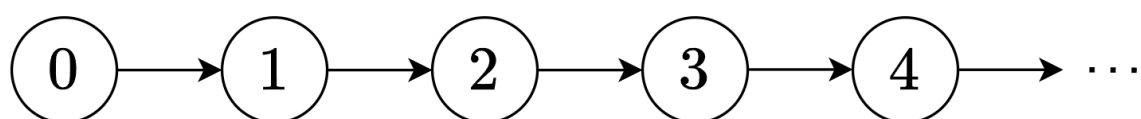
推荐题解：[灵神的题解](#)

这几天比较忙，来回于老家和家之间，在我更完今天的两道题之后，我马上又要坐20多分钟的高铁回老家去了(来回的票都是积分兑换的嘿嘿)。言归正传，我们直接开始二月的第一道题目。

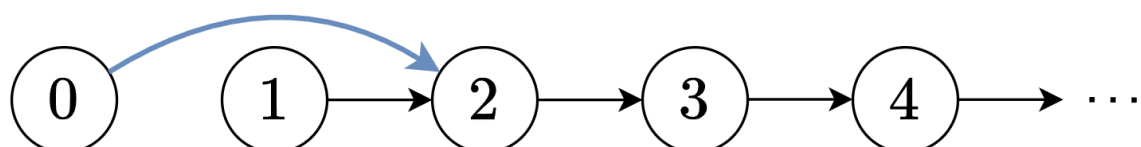
首先，我们先明确，这道题目依然需要使用哨兵节点。这样可以避免刚开始第一步的分类讨论，使得我们需要处理的全部情况都是这一段在中间的情况。

依旧先偷一张图过来：

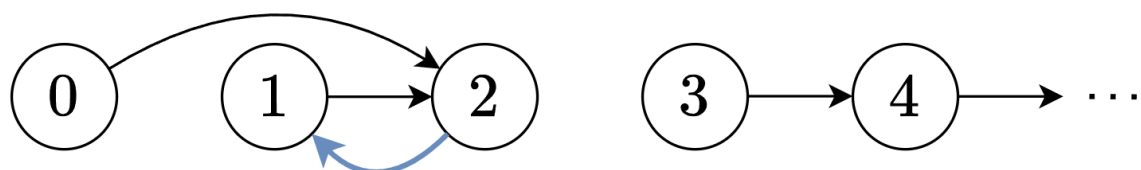
假设链表前面还有一个 0 号节点，
现在要交换 1 和 2。



先把 0 指向 2。

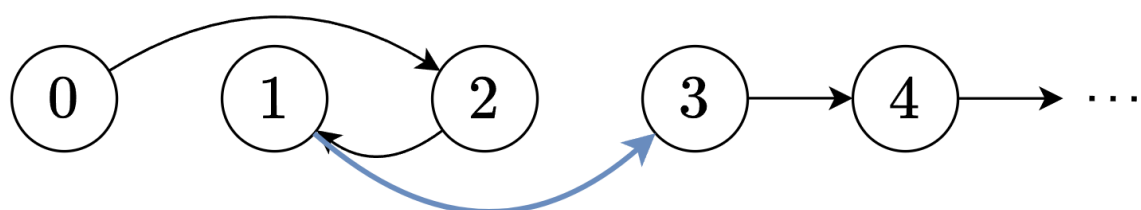


然后把 2 指向 1。



最后把 1 指向 3。

注意：对于下一轮交换，1 就是 3 的上一个节点。



算法：

创建哨兵节点 *dummy*，表示节点 0。

下面用 $node_0$ 表示 0， $node_1$ 表示 1，依此类推。

1. 把 $node_0$ 指向 $node_2$ 。

2. 把 $node_2$ 指向 $node_1$ 。

3. 把 $node_1$ 指向 $node_3$ 。

4. 更新 $node_0$ 为 $node_1$ ，更新 $node_1$ 为 $node_3$ 。

5. 如果 $node_1$ 和 $node_1.next$ 都不为空，

就回到第一步，执行下一轮交换。
6. 最后返回 *dummy.next*，作为新链表的头节点。

题解图片

就按照这张图里面的算法来即可。为了巩固我的记忆，我用自己的话再讲述一遍：

1. 首先，我们做这个翻转的动作，为了方便，先规定4个节点，我这里定义为p0~p3，顺序为从左到右，依次为：要交换的第一个节点的前驱，要交换的第一个节点，要交换的第二个节点，要交换的第二个节点的后继(建议配合上面的图理解)。
2. 开局我们先把p0和p1初始化完毕， `p0=&dummy, p1=head`。
3. 进入循环后，我们再现场获取p2和p3的值。分别是 p1的next 和 p1的next的next。
4. 然后，我们需要先把p0指向p2。然后再把p2指向p1。接着再把p1指向p3。
5. 经过这么一番操作，其实已经交换完毕了。接下来是为下一轮循环准备：把这四个节点的位置挪到下一轮它们应该在的位置上去。
6. 由于p1和p2已经交换了，因此p3的前驱为p1。又因为进入下一轮循环后，p3的身份变成了下一轮的p1，所以下一轮的p0就是当前的p1。换句话说，我们需要令 `p0=p1, p1=p3`。
7. 然后继续进入下一轮循环即可。至于进入循环的条件，即 节点至少有两个，否则不反转。即p1的next不为null。

上代码：

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode dummy(0, head);
        ListNode *p0 = &dummy, *p1 = head;
        while (p1 && p1->next) { //至少有两个节点
            p1 = p0->next;
            auto p2 = p1->next;
            auto p3 = p2->next;
            p0->next = p2;
            p2->next = p1;
            p1->next = p3;
            p0 = p1;
            p1 = p3;
        }
        return dummy.next;
    }
};
```

C++

就讲到这里。

29-25 K 个一组翻转链表

题目链接: <https://leetcode.cn/problems/reverse-nodes-in-k-group/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难

解题日期: 2026.2.1

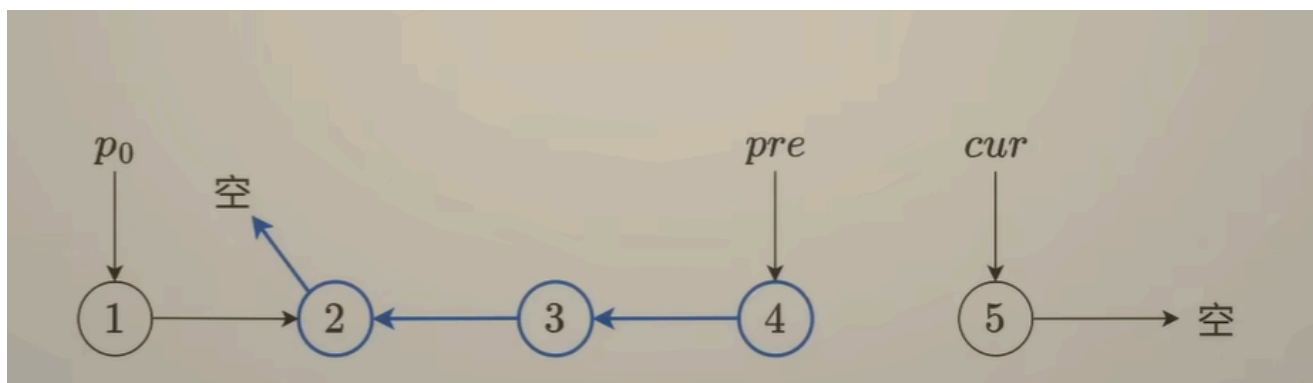
标签: 链表 递归

推荐题解: [灵神的题解\(记得看视频\)](#)

这题还是比较有难度的。推荐先把题解和视频看了。

这题也是有前置题目的: [92. 反转链表 II](#), 大致就是处理某个链表中间那一段的节点的反转。只要你会做这道题, 那么本题的思路就很明白了: 先从头遍历到尾, 把这个链表的长度算出来, 之后再代入题目给的k计算我需要反转几段, 最后单独开循环直接开始反转就完事了, 一段一段来, 最后留下不足k个的那一组不动, 然后返回头结点。

大体思路解决了, 我们来慢慢拆问题。首先, 如何做这道 反转链表II 的题目? 先拿一张图过来:



刚刚反转完的情况

这是我们刚刚用 反转链表 这道题的逻辑, 把2-4这一段的节点刚刚反转完时的状态。此时pre自然指向4(该区间反转后的第一个节点), cur自然指向5(该区间后面的第一个节点), 而2(该区间反转后的最后一个节点)却指向空。反转逻辑没问题, 主要是我们需要处理这里的烂摊子——让这个链表继续连下去。

首先, 我们需要把p0(区间左边的第一个节点, 也有可能是哨兵节点, 因此这道题我们依旧使用哨兵节点, 来一般化问题)指向pre, 把2指向cur, 我们顺着操作步骤是这样想的。但是这样有问题: 我们第二步访问不到2了。

发现顺着有问题, 就摸着回去, 容易发现: 我们让2先指后面就行了。因为2同为p0和3的后继, p0可以直接访问到它。即: `p0→next→next=cur`。然后再让p0指向pre, 就形成了完美闭环。

目前的节点赋值语句如下:

```
p0→next→next = cur; //让2先指后面
p0→next = pre; //然后再p0指向pre
```

C++

来到这道题, 其实我们就是需要处理连在一起的这一个个区间, 让它们内部反转。于是, 我们还需要管理操作下一轮区间时, p0, pre和cur的状态。

cur保持原状，这没问题。pre，我们需要把它从4换到2，因为反转之后的区间的末尾是这个区间反转之前的开头。

至于p0，我们和当前的pre保持一致即可，毕竟在开始反转下一段区间之前，它们都指向上一个区间反转后的末尾。

那么总结一下这道题我们需要新增的，在进入下一波反转之前的指针指向修正 (以上图为例)：将pre和p0都指向2，cur保持不变。

但是依旧遇到了问题。看上面的节点赋值语句，不难发现我们又没法指向2了——因为p0的next已经被改了。

一样的解决方案：在p0的next被改之前，再插入一行代码，把2的位置暂存一下，然后下面就能正常给p0和pre赋值了。

最终节点赋值语句如下：

```
C++
p0->next->next = cur;
auto pnxt = p0->next; // 暂存该反转完区间的末尾元素
p0->next = pre;
p0=pnxt;
pre = p0;
```

那么，题目还要求长度不到k的剩余元素保持不变，我们需要另外写逻辑吗？答案是不需要。因为在执行最后一次反转时，为可能到来的下一次反转(实际没有)所准备的事情(让2指后面，让p0指pre)其实已经保证了链表的完整性。此时我们直接返回结果即可。

上代码：

```

class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        //1.先统计链表长度
        int cnt=0;
        ListNode *p = head;
        while (p) {
            cnt++;
            p=p->next;
        }
        //2.开始一组一组反转链表
        int times = cnt/k; //计算要反转的组数
        ListNode dummy(0,head); //定义哨兵节点, next指向head
        ListNode *pre=nullptr,*cur=head,*p0=&dummy;
        for (int i=0;i<times;i++) {
            for (int j=0;j<k;j++) {
                auto nxt = cur->next;
                cur->next = pre;
                pre=cur;
                cur=nxt;
            }
            //下方为核心操作
            p0->next->next = cur;
            auto pnxt = p0->next;
            p0->next = pre;
            p0=pnxt;
            pre = p0;
        }
        return dummy.next; //返回哨兵节点的后继, 即头结点
    }
};

```

就讲到这里。

30-138 随机链表的复制

题目链接: <https://leetcode.cn/problems/copy-list-with-random-pointer/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.2

标签: 链表 哈希表

推荐题解: 无

这道题属于是非常新奇。做了前面这么多道题目, 都要求我操作原节点, 不让我复制, 这导致我看到这道题的第一反应是: 我要把节点原封不动的取出来然后再按照相同的逻辑连起来...那不是直接返回head就完事了。

后来一看, 噢, 原来是深拷贝啊。那没事了。

咳咳，言归正传。先顺着最正常的思路往下想想吧，说不定能发现啥问题。首先，我们沿着原链表往下遍历，把当前节点的值取出来，新建一个节点。然后，当前节点还会指向另一个随机的节点，我们需要再把那个随机的节点的值取出来，新建节点，然后把刚刚新建的第一个节点的随机指针域指向这个新建的节点。就这样一直进行下去...吗？

很明显，由于随机节点也是属于这个链表的，我在前面提前创建了这个随机节点的拷贝之后，我在原链表遍历到这个随机节点时依然会执行一样的动作。虽说我遍历链表的值不会出问题，但是这样重复创建节点的行为是题目不允许的。因此，我们需要引入映射来帮助我们：将原链表节点和新创建的节点一一对应，我们就能很轻松的掌握新链表的节点创建情况。

那么，思路就很明朗了。我们只需要在创建新节点之前查一下这个映射，看看这个节点是否在前面已经被创建过了，如果是，直接拿来用，否则再创建。这样就能避免重复的问题。就这样瞻前顾后，我写出了这道题的第一版代码：

```

// Definition for a Node.
class Node {
public:
    int val;
    Node* next;
    Node* random;

    Node(int _val) {
        val = _val;
        next = NULL;
        random = NULL;
    }
};

class Solution {
public:
    Node* copyRandomList(Node* head) {
        unordered_map<Node*, Node*> cors; // 原节点对应新节点
        Node dummy(-1);
        dummy.next = head;
        Node *p=head, *p0=&dummy;
        while (p) {
            if (cors.count(p)==0) {
                Node *nxt = new Node(p->val);
                p0->next = nxt;
                p0 = nxt;
                cors[p] = nxt;
            } else {
                p0->next = cors[p];
                p0 = p0->next;
            }
            if (p->random) {
                if (cors.count(p->random) == 0) {
                    Node *nxt = new Node(p->random->val);
                    p0->random = nxt;
                    cors[p->random] = nxt;
                } else {
                    p0->random = cors[p->random];
                }
            } else {
                p0->random = nullptr;
            }
            p=p->next;
        }
        return dummy.next;
    }
};

```

虽说一遍遍历就结束了比赛，但是这么多if-else分支，还是不够优雅，略显臃肿。

主要原因还是我在写的时候太强求一遍遍历搞定了，反而导致代码写的麻烦。既然一遍不行，两遍呢？

大体思路肯定不变。换成两遍遍历后，第一遍遍历，我们不需要任何判断，直接把旧节点和新节点按照1:1的方式丢进映射就行。至于第二遍，只需要按部就班的让新节点照着旧节点的连线就行了，依旧不需要任

何判断。代码如下：

```
C++
class Solution {
public:
    Node* copyRandomList(Node* head) {
        //1.新建映射
        unordered_map<Node*, Node*> m; //原节点对应新节点
        Node dummy(-1);
        dummy.next = head;
        Node *p=head, *p0=&dummy;
        //2.第一遍先全部丢进映射中
        while (p) {
            m[p] = new Node(p->val);
            p=p->next;
        }
        //3.第二遍遍历开始连线
        p=head;
        while (p) {
            p0->next = m[p];
            p0=p0->next;
            p0->random = m[p->random];
            p=p->next;
        }
        return dummy.next;
    }
};
```

其实这道题还有一种不用哈希表的方法，就是把新旧节点交叉连接，这样旧节点的后继就是新节点，天然完成了对应的任务。这里我就不展开了，感兴趣的同学可以移步 [灵神的题解](#)。

31-148 排序链表

题目链接：<https://leetcode.cn/problems/sort-list/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.2

标签：链表 双指针 分治 排序 归并排序

推荐题解：[灵神题解](#)

这道题比较偏算法底层了，且是我第一次在力扣里面接触真正的排序算法的底层，甚至还需要操作链表。这个题解，我想写的认真一点。

因此，这道题我准备来个一题三吃：从"作弊"的标准库排序(我的第一反应)，到 $O(\log n)$ 额外空间的递归版本归并排序(题解最推荐的写法，因为代码简洁，但是我觉得没达到题目 $O(1)$ 的进阶要求的话，还不如用第一种方法)，再到只用 $O(1)$ 额外空间，直接欧美上手原链表排序的迭代版本归并排序(也是最难写的)。坐好扶稳，我们发车！

首先来讲讲，这道题 如何使用标准库 `sort()` 排序，其实很简单：将节点本身一个个按顺序放进vector，然后 `sort()` 根据 `p->val` 自定义排序，最后再把节点按照排完之后的顺序一个个接回去。这样虽说耗费了O(n)的额外空间，但是代码写起来极其简单，并且标准库排序的效率也很高，时间复杂度控制在O(n log n)不在话下。代码如下：

```
C++
class Solution {
public:
    ListNode* sortList(ListNode* head) {
        //1.把节点一个个取出来
        ListNode *p = head;
        vector<ListNode*> nodes;
        while (p) {
            nodes.emplace_back(p);
            p=p->next;
        }
        //2.将节点按顺序排序
        sort(nodes.begin(), nodes.end(), {
            return a->val<b->val;
        });
        //3.将链表接回去
        ListNode dummy(0);
        ListNode *p0=&dummy;
        for (int i=0;i<nodes.size();i++) {
            p0->next = nodes[i];
            p0=p0->next;
        }
        p0->next=nullptr;
        return dummy.next;
    }
};
```

接着，我们来看第二种方法：递归版本的归并排序。

先复习一下归并排序的逻辑(来自Gemini)：

1. 第一板斧：原子化拆分 (Divide)

归并排序的第一步是疯狂地“套娃”。

- 逻辑：拿到一个序列，从中间劈开。左边一半，右边一半。
- 递归：对左边继续劈，对右边也继续劈。
- 到底为止：什么时候停？当你手里只剩下 1 个元素 的时候。
- 底层假设：归并排序最天才的直觉就在这里——“一个元素的序列，天生就是有序的。”这一步完成了从“无序”到“有序”的质变。

2. 第二板斧：有序化合并 (Conquer & Combine)

这是归并排序真正的“发动机”。当你手里有一堆长度为 1 的有序小块时，你需要把它们像拉链一样拉起来。

底层逻辑演示：假设你有两个已经排好序的小组： $L = [2, 5]$ 和 $R = [1, 4]$ 。

1. 比较：看两个组的第一个数。2 和 1 比，谁小？1 赢了。
2. 提取：把 1 拿出来，放到结果里。
3. 继续：现在看 2 和 4 比（因为 1 已经被拿走了）。2 赢了。
4. 填坑：把剩下的数按顺序直接接在后面。
5. 结果： $[1, 2, 4, 5]$ 。

大白话：就像两个班级按身高排队进礼堂，两个班主任只需要盯着各自队伍的最前面那个人，谁矮谁进去。这种方法比在乱序人群中找人要快得多！

做这道题之前，我们依然需要掌握两道前置题目，这两道题目都是我们之前做过或者间接做过的：[876. 链表的中间结点](#) 和 [21. 合并两个有序链表](#)。前者用于把链表切成两半(相当于 剪刀)，后者用于把链表按照升序拼起来(相当于 胶水)。我们只需要复现这两道题目的逻辑，并且稍做修改，这道题目就完成了80%。

首先，由于我们需要一个在中间剪短链表的 剪刀，我们只需要 在寻找到中间节点后，将其前驱节点的 `next`改成`null`。代码如下：

```
//找链表中点，并将前后断开
ListNode* middleNode(ListNode* head) { //对应876题(没做)
    ListNode *slow=head,*fast=head,*pre=head;
    while (fast && fast->next) {
        pre=slow;
        slow = slow->next;
        fast = fast->next->next;
    }
    pre->next = nullptr; //断开前后两半，这是区分于876题原题的部分
    return slow;
}
```

C++

然后合并两个有序链表的逻辑不需要修改，直接拿来就能用。并且，那道题目的测试用例对我们的考验反而促成了这段代码的 健壮性：当归并排序长度为3的链表时，剪刀会把它分成长度为2和1的两部分，这时候，我们这个合并的逻辑是支持长度不同链表的合并的，就能够完美处理这种情况。代码如下：

```

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) { //对应21题
    ListNode dummy(0);
    ListNode *p0 = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) {
            p0->next = l1;
            l1 = l1->next;
        } else {
            p0->next = l2;
            l2 = l2->next;
        }
        p0 = p0->next;
    }
    if (l1) p0->next = l1;
    else if (l2) p0->next = l2;
    return dummy.next;
}

```

是的，你没看错，把这两个逻辑搬过来并且稍作修改，你就完成了这道题目的80%。剩下的20%，需要我们回归归并排序的本质了。

首先，上面提到：一个元素的列表就是有序的。因此，当递归到只有head一个节点时，就返回。这是我们递归的返回条件。

然后，我们需要分别处理左边那一半和右边那一半，那就让它们各自递归调用这个函数，各自排序完毕之后返回新的head。

最后再用合并两半的函数，把两个有序序列合并，就完事了。代码如下：

```

ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;
    auto head2 = middleNode(head);
    head = sortList(head); //这里的head身份实际上更新了，如果想要写的更规范，可以新建两个变量
    head2 = sortList(head2);
    return mergeTwoLists(head, head2);
}

```

那么递归为啥能给它排序呢？我们结合递归的返回条件看，真正起排序作用的其实就是左半边和右半边都只有一个节点的时候——因为这样，递归直接到底，触发合并函数的调用，然后在这个合并函数里面，就会把这两个节点小的放在前面，如此往复，最终实现全部有序。

这就是递归的魅力。

接着，来到真正符合这道题目对于空间复杂度进阶要求的第三种做法：迭代归并排序。

说白了，就是把原本交给递归栈自动处理的切多长、怎么切、怎么接的逻辑全都揽过来，才能实现迭代。

因此，首当其冲肯定是把所需的工具函数写好。

首先是归并排序最重要的"并"字，我们依然需要合并两个升序链表的函数。但是真的能直接使用吗？

在第二种做法中，合并升序链表的工具函数只返回了一个头结点。由于我们需要自己切自己接，就需要像织毛衣一样把并完的链表接在前面已经并完链表的结尾，因此我们需要一直维护一个指向链表结尾的指针，此时我们就需要这个合并函数也返回尾指针，省的我们每次都要往后遍历找尾巴。

改造完之后是这样的：

```
pair<ListNode*, ListNode*> mergeTwoLists(ListNode* l1, ListNode* l2) { //合并完成后
    //返回链表头和尾
    ListNode dummy(0);
    ListNode *p0 = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) {
            p0->next = l1;
            l1 = l1->next;
        } else {
            p0->next = l2;
            l2 = l2->next;
        }
        p0 = p0->next;
    }
    if (l1) p0->next = l1;
    else if (l2) p0->next = l2;
    //改造点1: 继续往后推p0找尾巴
    while (p0 && p0->next) {
        p0 = p0->next;
    }
    //改造点2: 两个返回值
    return {dummy.next, p0};
}
```

那么既然我们需要自己控制切的长度，那么我们就需要自己开循环枚举步数，直到步数大于了当前链表长度才停止。那么我们就需要写一个获取链表长度的工具函数：

```
int getListLength(ListNode* head) {
    int length=0;
    ListNode *p = head;
    while (p) {
        length++;
        p=p->next;
    }
    return length;
}
```

最后呢，我们还需要解决"切"的问题。由于是迭代，我们没法像递归一样每次都切中间，因为 我们不具备自动分治的能力。因此，我们需要把每次都切中间的剪刀换成 每次靠左边切制定长度的菜刀：我们需要新写一个切的工具函数，使得每次传入链表头结点和要切的长度，就能在靠头结点这边切一段指定的长度出来。像不像你切菜的时候用弯曲的手指抵着菜刀来控制每一段的长度？

C++

```

ListNode* splitList(ListNode* head, int size) { //像切菜一样，一次切size长度
    //1.先找到切除点的前一个位置
    ListNode *p = head;
    for (int i=0; i<size-1 && p; i++) {
        p = p->next;
    }
    //如果截断的长度没达到size就不操作
    if (!p || !p->next) {
        return nullptr;
    }
    ListNode *nextHead = p->next;
    p->next = nullptr;
    return nextHead;
}

```

到这里，题目完成了50%(因为我们需要自己负责分治的逻辑)，下面的50%就在主逻辑里面。

理一下逻辑吧：

1. 先获取链表长度，并正常初始化；
2. 开始按照1 2 4 8...的翻倍节奏来循环每次归并的步长，也就是切链表切的长度；
3. 然后再开一个循环，利用切链表的工具函数返回的是下一个头结点的特性，相当于每一段的头结点的指针是跳着步长往后，到哪里哪里断，按照步长将链表切成两段两段，并且每两段都会用合并的工具函数来合并。做完上述操作，我们还得维护一个tail指针，使得它一直保持在当前链表的结尾，方便我们在尾部插入已经并好的链表；
4. 最后返回头结点即可。

大致的思路就是这样，我已经尽量讲的清楚了，直接上主函数的代码：

C++

```

ListNode* sortList(ListNode* head) {
    int length = getListLength(head); //先获取链表长度
    ListNode dummy(0, head); //例行初始化哨兵节点
    ListNode *p0 = &dummy;
    for (int step=1; step<length; step*=2) {
        ListNode *tail = p0, *cur = dummy.next; //tail一直指向链表的末尾，便于插入；
        //cur用于指向头结点，相当于切割的地方
        while (cur) {
            auto head1 = cur; //第一段的头
            auto head2 = splitList(head1, step); //此时才真正切出第一段，此时head2是刚
            //刚被切掉的上面那一段的头
            cur = splitList(head2, step); //此时才真正切出以head2为头的第二段，cur为下一
            //一段的头
            auto [h, t] = mergeTwoLists(head1, head2); //合并两段
            tail->next = h; //将合并完成的部分接在上一段合并完成部分的末尾
            tail = t; //将尾巴指针保持在当前归并完成链表的尾巴
        }
    }
    return dummy.next;
}

```

代码看完了吧？补充几个点：

- 外层 **step** 循环：是在决定“积木的大小”（1, 2, 4, 8...）。
- 内层 **while(cur)** 循环：是一次 全长扫描。它像是一个 步长为 **2*step** 的滑动窗口。每次窗口内做三件事：切出左块、切出右块、合并并接回主链。

这样应该更清晰了吧。

最后对比一下这三种方法：

方法	时间复杂度	额外空间	编写难度	推荐场景
标准库 sort	$O(n \log n)$	$O(n)$	★	面试救急/实际开发
递归归并	$O(n \log n)$	$O(\log n)$	★★★	算法面试常规解
迭代归并	$O(n \log n)$	$O(1)$	★★★★★	极客追求/硬核底层

可以根据实际情况来选择。

32-23 合并 K 个升序链表

题目链接：<https://leetcode.cn/problems/merge-k-sorted-lists/description/?envType=study-plan-v2&envId=top-100-liked>

难度：困难

解题日期：2026.2.3

标签：链表 分治 堆（优先队列） 归并排序

推荐题解：[灵神题解](#)

这道题的做法，有点前面 **10-239 滑动窗口最大值** 的影子。不过也对，貌似我刷hot100上次用堆就是在这道题，已经好长时间没用过堆了。

我看到这道题的第一反应，是拓展 **合并2个升序链表** 这道题的做法，把两个链表取最小头结点变成k个链表取最小头结点，然后想着好像元素有点多了不方便比，并且也想到了小根堆比较擅长处理这个。

然后呢，取了最小的那个节点(也就是堆顶)之后呢？怎么往堆里面补元素——换句话说，下一个可能的头结点来自哪里？由于每个序列都是升序的，所以，既然这个链表的头结点被取了，它的后继有可能是下一个可能的节点，当然其它头结点也有可能是。那么如果这个节点没有后继呢？那就不管了呗。

因此，这道题的基本思路就是，开头直接把所有链表(非空)的头结点加入堆，然后再开循环取堆顶节点，然后如果堆顶节点有后继就把后继结点入堆，如果没有就不管，一直循环直到堆空。

代码如下：

```

class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        priority_queue<pair<int,ListNode*>> pq;
        ListNode *p=nullptr;
        //1.先把全部头结点入堆
        for (auto head:lists) {
            if (head) pq.emplace(-head->val,head); //注意这里元素值要取负, 因为cpp默
            认大根堆
        }
        //2.开始慢慢弹元素
        ListNode dummy(0);
        ListNode *p0=&dummy;
        while (!pq.empty()) {
            auto [v,p]=pq.top();pq.pop();
            p0->next = p;
            p0 = p0->next;
            if (p->next) pq.emplace(-p->next->val,p->next); //这里也是
        }
        //3.收尾并返回
        p0->next=nullptr;
        return dummy.next;
    }
};

```

就讲到这里。

33-146 LRU 缓存

题目链接: <https://leetcode.cn/problems/lru-cache/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.3

标签: 设计 哈希表 链表 双向链表

推荐题解: [灵神题解](#)

非常新奇的一道题, 并且也是 面试必吃榜之一。所以, 这道题我将会使用Cpp和Go双语言实现。原理都一样, 所以我们先讲思路, 再上代码。

首先先讲讲 啥是LRU缓存 (来自Gemini 3 Pro):

简单来说，LRU 解决的是一个核心矛盾：地主家也没有余粮（内存昂贵且有限，而数据无穷无尽）。

1. 形象理解：你的“办公桌面”

想象你的桌子（内存）很小，只能放 5 本书。你最近看过的书放在手边。当桌子满了，你又要拿一本新书时，你会把那本 压在最下面、最久没碰过 的书放回书架（硬盘）。这就是 LRU。

2. 核心场景：哪里在用 LRU?

A. 数据库的“续命神器”（Buffer Pool）

像 MySQL 这样的数据库，数据是存在磁盘里的，但磁盘太慢了。

- 做法：MySQL 会在内存里开辟一块地方（Buffer Pool）。
- **LRU** 的作用：当你查询数据时，它会先从内存找。如果内存满了，MySQL 就会用 **LRU** 算法踢掉那些不常用的数据页，把新数据加载进来。这样可以保证“热点数据”永远留在内存里，速度起飞。

B. 操作系统（OS）的内存管理

你的电脑内存只有 16GB，但你可能同时开了浏览器、IDE、游戏，总需求超过了 16GB。

- 虚拟内存：操作系统把暂时不用的内存数据写到磁盘的“交换区”。
- **LRU** 的作用：当物理内存不够用时，系统会寻找最久没被访问的内存页（Page），把它交换到磁盘上。

C. 内容分发网络（CDN）

当你刷 B 站视频时，视频文件并不是都从总部服务器发的，而是从离你最近的 CDN 节点发的。

- **LRU** 的作用：CDN 节点的存储空间也是有限的。它会利用 LRU 缓存最热门的视频。如果某个视频没人看了，就会被新出的热门视频替换掉。

D. 应用层业务缓存

在 Go 后端开发中，你可能会遇到：

- 权限/ **Session** 缓存：频繁查数据库验证用户登录太慢，把最近登录的 10000 个用户信息存在内存 LRU 里。
- 防止缓存穿透/击穿：配合其他策略保护底层数据库。

3. 为什么面试爱考 LRU?

既然这么多地方都在用，面试官考你其实是在看：

- 1. 你懂不懂平衡：知道什么时候该“舍弃”旧数据。
- 2. 你对复杂度的敏感度：LRU 要求 `get` 和 `put` 都是 $O(1)$ 。如果你写个 $O(N)$ 的，高并发下一秒钟几万次请求，服务器直接宕机。
- 3. 结构设计能力：能不能想到把 `Map`（快查）和 `双向链表`（快排顺序）组合起来。

4. LRU 不是万能的（避坑指南）

LRU 有个天敌叫“全表扫描”。如果一个坏程序员写了个 SQL 查出几百万条冷门数据，这些冷门数据会一瞬间冲进缓存，把原本好好的热点数据全部挤出去。这叫 缓存污染。

进阶知识：为了解决这个问题，很多工业级实现（如 Redis 或 MySQL）会对 LRU 进行变种，比如 `LRU-K`（访问两次才进热点区）或者 分代 `LRU`。

各种缓存策略对比

策略	核心逻辑	缺点
FIFO (先进先出)	像排队，先来的先走	不考虑数据热度，可能踢掉刚用的
LRU (最近最少使用)	看时间。最久没用的走	容易被一次性的大规模扫描“洗掉”
LFU (最不经常使用)	看频率。用得最少的走	维护频率开销大；旧的热点数据占着位置不走

知道了啥是LRU缓存之后，我们就可以开始着手设计了。看题目要求，我们的 `put(int key, int value)` 和 `get(int key)` 的时间复杂度必须控制在 $O(1)$ ，因此我们能够天然想到利用 哈希表 来帮我们寻找目标节点的位置，换句话说，利用指针 指向目标节点，直接上手操作。

观察题目给出的定义框架：

```

class LRUCache {
public:
    LRUCache(int capacity){

    }

    int get(int key) {

    }

    void put(int key, int value) {

    }

};

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache* obj = new LRUCache(capacity);
 * int param_1 = obj->get(key);
 * obj->put(key,value);
 */

```

我们能发现，底部还给了个调用方式，这相当于对我们的代码进行了约束。

很明显，这个默认框架是不够用的，要capacity没capacity，要链表没链表，要哈希没哈希。因此，我们甚至还得自己对框架稍作修改(毕竟是设计题)。但是题目的最基础的思路甚至我们还没搞定，那么一步一步慢慢来吧：

1. 确定get和put操作要干的事情；
2. 确定这道题需要用到啥数据结构；
3. 修改框架使其达到我们的操作要求，同时符合题目规定的调用方式；
4. 填充get和put的操作，完成题目。

首先，确定get和put要干啥。

get，就是从这个缓存里面拿数据。回顾上面LRU的定义，如果我们访问一个数据，我们就需要将它取出并且放到最顶上，尽量防止它短时间内被淘汰，让它优先级最高。分解一下操作吧，取出，就是找到并删除；然后，放到最顶上，就是在链表头部插入它。还记得前面提到要O(1)时间复杂度吗？我们就需要快速找到这个结点，无论是在cpp和go里面，都是用哈希映射：将Key指向这个节点的指针。

put，就是把数据放入缓存。看似好像直接把数据放在链表头部就行了，实则有好几种情况要处理。如果容量够，并且key是新的，直接这样做没问题。如果key已经存在，就得当成是 **更新数据** 的操作，找到对应的节点，把值改了，放到头部。那么如果容量超了呢？就得把末尾的节点删掉，再插入新节点到头部。

接着，我们来确定数据结构。上面刚刚提到需要删除尾节点，那么，能够头尾访问自如的，肯定就不是单链表，而是双链表了吧。

然后，我们来修改这个框架：

```

struct Node {
    int key;
    int value;
    Node* prev;
    Node* next;
    //初始让自己前驱后继都指向自己
    Node(int k = 0, int v = 0) : key(k), value(v), next(this), prev(this){}
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {}

    int get(int key) {

    }

    void put(int key, int value) {

    }

private:
    int capacity;
    unordered_map<int, Node *> keyToNode;
    Node dummy;
    Node *p0 = &dummy;
};

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache* obj = new LRUCache(capacity);
 * int param_1 = obj->get(key);
 * obj->put(key,value);
 */

```

我定义了Node这个双向链表的节点，并且改了一下构造函数，初始化了容量这个参数。然后新建了private分区(直接写public里面也无所谓，算法题不用管封装性)，定义了capacity，key指向节点的哈希map，以及一个哨兵节点和它的指针。

接下来我们可以开始认真实现这些方法了。

首先是put方法，我们拆解一下它的操作：

如果 **Key** 已存在：

- 更新它的 **value** 。
- 把它移动到队头。

如果 **Key** 不存在：

- 检查容量：如果 `keyToNode.size() == capacity`，那就得牺牲 `tail→prev`（最久没用的那个）。
 - 从哈希表里删掉它。
 - 从链表里 `removeNode` 。
- 创建新节点：`new Node(key, value)`。
- 入库：把它加到哈希表，并且 `addToHead` 。

接着是get方法，我们拆解一下它的操作：

查表：如果在 `keyToNode` 里没找到，直接返回 `-1` 。

移动：如果找到了，先通过哈希表拿到这个 `Node*`，然后用上面的组合技把它 移动到队头 。

返回：返回它的 **value** 。

后面我们还得实现上面加到队头、删除的基本操作，我们也可以继续将它 封装成函数，这样代码可读性更强。

思路讲的差不多了，上最终cpp代码，有详细注释：

```

struct Node {
    int key;
    int value;
    Node* prev;
    Node* next;

    Node(int k = 0, int v = 0) : key(k), value(v), next(this), prev(this){}
};

class LRUCache {
public:
    LRUCache(int capacity) : capacity(capacity) {}
    //删除节点的基本操作
    void removeNode(Node* node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }
    //在队头插入节点的基本操作
    void addToHead(Node* node) {
        node->next = p0->next;
        node->prev = p0;
        p0->next->prev = node;
        p0->next = node;
    }
    //将节点移动到头部的复合操作
    void moveToHead(Node* node) {
        removeNode(node);
        addToHead(node);
    }

    int get(int key) {
        //如果哈希表里面没找到
        if (keyToNode.count(key)==0) return -1;
        //如果找到了, 将它取出, 移动到队头
        auto node = keyToNode[key];
        moveToHead(node);
        //返回它的值
        return node->value;
    }

    void put(int key, int value) {
        if (keyToNode.count(key)) { //如果哈希表里面有, 那就替换值
            auto node = keyToNode[key];
            node->value=value;
            moveToHead(node);
        }else { //如果哈希表里面没有
            if (keyToNode.size()==capacity) { //如果超出容量, 删掉末尾的
                auto toDel = p0->prev;
                keyToNode.erase(toDel->key);
                removeNode(toDel);
            }
            //然后创建新节点后加入, 放头结点
            auto newHead = new Node(key,value);
            keyToNode[key]=newHead;
        }
    }
};

```

```
        addToHead(newHead);
    }
}

private:
    int capacity;
    unordered_map<int, Node*> keyToNode;
    Node dummy;
    Node *p0=&dummy;
};

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache* obj = new LRUCache(capacity);
 * int param_1 = obj->get(key);
 * obj->put(key,value);
 */
```

34-94 二叉树的中序遍历

题目链接: <https://leetcode.cn/problems/binary-tree-inorder-traversal/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.4

标签: 栈 树 深度优先搜索 二叉树

推荐题解: [灵神题解](#)

长话短说。这道题其实就是数据结构课上教过的经典题目，有递归和迭代两种做法，后者只需要 $O(1)$ 的额外空间。

先说递归。中序遍历是左根右的顺序，我们只需要照着这个写就完事：

```

/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left),
right(right) {}
 * };
 */
class Solution {
public:
    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);
        ans.emplace_back(root->val);
        dfs(root->right);
    }
    vector<int> inorderTraversal(TreeNode* root) {
        dfs(root);
        return ans;
    }
private:
    vector<int> ans;
};

```

重点是线索树迭代做法，和数据结构课上讲的中序线索树是一个东西，只不过我们需要写代码来实现它，比用手模拟难多了。

Morris中序遍历算法的 核心思路 就是：在进入左子树之前，先找到左子树里最右边的那个节点（它是 **root** 的直接前驱），让它的 **right** 指针指向 **root**。这样，当你遍历完左子树，顺着这条“人造小路”就能直接走向 **root**。由于中序遍历是左根右，最大的障碍在左→根，根→右是可以直接走的。因此，只要能从左跳回根，遍历的回路就解决了。

我们可以把代码逻辑拆解为以下三个场景：

场景 A：如果没有左子树 (**if (!root->left)**)

既然左边没路了，按照中序遍历逻辑，直接访问自己，然后往右边走。

- **ans.push_back(root->val);**
- **root = root->right;**

场景 B：有左子树，且是第一次到达 **root**

当你第一次访问 **root** 时，你会先看一眼它的左子树。

1. 找到左子树中最右的节点 **pre**。
2. 发现 **pre->right** 是空的，说明这路还没修过。
3. 修路：**pre->right = root;**（建立线索）。
4. 继续深入：**root = root->left;**（去处理左子树）。

场景 C：有左子树，且是第二次到达 `root`

当你通过之前修好的“路”回到 `root` 时：

1. 再次找到左子树中最右的节点 `pre`。
2. 发现 `pre→right = root`，说明左子树已经全部逛完了。
3. 拆桥：`pre→right = nullptr`；（恢复树的原状，好习惯）。
4. 访问自己：`ans.push_back(root→val)`；
5. 去右边：`root = root→right`；

上代码：

```
C++
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> ans;
        while (root) {
            if (root→left) { //如果有左子树
                //那就访问到最右下角
                TreeNode *pre = root→left;
                //在即将通过线索跳到后继/到尽头时停
                while (pre→right && pre→right≠root) {
                    pre = pre→right;
                }
                //如果是第一次来，就建线索
                if (!pre→right) {
                    pre→right = root;
                    root = root→left; //由于是中序遍历，因此遍历左子树去了
                    continue;
                }
                pre→right = nullptr; //如果是第二次来这个左子树，就把线索删了，好习惯
            }
            //接着回到根
            ans.push_back(root→val);
            root = root→right; //跳转root到其后继，这里也有可能通过线索跳转
        }
        return ans;
    }
};
```

递归做法给个简单难度没问题，迭代做法不简单，起码给个中等。

35-104 二叉树的最大深度

题目链接: <https://leetcode.cn/problems/maximum-depth-of-binary-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.4

标签: 深度优先搜索 树 深度优先搜索 二叉树

推荐题解: 无

这题比较简单, 长话短说。我采用的是自底向上的递归写法, 直接上代码:

```
C++
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) return 0;
        int l = maxDepth(root->left)+1; // 左子树的最大深度
        int r = maxDepth(root->right)+1; // 右子树的最大深度
        return max(l,r); // 返回二者最大的即为最大深度
    }
};
```

顺带一提, 我发现了能让我递归写的更顺的金句: 在写递归函数时, 可以假设递归返回的结果一定是正确的。

我们就基于上面的代码来应用一下这句话(哈基米3Pro写的):

1. 第一步: 定义“契约” (Contract)

首先, 你得明确你的函数到底 承诺 了什么。

对于 `maxDepth(TreeNode* root)`, 它的契约是:

只要你给我一个二叉树的根节点, 我一定能算出以它为根的树的最大深度。

在写代码时, 你要无条件相信这个契约。

2. 第二步: 无条件信任 (The Leap of Faith)

当你写到这一行时:

```
C++
int l = maxDepth(root->left); // 此时不要进入函数内部去想它怎么算出来的
```

停住! 这里的核心就在于: 不要试图在脑子里跟着代码跳进左子树。

你要想: “根据契约, `maxDepth(root->left)` 一定 能准确地告诉我左子树的高度, 不管它里面有多复杂。既然它给我了结果, 我就把它记作 `l`。”

同理，你也这样信任右子树的结果 `r`。

3. 第三步：只解决“当下”的逻辑

既然你已经通过“信任”拿到了左右子树的高度 l 和 r ，那么对于当前这个节点来说，剩下的事就非常简单了：

- 整棵树的高度不就是左右两边最高的那个，再加 1（加上当前这一层）吗？
 - 于是你写出了：`return max(l, r) + 1;`
-

4. 第四步：用“边界条件”兜底

递归不能无限信任下去，必须有一个尽头。你只需要考虑最简单的情况：

- 如果树是空的（`root = nullptr`），深度是多少？
 - 显然是 0。于是你写出了：`if (!root) return 0;`
-

为什么这种方法有效？

这其实是 数学归纳法 (Mathematical Induction) 在编程中的应用：

1. 基础步 (Base Case)：证明 $n = 0$ 时成立。
2. 归纳步 (Inductive Step)：假设 $n = k$ 时成立，证明 $n = k + 1$ 也成立。

在二叉树中，每一个节点都在处理一个“比原问题规模更小”的子问题。如果你能保证：

1. 最小的问题（空树）是对的；
 2. 如果子问题（左右子树）是对的，当前问题也能推导正确。
- 那么整个递归在数学上就是无懈可击的。

36-226 翻转二叉树

题目链接：<https://leetcode.cn/problems/invert-binary-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.4

标签：深度优先搜索 树 二叉树

推荐题解：无

笑死我了，这不就是我数据结构期末考代码题原题吗。受上面的“递归信任”法的启发，我的代码写的非常简洁：

```
class Solution {  
public:  
    void invert(TreeNode* root) {  
        if (!root) return;  
        invert(root->left);  
        invert(root->right);  
        swap(root->left, root->right);  
    }  
    TreeNode* invertTree(TreeNode* root) {  
        invert(root);  
        return root;  
    }  
};
```

先把左边子树翻了，再把右边子树翻了，最后交换左右子树，就相当于这棵树整体被翻转了。这就是信任的力量，我相信我这个递归函数能做对应的事情，正常写就完事了。

依稀记得我期末考写了3个判断，看来根本用不着这么麻烦。

37-101 对称二叉树

题目链接：<https://leetcode.cn/problems/symmetric-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.4

标签：深度优先搜索 树 二叉树

推荐题解：[灵神题解](#)

这道题，其实可以直接拿上道题的函数，把这棵树翻转一下，再和原树对比是否相等。

这样做确实是简单难度，于是我想肯定有更加欧美，直接上手的方法，结果没想到。

看了一眼题解，这个做法也太优雅了吧：就把判断两棵树是否相等的函数照搬过来，然后掉换一下左右子树的顺序，就能完美运行在这道题上？？？

```

class Solution {
public:
    bool isEqual(TreeNode* leftTree, TreeNode* rightTree) {
        //这个判断语句很妙，把下面三个情况集中在一行：
        //都为空：!leftTree && !rightTree → 镜像是对称的，返回 true;
        //一个空一个不为空：!leftTree || !rightTree → 绝对不对称，返回 false;
        //（隐藏条件）都不为空但值不等：在最后一行 leftTree->val == rightTree->val 处
        //理。
        if (!leftTree || !rightTree) return leftTree == rightTree;
        //原本这里是左左比右左，左右比右右，下面改成了左左比右右，左右比右左这种对称形式
        return leftTree->val == rightTree->val && isEqual(leftTree->left, rightTree->right) && isEqual(leftTree->right, rightTree->left);
    }
    bool isSymmetric(TreeNode* root) {
        return isEqual(root->left, root->right);
    }
};

```

看了半天没看懂，我问了哈基米3Pro一个问题：

这个isEqual函数其实原本是判断两个二叉树是否相等的，只是我简单了调换了一下左右，其它都没改，就成功的运用到了这道题目上，给我一种它既判断镜像，又判断是否相等的职责不一的感觉。这是为啥？

看了它的回答，我好像懂了：这个函数看似是在对比相等，实则是对比两棵树在某种映射关系下是否完全一致。

这两道题的区别 仅仅在于映射规则不同：

题目	映射规则	递归逻辑
相同的树 (Same Tree)	全等映射 ($L \rightarrow L, R \rightarrow R$)	<code>isSame(L1, L2) && isSame(R1, R2)</code>
对称二叉树 (Symmetric)	镜像映射 ($L \rightarrow R, R \rightarrow L$)	<code>isEqual(L, R) && isEqual(R, L)</code>

我感觉它既能判断相等又能判断镜像，是因为这个函数本质上是在问：

“这两根‘树杈’按照我给你的对应方式，能不能对得上？”

- 如果你给它的指令是“左对左，右对右”，它就是在查 克隆。
- 如果你给它的指令是“左对右，右对左”，它就是在查 倒影。

所以，并不是函数的职责变了，而是 你赋予了参数不同的物理意义。在递归看来，它只是在 机械地对比：

1. 当前这两个节点值相等吗？
2. 第一对“指定伙伴”相等吗？
3. 第二对“指定伙伴”相等吗？

看完这里的解释，再回去看上面的代码，配合上面学到的“信任”，是不是好多了？

或者说，我们暂时先不管信任，有没有感觉，这个递归就是在同时操作两个指针来深度优先遍历并且一个个比对两边是否相等？

这其实就是它的本质。

38-543 二叉树的直径

题目链接：<https://leetcode.cn/problems/diameter-of-binary-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.5

标签：深度优先搜索 树 二叉树

推荐题解：[灵神讲解的视频，讲的确实不错](#)

我不行了，依旧被简单题单防。这道题其实改编一下上面 二叉树的最大深度 这道题的逻辑就行了。

直径的概念其实就是把二叉树看成图，找到哪两个点之间的距离最长。

在二叉树里面，我们拆一下子问题：假设当前的根节点就是中转点，那么最长的长度就是左子树的最大深度-1，加上右子树的最大深度-1。

但是实际上，我们不知道这条最长的路径会在哪个根节点中转。于是我们得另外维护一个最大值变量，在dfs这个二叉树的同时也把这个中转点给枚举了。

代码如下：

```
C++
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) return -1; // 由于要减一，所以底层返回值改为-1
        auto ld = maxDepth(root->left)+1; // 依旧信任这个函数能算出左子树的最大深度
        auto rd = maxDepth(root->right)+1; // 依旧信任这个函数能算出右子树的最大深度
        ans = max(ans, ld+rd); // 假设当前根节点就是中转点，直接维护最大值
        return max(ld, rd); // 选择左右最深的往上传，使得整体能够实现子树的最大深度-1
    }
    int diameterOfBinaryTree(TreeNode* root) {
        maxDepth(root);
        return ans;
    }
private:
    int ans=0;
};
```

个人觉得这道题难度设定为中等比较合适。

39-102 二叉树的层序遍历

题目链接: <https://leetcode.cn/problems/binary-tree-level-order-traversal/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.5

标签: 广度优先搜索 树 二叉树

推荐题解: 无

这道题倒是套用以前的老BFS思路做出来了。一开始想了大半天, 怎么记录dist, 然后灵机一动想到了用哈希表(映射), 然后剩下的逻辑就迎刃而解了, 直接套用我走迷宫时学的bfs算法, 这玩意在图、树、矩阵都能用, 有权无权都能写, 属于是一招鲜吃遍天了。

```
C++
class Solution {
public:
    // 用于决定是新增一行还是加在最后一行以插入答案的工具函数
    void emplaceAns(int val, int dist) {
        int l = ans.size();
        if (dist > l - 1) { // 如果不是最后一层
            ans.emplace_back(vector<int>{val}); // 新加一行
        } else {
            ans[l - 1].emplace_back(val);
        }
    }

    vector<vector<int>> levelOrder(TreeNode* root) {
        unordered_map<TreeNode*, int> dist; // 用于记录距离
        queue<TreeNode*> q;
        if (!root) return ans;
        dist[root] = 0;
        q.emplace(root);
        while (!q.empty()) {
            auto cur = q.front(); q.pop();
            emplaceAns(cur->val, dist[cur]);
            if (cur->left) {
                q.emplace(cur->left);
                dist[cur->left] = dist[cur] + 1;
            }
            if (cur->right) {
                q.emplace(cur->right);
                dist[cur->right] = dist[cur] + 1;
            }
        }
        return ans;
    }
private:
    vector<vector<int>> ans;
};
```

当然, 还有能够节省额外空间的做法:

```

class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (!root) return {};
        vector<vector<int>> ans;
        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int currentLevelSize = q.size(); // 关键: 这一层的节点个数
            vector<int> currentLevelNodes;

            for (int i = 0; i < currentLevelSize; ++i) {
                TreeNode* node = q.front();
                q.pop();
                currentLevelNodes.push_back(node->val);

                if (node->left) q.push(node->left);
                if (node->right) q.push(node->right);
            }
            // 跑完上面的 for 循环, 意味着这一层处理完了
            ans.push_back(move(currentLevelNodes));
        }
        return ans;
    }
};

```

二叉树其实是一种特殊的拓扑结构，它比矩阵和普通图少了两个麻烦：

- 没有环（Cycle-free）：你永远不需要担心回到已经访问过的节点，所以 `dist` 失去了“去重”的意义。
- 方向唯一：树的层次是严格向下的。在 BFS 时，第 k 层推入队列的一定是第 $k+1$ 层的节点。

所以，其实咱们可以把这个映射给省掉。

就讲到这里。今天休息一下，晚上酸角粥走起~

40-108 将有序数组转换为二叉搜索树(BST)

题目链接：<https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.6

标签：树 二叉搜索树 数组 分治 二叉树

推荐题解：[灵神题解](#)

依旧数据结构题目，不过代码版。

先讲讲啥是二叉搜索树：左子树根节点 < 根节点 < 右子树根节点 的树。

因此，由于序列是有序的，只需要一直切，然后分治就行了：

```
C++
class Solution {
public:
    TreeNode* dfs(vector<int>& nums,int left,int right) {
        if (left>right) return nullptr;
        int mid = left+(right-left)/2;
        return new TreeNode(nums[mid],dfs(nums,left,mid-1),dfs(nums,mid+1,right));
    }
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return dfs(nums,0,nums.size()-1);
    }
};
```

我的代码是左右闭区间写法，和灵神的左闭右开写法不一样，这里需要注意一下。

重点讲讲这个 `return new TreeNode(nums[mid],dfs(nums,left,mid-1),dfs(nums,mid+1,right));` 构造节点的操作：

1. 将当前序列中点作为新子树的根节点；
2. 相信我这个递归函数返回的就是我想要的子树，直接传入序列左半部分构建一个左半子树并返回；右边同理。
3. 这样，回传的节点就是一个符合条件的BST子树。

就讲到这里。

41-98 验证二叉搜索树

题目链接：<https://leetcode.cn/problems/validate-binary-search-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.6

标签：树 深度优先搜索 二叉搜索树 二叉树

推荐题解：[灵神题解](#)

依旧先把 二叉搜索树的性质 贴在这里：左子树根节点<根节点<右子树根节点。

那么如何判断这个二叉树是否符合这个条件呢？等等，左根右...怎么和中序遍历有点像？

先贴上中序遍历的代码：

C++

```

class Solution {
public:
    void dfs(TreeNode* root) {
        if (!root) return;
        dfs(root->left);
        ans.emplace_back(root->val);
        dfs(root->right);
    }
    vector<int> inorderTraversal(TreeNode* root) {
        dfs(root);
        return ans;
    }
private:
    vector<int> ans;
};

```

是不是很容易发现，只要中序遍历的序列严格递增，那它就是二叉搜索树了？

那么，根据我们写冒泡排序的经验，是不是两个相邻元素两两比较，只要前者全部小于后者，就能确定这个序列严格递增了？

当然，我们并不需要像输出中序序列一样，把序列存进数组再判断是否严格递增。我们只需要维护一个前驱变量，每次都把中序遍历当前值和前驱比大小不就完事了？直接省了一个数组的空间。

代码如下：

C++

```

class Solution {
public:
    bool isValidBST(TreeNode* root) {
        if (!root) return true;
        if (!isValidBST(root->left)) return false; // 看看左子树是不是BST
        if (root->val <= pre) return false; // 将当前值(根节点的值)和中序前序对比
        pre = root->val; // 维护前驱
        return isValidBST(root->right); // 左和中如果都没问题，再去处理右，和中序遍历的顺序是一样的
    }
private:
    ll pre = LLONG_MIN;
};

```

当然，如果想要更加自底向上，更加DP味的递归，也可以考虑 后序遍历，就是灵神题解所推荐掌握的第三种方法。这里由于时间问题，我就不掺和了。

附表：

做法	视角	核心重点
前序遍历 (自顶向下, 范围限定)	父亲对儿子的期望	传达 <code>(min, max)</code> 限制范围
中序遍历 (前驱对比)	此时与彼刻的联系	维护一个 <code>pre</code> 变量，看是否递增
后序遍历 (自底向上)	儿子对父亲的汇报	子树上报 <code>(min, max)</code> 范围

就讲到这里。

42-230 二叉搜索树中第 K 小的元素

题目链接：<https://leetcode.cn/problems/kth-smallest-element-in-a-bst/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.6

标签：树 深度优先搜索 二叉搜索树 二叉树

推荐题解：[灵神题解](#)

咱们刚刚用中序遍历做完上面这道验证BST的题目，这道题应该可以秒出思路吧：中序遍历递归+计数器触发回溯。看代码：

```
class Solution {
public:
    void dfs(TreeNode *root) {
        if (!root || ans != -1) return;
        dfs(root->left); //1
        cnt++;
        if (cnt == target) {
            ans = root->val;
            return;
        }
        dfs(root->right); //2
    }
    int kthSmallest(TreeNode* root, int k) {
        target = k;
        dfs(root);
        return ans;
    }
private:
    int cnt = 0;
    int target = 0;
    int ans = -1;
};
```

上面还有个小细节，我在触底反弹的条件里，加了 `ans != -1` 这一条。这是一个递归剪枝的操作：一旦 `ans` 被赋了值（即找到了答案），后续所有的 `dfs` 调用在进门的第一时间就会看到 `ans != -1`，然后直接 `return`。这就像是拉响了“下班铃”，剩下的路不用走了，整棵树的递归调用栈会迅速“自杀式”返回。

情况	不加 <code>ans != -1</code>	加上 <code>ans != -1</code>
执行动作	必须遍历完 整棵树	找到第 k 个后 立刻停止
访问节点数	N （总节点数）	$k + h$ （ h 是树的高度）

从上表可以看出，加了这个，还是能偷不少懒的，毕竟找到了就返回。

那么灵神题解里面的思考题：改成求第 k 大 要怎么做呢？答案：反向中序遍历(右根左)，直接调换上述代码的注释1和注释2即可。

就讲到这里。

43-199 二叉树的右视图

题目链接：<https://leetcode.cn/problems/binary-tree-right-side-view/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.6

标签：树 深度优先搜索 广度优先搜索 二叉树

推荐题解：[灵神题解](#)，[递归版本的思路](#)

这道题我看第一眼：不是无脑取右边就行了吗，右边到底了再换到左边，迭代就完事了；

第二眼：我去，这样没法自如切换到左边，让我想想怎么递归；

第三眼：递归没想出来，倒是发现这好像就是 **层序遍历(BFS)**取每一层最右边那个节点？

于是就完事了：

```

class Solution {
public:
    void appendAns(TreeNode* root, int dist) {
        int l = bfsResult.size();
        if (dist > l - 1) bfsResult.emplace_back(vector<int>{root->val});
        else bfsResult[l - 1].emplace_back(root->val);
    }

    void bfs(TreeNode* root) { //39-102 二叉树的层序遍历 原代码 略有适应性的改编
        if (!root) return;
        queue<TreeNode*> q;
        unordered_map<TreeNode*, int> dist;
        q.emplace(root);
        dist[root] = 0;
        while (!q.empty()) {
            auto cur = q.front(); q.pop();
            appendAns(cur, dist[cur]);
            if (cur->left) {
                q.emplace(cur->left);
                dist[cur->left] = dist[cur] + 1;
            }
            if (cur->right) {
                q.emplace(cur->right);
                dist[cur->right] = dist[cur] + 1;
            }
        }
    }

    vector<int> rightSideView(TreeNode* root) {
        bfs(root);
        for (auto &row : bfsResult) {
            ans.emplace_back(row[row.size() - 1]); //取每层最后一个
        }
        return ans;
    }

private:
    vector<vector<int>> bfsResult;
    vector<int> ans;
};

```

思路倒是挺顺，运行速度也挺快(top 0%)，不过给我一种不够native的感觉，直觉告诉我肯定有递归的做法。于是我去看了题解。

好吧，看来这个递归的思路是挺巧，看了一眼就理解了，不过想不出来就是想不出来：先递归右子树，再递归左子树，当某个深度首次到达时，对应的节点就在右视图中。

它甚至还利用了答案数组的长度这个参数作为记录层数的工具，妙哉，这里确实也有点bfs的味道在。

代码见灵神题解吧，我就不掺和了。换个角度想，至少我现在学会了。

44-114 二叉树展开为链表

题目链接: <https://leetcode.cn/problems/flatten-binary-tree-to-linked-list/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.7

标签: 栈 树 深度优先搜索 链表 二叉树

推荐题解: [灵神题解](#), 可以参考一下分治法

看到先序遍历, 我竟然一时没想出来。后来看了一眼题解豁然开朗——这不就和 **42-230** 二叉搜索树中第 **K** 小的元素 这道题的变式: 求第k大时的用反向中序遍历的做法类似? 我们直接 反向先序遍历(右左根), 然后整个哨兵节点 头插法 不就完事了?

```
class Solution {  
public:  
    void flatten(TreeNode* root) {  
        if (!root) return;  
        flatten(root->right);  
        flatten(root->left);  
        root->left = nullptr; //1  
        root->right = dummy->right;  
        dummy->right = root;  
    }  
private:  
    TreeNode *dummy = new TreeNode;  
};
```

C++

还有一个很迷的点, 就是注释1这一句, 如果不加会报错, 加了就没事。题目也确实说了left必须为null, 这也怪我没看到了哈哈。

45-105 从前序与中序遍历序列构造二叉树

题目链接: <https://leetcode.cn/problems/construct-binary-tree-from-preorder-and-inorder-traversal/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.7

标签: 树 数组 哈希表 分治 二叉树

推荐题解: [灵神题解](#)

这道题还是挺有意思的, 相当于将数据结构里面的手动构造法转换成代码。

基本思路不难, 就是写代码还是比较吃力: 利用了 先序序列第一位一定是该子树根节点 的特性。然后我们再去中序序列找这个根节点在其中的位置, 就能通过这个位置, 将中序序列一分为二, 左边就是左子树, 右边就是右子树。

然后，我们可以对着题目的用例，代数字进去找左右分治的边界。继续利用我们的递归信任法，很容易(并非，找边界找半天)就写出了我们的第一版代码：

```
C++
class Solution {
public:
    TreeNode* build(int preL,int preR,int inL,int inR) { //0 4 0 4
        if (preL>preR) return nullptr;
        int root = pre[preL];
        int inRootPos = inOrderPos[root]; //1
        int leftTreeSize = inRootPos-inL; //1
        auto leftTree = build(preL+1,preL+leftTreeSize,inRootPos-
leftTreeSize,inRootPos-1); //1 1 0
        auto rightTree = build(preL+leftTreeSize+1,preR,inRootPos+1,inR);
        return new TreeNode(root,leftTree,rightTree);
    }

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        pre = preorder;in = inorder;
        int n = preorder.size();
        //1.先把中序遍历丢进映射
        for (int i=0;i<n;i++) {
            inOrderPos[inorder[i]]=i;
        }
        //2.调用递归分治
        return build(0,n-1,0,n-1);
    }
private:
    unordered_map<int,int> inOrderPos;
    vector<int> pre,in;
};
```

看这个注释，就是我代入用例找边界的过程。看出来了吧？和我 手动在卷子上做题的时候的分治思想是一样的。

当然，我们可以发现， `inR` 这个参数是无效递归参数，因为它 没有参与任何运算 。因此我们可以拿掉。

这就是灵神题解里面代码让我一知半解的原因：他莫名其妙直接把 `inR` 去掉了，导致我一头雾水，感觉左右一点都不对称。如果能讲清楚会更好一些。

这是我最终的代码，和灵神题解思路基本一致，只不过我依然更喜欢闭区间写法：

```

class Solution {
public:
    TreeNode* build(int preL,int preR,int inL) {
        if (preL>preR) return nullptr;
        int root = pre[preL];
        int inRootPos = inOrderPos[root];
        int leftTreeSize = inRootPos-inL;
        auto leftTree = build(preL+1,preL+leftTreeSize,inRootPos-leftTreeSize);
        auto rightTree = build(preL+leftTreeSize+1,preR,inRootPos+1);
        return new TreeNode(root,leftTree,rightTree);
    }

    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        pre = preorder;in = inorder;
        int n = preorder.size();
        //1.先把中序遍历丢进映射
        for (int i=0;i<n;i++) {
            inOrderPos[inorder[i]]=i;
        }
        //2.调用递归分治
        return build(0,n-1,0);
    }
private:
    unordered_map<int,int> inOrderPos;
    vector<int> pre,in;
};

```

就讲到这里。

46-437 路径总和 III

题目链接: <https://leetcode.cn/problems/path-sum-iii/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.7

标签: 树 深度优先搜索 二叉树

推荐题解: [灵神题解](#)

我是真的想不到, 这道题居然能和 **9-560 和为 K 的子数组** 这道题联系在一起???

基本思路就是, 一边dfs遍历这个二叉树(根左右, 相当于前序遍历), 一边用哈希表构建前缀和, 一边计算。在处理根节点这部分, 总体和上面提到那道题的思路差不多。就是有一个很重要的点不太一样: 回溯撤销。

先上代码, 我要讲的话在注释里面:

```

using ll = long long;

class Solution {
public:
    void dfs(TreeNode* root, ll curSum) {
        if (!root) return;
        // 此处处理root以及前缀和逻辑
        curSum += root->val; // 累加当前路径总和，便于记录前缀和
        auto tar = curSum - target; // 1 需要探寻是否存在的符合当前和的目标起点
        if (prefixCnt.count(tar)) { // 如果有这个起点
            counter += prefixCnt[tar]; // 起点的数量就是符合条件区间的数量
        }
        prefixCnt[curSum]++; // 将当前前缀和数量加一
        // 下方递归处理左子树和右子树
        dfs(root->left, curSum);
        dfs(root->right, curSum);
        // 注意：此处必须在离开当前节点之前撤销当前节点的前缀和，防止已经不用的脏数据干扰对数量
        // 的探求
        prefixCnt[curSum]--;
    }
    int pathSum(TreeNode* root, int targetSum) {
        target = targetSum;
        prefixCnt[0] = 1; // 预先记录起点，前缀和为0，防止漏掉从起点开始的路线
        dfs(root, 0);
        return counter;
    }
private:
    unordered_map<ll, ll> prefixCnt;
    int target = 0;
    int counter = 0;
};

```

这是我初版代码犯的错：忘记“撤销”（回溯）

这是你在树上用 `unordered_map` 最容易犯的错。

- 问题所在：在数组里，前缀和是线性的。但在树里，当你处理完左子树、回到父节点、准备去右子树时，左子树的路径已经不再属于当前路径了。
- 后果：如果你不把左子树产生的前缀和从 `cnt` 中减掉，当你遍历右子树时，你会错误地把左子树里符合条件的路径也算进去。这会导致“跨越分支”的非法路径出现。

还有一个问题，就是数据溢出问题。在我注释1开头的那一行，前面的标如果写int，就会卡最后一个用例，写ll(long long)和auto就没问题。

因此，以后能用auto就用auto吧，毕竟auto虽然没法自动帮你决定用ll还是int，也没法自动处理溢出后的数据类型转换(例如 `auto res = INT_MAX + INT_MAX`，该溢出还是溢出)，但是它可以帮你自动处理这种不同数据类型(long long减int)运算后，用哪种数据类型最合适。

47-236 二叉树的最近公共祖先

题目链接: <https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.8

标签: 树 深度优先搜索 二叉树

推荐题解: [native的递归做法, 比较难想到](#)

这道题, 我自己想出来的办法是 分别求路径, 并且使用哈希倒序匹配二者, 最先匹配上的就是最近公共祖先, 时间复杂度 $O(n)$ 。

```
class Solution {
public:
    void dfs(TreeNode* root, TreeNode* target) {
        if (!root || (!path.empty() && path.back() == target)) return; //回溯剪枝
        if (root == target) {
            path.emplace_back(root);
            copiedPath = path; //定格path, 防止回溯过程的干扰
            return;
        }
        path.emplace_back(root);
        dfs(root->left, target);
        dfs(root->right, target);
        path.pop_back();
    }

    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        //1.先获得路径
        dfs(root, p);
        auto path1 = copiedPath;
        path.clear(); copiedPath.clear();
        dfs(root, q);
        auto path2 = copiedPath;
        path.clear(); copiedPath.clear();
        //2.给path1丢进哈希
        unordered_set<TreeNode*> set;
        for (auto x: path1) {
            set.emplace(x);
        }
        //3.反向遍历path2, 第一个出现的就是最近公共祖先
        for (int i = path2.size() - 1; i >= 0; i--) {
            if (set.count(path2[i])) {
                return path2[i];
            }
        }
        return nullptr;
    }

private:
    vector<TreeNode*> path, copiedPath;
};
```

值得一提的是，我这个 `copiedPath` 是偷懒做法，用空间复杂度(多存一个数组)和慢一丢丢的时间(拷贝会有时间开销)换我写代码的方便——因为我的撤销操作，在找到节点后回溯的时候会出问题(正常回溯当然没问题，是因为我这个if分支的return操作才导致的增删错位)，并且我还进行了剪枝操作，想让递归快点结束，因此会莫名其妙删掉一些不该删的东西，使得path不完整。

为了偷懒，我就直接在自顶向下找到目标节点时，直接复制一份保存，这样就没有后顾之忧了。

native的做法我这里就不展开了，依旧是有 信任法 的味道在，但是我还是节省一下我大脑的内存吧——毕竟还是记忆我自己整合出来的方法更省力。

48-124 二叉树中的最大路径和

题目链接: <https://leetcode.cn/problems/binary-tree-maximum-path-sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难

解题日期: 2026.2.8

标签: 树 深度优先搜索 二叉树

推荐题解: [我就是参考这个解法的](#)

你还真别说，老生常谈的"信任法"在解决这道题上面还真的有用武之地。

这题的基本思路，其实和 **46-437 路径总和 III** 和 **38-543 二叉树的直径** 这两道题都有关系。我们需要用前者累加路径和的思路，配合上后者枚举可能的转弯根节点(turning point)，来解决这道题。直接上代码：

```
class Solution {
public:
    int findMax(TreeNode* root) {
        if (!root) return 0;
        auto lMax = findMax(root->left); // 信任它能给我算出左子树总和最大的链
        auto rMax = findMax(root->right); // 信任它能给我算出右子树总和最大的链
        ans = max(ans, lMax+rMax+root->val); // 左右最大链和加上当前转折点的值，就是潜在
        的最大路径和
        return max(max(lMax, rMax)+root->val, 0); // 返回左右链中总和最大的，作为当前树最
        长的链条总和
    }
    int maxPathSum(TreeNode* root) {
        findMax(root);
        return ans;
    }
private:
    int ans = INT_MIN;
};
```

并且这个相当于后序遍历的dfs，也就是先处理完左右子树再来处理根。也相当于是一边dfs一边枚举可能的转折点。

树总算是告一段落了，咱们下一题 图 再见。

49-200 岛屿数量

题目链接: <https://leetcode.cn/problems/number-of-islands/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.8

标签: 深度优先搜索 广度优先搜索 并查集 数组 矩阵

推荐题解: [和我的解法一样, 附带一些优化](#)

基本思路很简单, 和我当时备战csp的时候刷的题目一样, 就是开个二层遍历, 然后如果当前格子是1就调用dfs"登陆"这个岛, 并且给这个岛的每个角落都遍历并留下记号(我这里选择老派做法: vis数组, 其实直接上手改这个矩阵, 将经过的地方改成2作为已访问就完事)。后面遍历就只会登陆格子是1的岛屿, 这样就能不漏死角的把每个岛都访问一遍, 且不会重复上岛。

```
C++
class Solution {
public:
    void dfs(vector<vector<char>> &mat,int x,int y) {
        if (x<0 || y<0 || x>=m || y>=n) return;
        if (vis[x][y] || mat[x][y]!='0') return;
        vis[x][y]=true; //做记号
        for (int d=0;d<4;d++) {
            auto nx = x+dx[d];
            auto ny = y+dy[d];
            dfs(mat,nx,ny);
        }
    }

    int numIslands(vector<vector<char>>& grid) {
        m = grid.size();
        n = grid[0].size();
        int cnt=0;
        vis.assign(m,vector<bool>(n,false));
        for (int i=0;i<m;i++) {
            for (int j=0;j<n;j++) {
                if (grid[i][j]=='1' && !vis[i][j]) { //只上没上过的岛
                    dfs(grid,i,j);
                    cnt++;
                }
            }
        }
        return cnt;
    }

private:
    vector<int> dx = {-1,1,0,0},dy = {0,0,-1,1};
    vector<vector<bool>> vis;
    int m=0,n=0;
};
```

当然, 写这个的时候我还遇到了问题: 我弄混了"走迷宫"式dfs和"雨露均沾"式dfs。我原先的写法是这样的, 有十几个用例过不去:

```
bool dfs(vector<vector<char>> &mat,int x,int y) {
    if (x<0 || y<0 || x>=m || y>=n) return false;
    if (vis[x][y] || mat[x][y]!='0') return false;
    vis[x][y]=true;
    for (int d=0;d<4;d++) {
        auto nx = x+dx[d];
        auto ny = y+dy[d];
        if (dfs(mat,nx,ny)) {
            return true; //找到一条路就直接剪枝返回了,后面的路就不走了
        }
    }
    return true;
}
```

问题在于：当我从点 (x, y) 出发，去探测四个方向时，只要第一个方向（比如向上）返回了 `true`，整个函数就立刻 `return true` 结束了。

后果：它剩下的三个方向（下、左、右）完全没有被探索。这导致我的 DFS 像是一条细细的“探针”，只顺着一个方向钻到底，而没有像“洪水”一样把整个岛屿全部淹没（标记为 `vis`）。

现象：因为一个岛屿没有被完整标记，外层的 `numIslands` 循环在扫到该岛屿剩下的部分时，会以为那是另一个新岛屿。所以我的 `cnt` 结果会比实际大得多，导致过不去用例。

💡 避坑总结

- 岛屿/填充类问题（Flood Fill）：DFS 必须是“贪婪”的，不撞南墙不回头，且要把所有南墙都撞一遍。通常建议写成 `void`，逻辑最纯粹。
- 路径/匹配类问题：DFS 是“目标导向”的，找到了就赶紧跑。这时候才用 `bool` 并配合 `return true`。

就讲到这里。

50-994 腐烂的橘子

题目链接：<https://leetcode.cn/problems/rotting-oranges/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.9

标签：资深工程师 广度优先搜索 数组 矩阵 第 124 场周赛

推荐题解：[和我的解法一样](#)

Congratulations!

一眨眼，咱们做到 第50题 了！进度条已经过半(实则不止一半，我看了一眼，后面的好多题目都在刷灵神题单的时候做过了。但是只要我记忆不尤新的题，我都会重新写的)，简直是时间飞逝啊。

简单庆祝一下，回归正题。这道题其实是一个很经典的考点(之前做csp题单以及二分题单都接触过)——多源bfs。

和正常的bfs几乎没啥区别，唯一区别就是咱们多源bfs开局需要预先将 同步扩散的多个点 都丢进队列中，后续操作就一样了。

顺便普及一下如何计算同步扩散的时间：使用求最短距离的dist数组即可，步数就是同步扩散了几轮，就是时间。

代码如下，顺带一提今天是第一天抛弃CLion，在力扣网页版上直接写题目，除了少了点自动提示，敲的慢了点，由于大多数stl都烂熟于心，因此影响不大：

```

class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        queue<pair<int,int>> q;
        //1.先扫描出腐烂橘子，并丢进队列
        m = grid.size();
        n = grid[0].size();
        int cnt=0;
        vector<vector<int>> dist(m,vector(n,-1));
        for (int i=0;i<m;i++){
            for (int j=0;j<n;j++){
                if (grid[i][j]==2) {
                    q.emplace(i,j);
                    dist[i][j]=0;
                }
                if (grid[i][j]==1) cnt++;
            }
        }
        //2.开始bfs
        int ans=0;
        vector<int> dx = {-1,1,0,0};
        vector<int> dy = {0,0,-1,1};
        while (!q.empty()){
            auto [x,y] = q.front();q.pop();
            for (int d=0;d<4;d++){
                auto nx = x+dx[d];
                auto ny = y+dy[d];
                if (nx<0 || ny<0 || nx>=m || ny>=n) continue;
                if (dist[nx][ny]!=-1 || grid[nx][ny]==2 || grid[nx][ny]==0)
                    continue;

                q.emplace(nx,ny);
                cnt--;
                dist[nx][ny]=dist[x][y]+1;
                grid[nx][ny]=2;
                ans = max(ans,dist[nx][ny]);
            }
            if (cnt<=0) break; //如果腐烂完了，可以提前退出，只不过灵神题解将其集中在了
while条件中
        }
        //记得返回答案前特判一下没腐烂完的情况
        if (cnt<=0) return ans;
        return -1;
    }
private:
    int m,n;
};

```

这里还有一个很讨巧的优化点：`cnt` 变量。我刚刚开始做这道题的时候，其实没想到要统计新鲜橘子的数量，还傻傻的写了个 `bool check()` 函数，扫描整个矩阵来检测是否有剩余的新鲜橘子(晕)：

```
bool check(vector<vector<int>>& grid){  
    for (int i=0;i<m;i++){  
        for (int j=0;j<n;j++){  
            if (grid[i][j]==1) return false;  
        }  
    }  
    return true;  
}
```

改成这个 `cnt` 计数器检测之后，时间直接从4ms变成0ms，没话说。

51-207 课程表

题目链接: <https://leetcode.cn/problems/course-schedule/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.9

标签: 深度优先搜索 广度优先搜索 图 拓扑排序

推荐题解: [是我的思路来源](#)

首先转换一下题意: 就是让我们 求这个图有没有环。

一开始我还很疑惑, 万一没环也没法修完呢? 后来转念一想, 起点又没有限制, 只要没环, 就能一门一门给它处理完毕。

那我们就要引入上方题解中的 **dfs**三色标记法 了:

- 0: 节点 x 尚未被访问到。
- 1: 节点 x 正在访问中, dfs(x) 尚未结束。
- 2: 节点 x 已经完全访问完毕。注意这还说明从 x 出发无法找到环。所以当我们遇到状态值为 2 的节点 x 时, 无需递归 x。

重点是这个状态1, 如果我在递归的过程中, 访问到了状态为1的节点, 就能石锤有环存在。

直接上代码:

```

class Solution {
public:
    bool dfs(int cur){
        if (colors[cur]==1) return true; //访问到了访问中的节点，说明有环
        if (colors[cur]==2) return false; //该节点访问过了，不访问
        colors[cur]=1; //排除上述情况，正常进入该节点，标记为访问中
        for (auto next:graph[cur]){ //找新节点
            if (dfs(next)) return true;
        }
        colors[cur]=2; //访问完毕，千万别忘记改成访问过
        return false; //能到这里，说明这一步没找到环
    }
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
        //1.先构造图
        graph.assign(numCourses,vector<int>());
        for (auto row:prerequisites){
            graph[row[1]].emplace_back(row[0]);
        }
        //2.三色标记法dfs
        colors.assign(numCourses,0);
        for (int i=0;i<numCourses;i++){
            if (colors[i]==0){ //如果没访问过，直接dfs
                if (dfs(i)) return false;
            }
        }
        return true;
    }
private:
    vector<int> colors;
    vector<vector<int>> graph;
};

```

我和灵神代码的逻辑基本一致，就是写法不同，他更喜欢把条件在遍历新节点的时候就都判断一下，个人感觉要思考的太多了，不如我这样纯粹(无拉踩，纯主观感受)。

52-208 实现 Trie (前缀树)

题目链接：<https://leetcode.cn/problems/implement-trie-prefix-tree/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.10

标签：设计 字典树 哈希表 字符串

推荐题解：[是我的思路来源](#)

又又又是一道设计题！吓得我赶紧回忆了一下LRU缓存。

这道题的基本思路是这样的：构造一个26叉树，把单词中的每个字母作为一个节点，一步一步往下走，走到这个单词的尽头，节点就被标为true。

这是我设计的数据结构：

```
struct Node {
    Node(){
        childs.assign(26, nullptr);
        end = false;
    }
    vector<Node*> childs; //存放26个可能的下一个字母，和字母序对应
    bool end; //标记是否为某个单词的末尾
};
```

这个标识可别小看它的作用，可以帮助我们完美区分到底是前缀(`end=false`)还是整个单词(`end=true`)。

数据结构搞定了，咱们来盘一盘逻辑。

首先是插入(Insert)操作，概括起来就是：有路就走，没路就修路再走。

遍历这个准备插入的word，提取出当前位的字母c，并且准备一个cur指针指向当前遍历到的位置，咱们分情况讨论：

1. 有路，也就是 `childs` 数组在c的字母序的对应位置上有对应节点存在，那就将 `cur` 指针往下指一位；
2. 没路，也就是 `childs` 数组在c的字母序的对应位置上是 `nullptr`，那就 `new` 一个 `Node`，再下移 `cur` 指针。

接着是找前缀(`startsWith`)和找单词是否存在(`search`)，咱们可以共用一套逻辑：有路就走(可能是前缀或者单词本身)，没路拉倒(返回 `false`)。

1. 如果中途断开了，说明不可能是前缀或者单词本身；
2. 如果给定单词遍历完了，终点的 `end=false`，那说明是前缀；
3. 如果 `end=true`，那说明就是单词本身。

我们写一个 `find` 函数，给上述三种逻辑编号0,1,2，然后就可以愉快的复用这个find函数了。

逻辑盘完了，上代码：

```

struct Node {
    Node(){
        childs.assign(26,nullptr);
        end = false;
    }
    vector<Node*> childs;
    bool end;
};

class Trie {
public:
    Trie() {
        root = new Node;
    }

    void insert(string word) {
        auto cur = root;
        for (auto c:word){
            if (!cur->childs[c-'a']){
                auto newNode = new Node;
                cur->childs[c-'a'] = newNode;
                cur = newNode;
            }else{
                cur = cur->childs[c-'a'];
            }
        }
        cur->end = true;
    }

    int find(string word){
        auto cur = root;
        for (auto c:word){
            if (!cur->childs[c-'a']){
                return 0;
            }else{
                cur = cur->childs[c-'a'];
            }
        }
        if (!cur->end) return 1; //说明是前缀
        return 2; //说明完全匹配
    }

    bool search(string word) {
        if (find(word)==2) return true;
        return false;
    }

    bool startsWith(string prefix) {
        return find(prefix);
    }
private:
    Node* root;
};

```

设计题的逻辑本身不难，就是接触这个新概念，需要花时间去理解。

53-46 全排列

题目链接: <https://leetcode.cn/problems/permutations/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.10

标签: 数组 回溯

推荐题解: 无

图这个章节咱们也是结束了, 接着来进行回溯。也是好久不见。

这道题我没看题解, 找了哈基米3Pro给我纠偏, 它告诉了我回溯的金律: 谁污染, 谁治理; 在哪儿 **push**, 就在哪儿 **pop**。

于是我也是一改我第一版跑不通的史山代码, 改成了这一版 回溯标准模板, 我们来看看它是如何体现回溯金律的:

```
C++
class Solution {
public:
    void dfs(){
        if (path.size()==length){ //长度够了, 就把这个路径丢进答案
            paths.emplace_back(path);
            return;
        }
        for (auto nextNum:numList){
            if (!pathSet.count(nextNum)){ //之前没选过才加入, 避免重复选择
                //入
                path.emplace_back(nextNum);
                pathSet.emplace(nextNum);
                //递归
                dfs();
                //出
                path.pop_back();
                pathSet.erase(nextNum);
            }
        }
    }

    vector<vector<int>> permute(vector<int>& nums) {
        numList = nums;
        length = nums.size();
        dfs();
        return paths;
    }
private:
    vector<int> numList;
    vector<int> path; //用于临时记录当前路径
    vector<vector<int>> paths; //用于记录准备返回的最终答案
    unordered_set<int> pathSet; //用于去重
    int length = 0;
};
```

看样子，只要外部的变量给全了，这个递归函数是可以一个参数都不用写的，非常简洁。并且，我们这个金律还确保了我不需要管它是刚开始还是正在排列，只需要调用它，它就能自动进入状态。(具有自启动效果)

54-78 子集

题目链接: <https://leetcode.cn/problems/permutations/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.10

标签: 数组 回溯

推荐题解: 无

求子集，其实就是求我 对于这一位数字选与不选的全部情况的集合。

举极端情况看看吧：如果是空集，说明每一位都不选；如果是全集，说明每一位都选了。

然后再搭配上一道题学到的金律：谁污染，谁治理；在哪儿 **push**，就在哪儿 **pop**。

我们可以写出如下代码，称为 方法一：考虑这一位选还是不选。

```
C++
class Solution {
public:
    void dfs(int curIndex){
        if (curIndex==numList.size()){ //所有情况都考虑完了，给路径加入最终返回的答案集
            paths.emplace_back(path);
            return;
        }
        //选这一位
        path.emplace_back(numList[curIndex]);
        dfs(curIndex+1);
        path.pop_back();
        //不选这一位
        dfs(curIndex+1);
    }

    vector<vector<int>> subsets(vector<int>& nums) {
        numList = nums;
        dfs(0);
        return paths;
    }
private:
    vector<int> path,numList;
    vector<vector<int>> paths;
};
```

还有第二种方法，也是哈基米推荐我掌握的逻辑：下一位我该选谁？

逻辑：每一个子集都是以某个数字开头的。我先选一个数，然后剩下的数里再随便选。

- 特点：有 `for` 循环。每进一次递归，就产生一个合法的子集。
- 诀窍：这种写法不需要显式的“不选”逻辑，因为 `for` 循环的下一轮自动就是“不选当前，选下一个”的逻辑。

```
C++
void dfs(int startIndex) {
    // 只要进来了, 当前的 path 就是一个合法的子集 (包括空集)
    paths.emplace_back(path);

    // 从起始位置开始往后挑
    for (int i = startIndex; i < numList.size(); i++) {
        path.emplace_back(numList[i]); // 选这个数
        dfs(i + 1);                    // 递归, 下次从 i+1 开始挑 (保证不回头, 不重复)
        path.pop_back();               // 回溯
    }
}
```

哈基米建议我优先掌握第二种(但是我第一种已经掌握的滚瓜烂熟了哈哈)，因为它是处理“组合”、“子集”、“切割”类问题的通用模板。那就等做到这些题目我再来顺带着加深印象吧~

今天就讲到这里。

55-17 电话号码的字母组合

题目链接：<https://leetcode.cn/problems/letter-combinations-of-a-phone-number/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.11

标签：哈希表 字符串 回溯

推荐题解：无

依旧先把回溯圣经放在开头：谁污染，谁治理；在哪儿 `push`，就在哪儿 `pop`。

这个圣经真挺好用的。这道题，我做完了才发现，好像就是上面这道题哈基米推荐的“下一位我该选谁”思路的变种：

每一位我都必须挑一个，然后挑了就直接加进路径里，然后递归，然后递归出来后把它拿掉。

然后记得往后传递当前选择到的数字，这样dfs才知道自己需要在哪个数字按键的字母集合里面选。

上代码：

```

class Solution {
public:
    void dfs(int curIndex){
        if (curIndex==digitsLength){
            ans.emplace_back(path);
            return;
        }
        for (auto next:numToAlpha[digits2[curIndex]-'0']){
            path+=next;
            dfs(curIndex+1);
            path.pop_back();
        }
    }

    vector<string> letterCombinations(string digits) {
        digitsLength = digits.size();
        digits2 = digits;
        dfs(0);
        return ans;
    }
private:
    vector<vector<char>> numToAlpha = {
        {}, {},
        {'a', 'b', 'c'},
        {'d', 'e', 'f'},
        {'g', 'h', 'i'},
        {'j', 'k', 'l'},
        {'m', 'n', 'o'},
        {'p', 'q', 'r', 's'},
        {'t', 'u', 'v'},
        {'w', 'x', 'y', 'z'}
    };
    int digitsLength = 0;
    string digits2;
    vector<string> ans;
    string path;
};

```

就讲到这里。

56-39 组合总和

题目链接: <https://leetcode.cn/problems/combination-sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.11

标签: 数组 回溯

推荐题解: [方法1是我的灵感来源](#)

这道题就比上道题棘手很多了。是的，我又和做前一道 **54-78 子集** 一样，把 这一位选不选 和 下一位我该选谁 两个做法搞混了，写了超级加倍循环，然后修bug修半天——还是没修好。

去看题解，有了点灵感，修修补补总算是修对了。最后我还是采用了 这一位选不选 的做法：

```
C++
class Solution {
public:
    void dfs(int curSum, int curIndex) {
        if (curIndex == candidates2.size()) {
            if (curSum == target2) {
                ans.emplace_back(path);
            }
            return;
        }
        // 选
        if (curSum + candidates2[curIndex] <= target2) {
            path.emplace_back(candidates2[curIndex]);
            dfs(curSum + candidates2[curIndex], curIndex);
            path.pop_back();
        }
        // 不选
        dfs(curSum, curIndex + 1);
    }

    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        sort(candidates.begin(), candidates.end());
        candidates2 = candidates;
        target2 = target;
        dfs(0, 0);
        return ans;
    }

private:
    vector<int> path, candidates2;
    vector<vector<int>> ans;
    int target2;
};
```

咱们好好盘一盘这两个方法的区别，下次千万别再搞混了：

维度	二分决策 (选或不选)	多路搜索 (for 循环)
关注点	关注 “当前元素” 的去留	关注 “当前位置” 填哪个元素
递归层级	递归了 N 层（每层决定一个元素）	递归了 K 层（每层填一个位置）
回溯位置	在 <code>dfs(选)</code> 之后立即 <code>pop</code>	在 <code>for</code> 循环内部 <code>push/pop</code>
你的代码习惯	适合追求逻辑纯粹、背包思维	适合追求路径清晰、搜索思维

选与不选方法相当于：

“第一个数，要吗？要！那还要吗？要！... 不要了。那第二个数，要吗？...”

多路搜索相当于：

“第一位我挑谁？挑 2。那下一位我从 [2, 3, 6, 7] 里挑谁？再挑 2。那下一位...”

就讲到这里。

57-22 括号生成

题目链接：<https://leetcode.cn/problems/generate-parentheses/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.11

标签：字符串 动态规划 回溯

推荐题解：无

这道题，也是不看题解一遍过了，运气不错哈哈。

讲讲我的理解。这道题看似限制重重，其实转换一下题意就是：给定 n 个左括号和 n 个右括号，求它们括号能成功闭合的全排列。

先解决全排列的问题，就是经典的：这一位我选左括号或者右括号。

然后咱们加上条件。

先思考，啥时候左括号 不能 选？因为左括号右边一定要有右括号，很容易知道左括号只有选够 n 个之后才不能选，因此选左括号的限定条件搞定了。

那么啥时候右括号 不能 选？根据括号的性质，右括号是左括号的“小跟班”，永远在选了左括号之后才选右括号，右括号不能先于左括号选。因此，左括号数量小于等于右括号时，不能选右括号。

给上面两个条件一套，然后dfs参数整两个计数器，完事：

```

class Solution {
public:
    void dfs(int lCnt, int rCnt){
        if (rCnt==n2){ //右括号选的晚于左括号，因此右括号够了就是够了
            ans.emplace_back(path);
            return;
        }
        if (lCnt<n2){
            path+='(';
            dfs(lCnt+1, rCnt);
            path.pop_back();
        }
        if (lCnt>rCnt){
            path+=')';
            dfs(lCnt, rCnt+1);
            path.pop_back();
        }
    }

    vector<string> generateParenthesis(int n) {
        n2 = n;
        dfs(0, 0);
        return ans;
    }
private:
    int n2;
    string path;
    vector<string> ans;
};

```

就讲到这里。

58-79 单词搜索

题目链接: <https://leetcode.cn/problems/word-search/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.12

标签: 深度优先搜索 数组 字符串 回溯 矩阵

推荐题解: 无

这道题也是颇有挑战性，但是也是让我做出来了。和 **49-200 岛屿数量** 这道题特别像。

像在哪里？**49-200 岛屿数量** 这道题，我们需要遍历找岛屿的落点；这道题，我们需要遍历找单词的开头，然后才开始dfs。

一开始写这道题的时候，我总觉得好像和回溯没啥关系，于是我的初版代码是这样的：

```

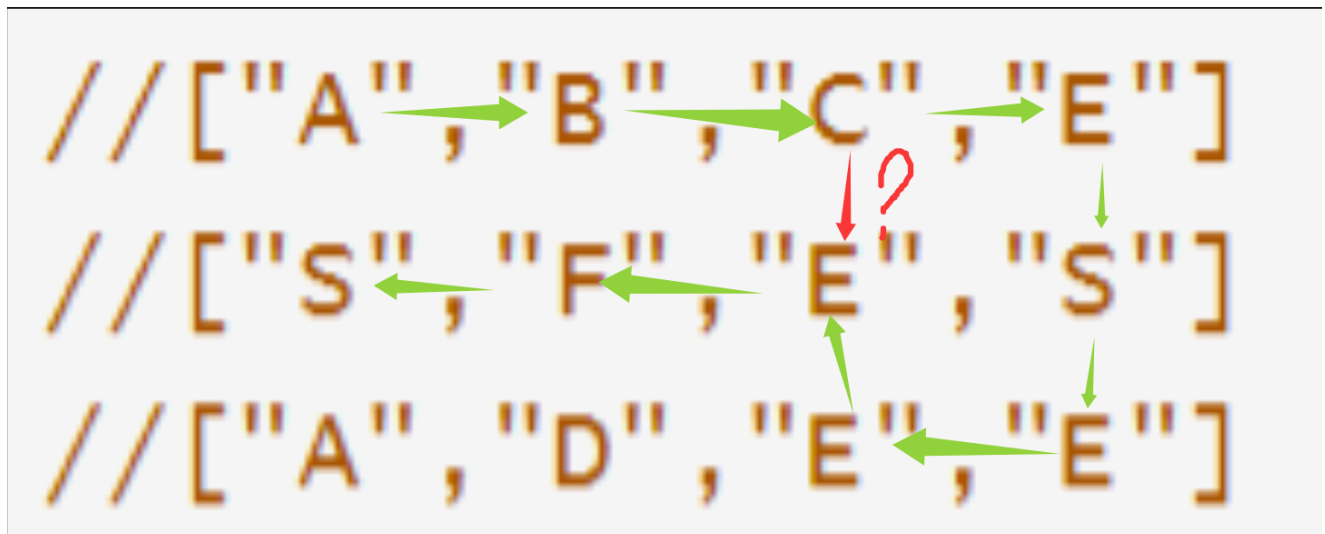
class Solution {
public:
    bool dfs(int x,int y,int index){
        if (x<0 || y<0 || x≥m || y≥n || vis[x][y]) return false;
        if (word2[index]≠board2[x][y]) return false;
        vis[x][y]=true;
        if (index==word2.size()-1) return true;
        for (int d=0;d<4;d++){
            auto nx = x+dx[d];
            auto ny = y+dy[d];
            if (dfs(nx,ny,index+1)) return true;
        }
        return false;
    }

    bool exist(vector<vector<char>>& board, string word) {
        m = board.size();
        n = board[0].size();
        board2 = board;
        word2 = word;
        for (int x=0;x<m;x++){
            for (int y=0;y<n;y++){
                if (board[x][y]==word[0]) {
                    vis.assign(m,vector(n,false));
                    if (dfs(x,y,0)) return true;
                }
            }
        }
        return false;
    }
private:
    string word2;
    vector<vector<char>> board2;
    vector<int> dx = {-1,1,0,0};
    vector<int> dy = {0,0,-1,1};
    int m,n;
    vector<vector<bool>> vis;
};
//

```

过了81/88个用例，看似逻辑天衣无缝，其实问题就在没撤销访问上面——为什么？

看着我没过的用例，我好像发现了什么，我们来观察一下：



没过的用例

其中，绿色箭头标注的是这个用例的正确路径(ABCESEEEFS)，按照我上面的逻辑，如果走到了C，也就是我打问号的地方，有两个E可以选择，但是选择下面的E，后面的字母就对不上；走右边的E，才是正确道路。

我的这一版代码，没有回溯撤销访问机制，就导致这条路能不能走通变成了看运气(dx dy数组是上下左右顺序，显然按照我的代码，我是先尝试下面的，于是就直接走不通，返回false)。然后走不通不要紧，但是要紧的是，我没有撤销我vis数组对应这个格子的访问状态，因此E这个格子会被标记为已访问，然后我走绿色正解路线时，就会发现，这个原本正确的道路居然已访问，然后返回false，导致答案错误。

当然，谁污染，谁治理，在哪里标记true，就在哪里标回false。代码改编如下，直接ac：

```
C++
bool dfs(int x,int y,int index){
    if (x<0 || y<0 || x≥m || y≥n || vis[x][y]) return false;
    if (word2[index]≠board2[x][y]) return false;
    vis[x][y]=true; //标记访问
    if (index==word2.size()-1) return true;
    for (int d=0;d<4;d++){
        auto nx = x+dx[d];
        auto ny = y+dy[d];
        if (dfs(nx,ny,index+1)) return true;
    }
    vis[x][y]=false; //撤销访问
    return false; //四个方向都尝试完毕了还没成功，那就是失败了，返回false
}
```

这是另一个版本的谁污染谁治理，因为我是更喜欢把判断逻辑放在函数的开头。之前写的都是和dfs调用的那一行靠在一起的，看似不一样，实则是一样的。

所以，这道题确实是个回溯题，这个用例给我点醒了，得给它后悔的机会。不能因为走错了一个格子，就抹杀它后面所有的可能性。

59-131 分割回文串

题目链接: <https://leetcode.cn/problems/palindrome-partitioning/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.12

标签: 字符串 回溯 动态规划

推荐题解: [方法一是我的思路来源](#)

哎, 这道题费了我老多时间了, 前前后后看了题解问了哈基米, 总算做出来了。

题解的方法一我觉得挺有意思, 把原本需要复杂处理"接在后面还是新起一个串"的类似于"最长上升子序列"的dp问题, 转化回了"这个逗号我是选还是不选"的经典到不能再经典的回溯问题。

当然, 这个逗号如果选, 也是有条件的, 这个子串在分隔之后必须得是回文串, 我才能选这个逗号。

```

class Solution {
public:
    //检查是否为回文的函数
    bool check(string &s, int left, int right) {
        while (left < right) {
            if (s[left++] != s[right--]) {
                return false;
            }
        }
        return true;
    }

    void dfs(int start, int i) {
        if (i == n) {
            ans.emplace_back(path);
            return;
        }
        //不在这一位的后面切
        if (i < n - 1) dfs(start, i + 1); //最后一位就必须切了
        //在这一位的后面切
        if (check(s2, start, i)) { //只有切了是回文的情况下才切
            //谁污染谁治理
            path.emplace_back(s2.substr(start, i - start + 1));
            dfs(i + 1, i + 1);
            path.pop_back();
        }
    }

    vector<vector<string>> partition(string s) {
        s2 = s;
        n = s.size();
        dfs(0, 0);
        return ans;
    }

private:
    vector<vector<string>> ans;
    vector<string> path;
    string s2;
    int n;
};

```

就讲到这里。

60-51N 皇后

题目链接: <https://leetcode.cn/problems/n-queens/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难

解题日期: 2026.2.13

标签: 数组 回溯

推荐题解: [是我的思路来源](#)

来到了我们hot100笔记的第7道困难题! 之前我其实也是跳过了一道接雨水, 这题听说也是非常恐怖, 等我全部题目做完了再去一探究竟。

这题的思路倒也不难, 虽说我一开始没想到, 但是看完题解秒懂:

1. 由于需要每行每列不重复, 因此可以把问题转化成: 我在第1,2,3,4行, 选择我列号的全排列, 例如2341,1423等等, 就转化成了全排列问题, 也就是我们回溯系列的第一题。
2. 然后还要求斜线也不能重复, 否则就会被攻击到。此时就可以利用一个特性: 左上右下斜线的各个点之间, 行号+列号相等; 右上左下斜线的各个点之间, 行号-列号相等。我们可以利用集合来做到这件事: 只有当前行号列号加减在集合里没出现并且元素本身之前也没出现, 才说明当前元素合法。
3. 然后生成了合法的序列之后, 再在主函数部分开个循环, 根据这个来拼最终返回的答案字符串。

上代码:

```

class Solution {
public:
    void dfs(){
        if (path.size()==m){
            paths.emplace_back(path);
            return;
        }
        for (int next=0;next<m;next++){
            //需要满足不重复, 斜线内无可互相攻击元素才可放入
            if (!pathSet.count(next) && !canBeAttacked.count(path.size()-1+next)
&& !canBeAttacked2.count(path.size()-1-next)){
                canBeAttacked.emplace(path.size()-1+next);
                canBeAttacked2.emplace(path.size()-1-next);
                path.emplace_back(next);
                pathSet.emplace(next);
                dfs();
                path.pop_back();
                pathSet.erase(next);
                canBeAttacked2.erase(path.size()-1-next);
                canBeAttacked.erase(path.size()-1+next);
            }
        }
    }

    vector<vector<string>> solveNQueens(int n) {
        m=n;
        //1.先获取所有合法的列的全排列
        dfs();
        //2.遍历全排列, 生成答案
        vector<vector<string>> finalAns;
        for (auto row:paths){
            vector<string> temps;
            for (auto col:row){
                string temp;
                for (int i=0;i<n;i++){
                    if (i==col){
                        temp+='Q';
                    }else{
                        temp+='.';
                    }
                }
                temps.emplace_back(temp);
            }
            finalAns.emplace_back(temps);
        }
        return finalAns;
    }

private:
    unordered_set<int> canBeAttacked, canBeAttacked2, pathSet;
    vector<int> path;
    vector<vector<int>> paths;
    int m;
};

```

我这个代码有很多可以优化的地方，并且速度和代码简洁程度远远不及位运算：

```
C++  
void dfs(int row, int cols, int ld, int rd) {  
    if (row == n) {  
        ans++; return;  
    }  
    // 获取当前行所有可用的位置 (0 表示被占, 1 表示可用)  
    int availablePositions = ((1 << n) - 1) & ~(cols | ld | rd);  
  
    while (availablePositions > 0) {  
        // 取出最低位的 1 (放一个皇后)  
        int position = availablePositions & -availablePositions;  
        availablePositions -= position;  
  
        // 递归下一行: ld 左移, rd 右移, 冲突位置自动随行变换  
        dfs(row + 1, cols | position, (ld | position) << 1, (rd | position) >>  
1);  
    }  
}
```

但是这个确实是我受启发之后，想出来的最朴素的思路了。

61-35 搜索插入位置

题目链接: <https://leetcode.cn/problems/search-insert-position/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.14

标签: 数组 二分查找

推荐题解: 无

总算是来到了我最熟悉的二分查找环节！顺带宣传一下我之前写好的二分题单题解: [算法笔记-二分算法已完结](#)

言归正传。这道题其实就是最简单的红蓝染色法的应用。先分类讨论一下：

1. 如果target存在于数组中，那我们最终需要指向target本身；
2. 如果不存在，那插入它的位置是 比target大的第一个数 的位置。

因此，根据红蓝染色法，r最终指向蓝区最后一个数，l最终指向红区第一个数。

因此，我们肯定不能返回r，只能返回l，来确保这个"第一个数"。按照惯例，我们设定入蓝区的条件为：小于target的数。这样，最终返回的l就会天然符合上面两个条件。

```
class Solution {  
public:  
    int searchInsert(vector<int>& nums, int target) {  
        int l=0,r=nums.size()-1;  
        while (l<=r){  
            int mid = l+(r-l)/2;  
            if (nums[mid]<target){ //划入蓝区  
                l = mid+1;  
            }else{  
                r = mid-1;  
            }  
        }  
        return l;  
    }  
};
```

就讲到这里，温故而知新。

62-74 搜索二维矩阵

题目链接: <https://leetcode.cn/problems/search-a-2d-matrix/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.14

标签: 数组 二分查找 矩阵

推荐题解: 无

这题也是自己做出来的，之前倒是没做到过。

注意哈，这道题和前面分治的 **19-240 搜索二维矩阵 II** 是完全不一样的。这道题可以理解成一层一层扫描，形成的最终一整条的序列是非递减的；而前面那道题如果这样扫描，是总体呈现上升趋势，但是每上升一段就有可能下降一次。

正是因为这道题严格非递减，所以我们可以把每一行看成一个区间，将每一行的第一个元素看成区间开头，对第一个元素这一列二分一次，就能定位到target所在的行；进入目标行后，再二分一次，就能揭晓target是否存在了。

```

class Solution {
public:
    bool searchMatrix(vector<vector<int>>& matrix, int target) {
        if (target<matrix[0][0]) return false;
        //1.先定位行
        int l=0,r=matrix.size()-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (matrix[mid][0]<target){
                l = mid+1;
            }else if (matrix[mid][0]==target){ //有可能在确定行的时候每行的第一个元素
                return true;
            }else{
                r = mid-1;
            }
        }
        int line = r;
        //2.在行里面找
        l=0;
        r=matrix[line].size()-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (matrix[line][mid]<target){
                l = mid+1;
            }else if (matrix[line][mid]==target){ //如果元素存在，是一定会进入这个分支
                return true;
            }else{
                r = mid-1;
            }
        }
        return false;
    }
};

```

恰巧就是

的

话说为啥要把等于的情况单独拉出来讨论呢？因为根据二分的性质，如果元素存在，mid在经过我"这个小了，去找大的；这个大了，去找小的"的反复修正后，是一定会指向这个元素的(注意：是指到这个元素，不是循环退出后还指这个元素)。如果循环都跑完了还没进入过这个分支，那一定就是没有，可以直接判false。

63-34 在排序数组中查找元素的第一个和最后一个位置

题目链接：<https://leetcode.cn/problems/find-first-and-last-position-of-element-in-sorted-array/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.14

标签：数组 二分查找

推荐题解：无

依旧两次二分查找就完事。

第一次找第一个元素，因此最终返回的是红区的第一个指针l，我们需要将 小于**target** 的元素框进蓝区；

第二次找最后一个元素，因此最终返回的是蓝区的最后一个指针r，我们需要将 小于等于**target** 的元素框进蓝区。

直接秒了，记得特判一下特殊情况：

```
C++
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        if (nums.size()==0) return vector<int>{-1,-1};
        //1.先找开始位置
        int l=0,r=nums.size()-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (nums[mid]<target){ //将小于target的元素框进蓝区
                l = mid+1;
            }else{
                r = mid-1;
            }
        }
        int first=l;
        //如果越界了，或者没越界但是最终指向的不是目标，那就说明没找到
        if (first<0 || first>=nums.size() || nums[first]!=target) return
vector<int>{-1,-1};
        //2.再找结束位置
        l=0;
        r=nums.size()-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (nums[mid]<=target){ //将小于等于target的元素框进蓝区
                l = mid+1;
            }else{
                r = mid-1;
            }
        }
        int second = r;
        //这里就不需要特判了，因为上面既然找到了，这里也一定不会出问题
        return vector<int>{first,second};
    }
};
```

就讲到这里。

64-153 寻找旋转排序数组中的最小值

题目链接: <https://leetcode.cn/problems/find-minimum-in-rotated-sorted-array/description/>

难度: 中等

解题日期: 2026.2.15

标签: 数组 二分查找

推荐题解: [我的思路来源](#)

我去, 这道题属于是超出我刷完一整个二分题单的基础题的势力范围了。那就现学现用吧。

这道题的重中之重, 是 把当前二分的值和数组的最后一个数对比。

1. 如果小于等于, 那么说明这个数要么是最小值, 要么是在最小值的右边。因此, 我们需要收缩右边界, 因为它右边的数都不可能是最小值了。
2. 如果大于, 那么说明这个数在第一段, 最小值在它右边。我们需要收缩左边界, 因为左边的数在第一段, 绝对不会是最小值。
3. 最后二分完毕, 引入红蓝染色法(貌似是变种)判断谁是最小值: 由于最小值一定是小于等于数组的最后一个数的, 在二分结束的最后一步, 肯定还是在触发1的条件, 收缩右边界, 然后右边界缩的比左边界还小了, 退出循环。此时很轻松的知道, l指向的就是最小值。和红蓝染色法判边界的步骤不太一样, 但是还是挺神似的。

上代码:

```
C++
class Solution {
public:
    int findMin(vector<int>& nums) {
        int n=nums.size(), l=0, r=n-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (nums[mid] <= nums[n-1]){ //如果小于等于尾巴的值, 说明nums要么是最小值, 要么在最小值右侧
                r = mid-1;
            }else{
                l = mid+1;
            }
        }
        return nums[l];
    }
};
```

就讲到这里。下一道题会直接用到这道题的思路。

65-33 搜索旋转排序数组

题目链接: <https://leetcode.cn/problems/search-in-rotated-sorted-array/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.15

标签: 数组 二分查找

推荐题解: [我的思路来源](#)

这道题我选择了题解的方法一: 用上一道题的思路, 分隔数组, 分成左右两部分。

再把target和最后一个数对比:

1. 若大于, 则说明在左部分;
2. 若小于, 则说明在右部分;
3. 若等于, 说明撞大运了, 直接返回n-1。

知道在哪部分后, 第二次二分就选择性初始化l和r, 正常二分即可。

上代码:

```

class Solution {
public:
    int search(vector<int>& nums, int target) {
        //1.先通过153题的方法找到最小值
        int n = nums.size(), l=0, r=n-1;
        while (l<=r){
            int mid = l+(r-l)/2;
            if (nums[mid]<=nums[n-1]){
                r = mid-1;
            }else{
                l = mid+1;
            }
        }
        int div = l;
        //2.判断target所在的段, 然后再次二分
        if (target<nums[n-1]){
            l=div;
            r=n-1;
        }else if (target==nums[n-1]){
            return n-1;
        }else{
            l=0;
            r=div-1;
        }
        while (l<=r){
            int mid = l+(r-l)/2;
            if (nums[mid]<target){
                l = mid+1;
            }else if (nums[mid]==target){
                return mid;
            }else{
                r = mid-1;
            }
        }
        return -1;
    }
};

```

就讲到这里。

66-20 有效的括号

题目链接: <https://leetcode.cn/problems/valid-parentheses/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.16

标签: 栈 字符串

推荐题解: 无

这道题也是非常经典了, 属于是高中信息技术就教过的很经典的栈的用途——括号匹配。

具体做法就是：如果遍历到左括号就丢进栈，然后如果遍历到右括号，就和栈顶对比，对得上就出栈，对不上或者栈空了就说明不合法。最后遍历完了如果栈还是非空依然不合法，只有栈刚好空了才合法。

代码如下：

```
C++
class Solution {
public:
    bool isValid(string s) {
        stack<char> st;
        unordered_map<char, char> cor={{'(', ')'},{'[', ']'},{'{', '}'}};
        for (char c:s){
            if (c=='(' || c=='{' || c=='['){
                st.push(c);
            }else{
                if (!st.empty() && c==cor[st.top()]){
                    st.pop();
                }else{
                    return false;
                }
            }
        }
        return st.empty();
    }
};
```

就讲到这里。

67-155 最小栈

题目链接：<https://leetcode.cn/problems/min-stack/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.16

标签：栈 设计

推荐题解：[我的思路来源](#)

又是一道设计题。挺惊讶的，看了一眼题解才发现能继续用stl的栈结构。也是，这道题考察的侧重点还是在 $O(1)$ 获取当前栈最小值。

思路挺简单：利用最小值的链式传递性质。在栈的每个元素后面维护一个当前元素为栈顶时的最小值，然后加入新元素时，取 旧栈顶元素的最小值 和 当前元素 二者的最小值为 当前的最小值 即可(是不是挺绕?)。

上代码：

```

class MinStack {
public:
    MinStack() {
        st.emplace(INT_MAX, INT_MAX);
    }

    void push(int val) {
        st.emplace(val, min(st.top().second, val));
    }

    void pop() {
        st.pop();
    }

    int top() {
        return st.top().first;
    }

    int getMin() {
        return st.top().second;
    }
private:
    stack<pair<int, int>> st;
};

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack* obj = new MinStack();
 * obj->push(val);
 * obj->pop();
 * int param_3 = obj->top();
 * int param_4 = obj->getMin();
 */

```

值得一提的是，上面代码用了emplace的部分，我用push会报错。解释如下：

st.push() 的逻辑：它要求你先提供一个完整的对象。如果你写 `st.push(INT_MAX, INT_MAX);`，编译器会报错，因为它认为你传了两个参数给一个只接受一个参数（一个 `pair` 对象）的函数。

- 正确写法：`st.push({INT_MAX, INT_MAX});`（加上大括号，手动构造一个 `pair`）。

st.emplace() 的逻辑：它是“就地构造”。它允许你把构造 `pair` 所需的参数直接传进去，它会在栈内部帮你把 `pair` 捏出来。

- 正确写法：`st.emplace(INT_MAX, INT_MAX);`（直接传两个参数，合法）。

所以，下次遇到类似情况，知道要怎么处理了吧？

ok，今天也是跳了上面的一道hard题目(实在是没心思看)，留着后面做，就写了这两道比较简单的题目。

大家除夕快乐！就讲到这里。

68-394 字符串解码

题目链接: <https://leetcode.cn/problems/min-stack/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.17-18

标签: 栈 递归 字符串

推荐题解: [我的灵感来源](#)

各位新年快乐! 这两天也是玩爽了, 所以刷题频率降低了, 这道题做了我两天——不过是一题两吃! 17号做了递归做法, 18号做了用栈模拟递归的做法。

咱们一个个来讲。

方法一: 递归做法

首先, 依旧搬出我们的 信任大法: 我们相信 这个函数能够将它托付给它的内容处理成符合要求的字符串。

那这样就很简单了, 我们只需要分类讨论当前字符串第一位是啥就行了, 一位一位啃掉第一位, 连循环都省了——一切交给函数递归栈来自动处理:

1. 先初始化一个临时字符串 `temp` 和一个数字 `num`, 前者用来拼, 后者用来记录次数。
2. 如果 `s` 为空, 返回空串;
3. 如果第一位是字母, 把后续字符串交给函数递归处理, 然后和这一位拼起来返回;
4. 如果第一位是数字, 那就初始化一个指针往后挪, 使用累乘相加大法, 把数字串转化成对应的数字, 最后 指针会停在左括号上; 接着, 精准找到与这个左括号对应的右括号, 把两个括号包裹的字符串, 直接交给这个函数进行递归调用, 让它递归处理这一坨字符串; 处理完之后拿到结果, 再用循环拼这个结果到`temp`后面; 最后处理完毕, 返回这个结果到上层。
5. 在这个递归函数的最后返回空串, 这是出自 语法要求和程序健壮性 的考虑, 正常情况不会走到这里。

思路很清晰了, 上代码:

```

class Solution {
public:
    string decodeString(string s) {
        if (s.size()==0) return "";
        if (s[0] >= 'a' && s[0] <= 'z'){
            return s[0]+decodeString(s.substr(1));
        }
        if (s[0] >= '0' && s[0] <= '9'){
            int p=0,num=0;
            //先解析出数字
            while (p<s.size() && s[p] >= '0' && s[p] <= '9'){
                num=num*10+(s[p]-'0');
                p++;
            }
            //此时p会停在第一个括号上
            //然后寻找与之匹配的右括号
            int balance=0,start=p;
            while (p<s.size()){
                if (s[p]=='['){
                    balance++; //这个是用来计算左右括号数量的，用于找到和当前括号匹配的右
                    括号
                }else if (s[p]==']'){
                    balance--;
                }
                if (balance==0) break;
                p++;
            }
            //此时p会停在右括号上
            auto end=p;
            //我们拼接子串
            string temp;
            auto target = decodeString(s.substr(start+1,end-start-1));
            for (int i=0;i<num;i++){
                temp+=target;
            }
            if (p<s.size()-1) temp+=decodeString(s.substr(end+1));
            return temp;
        }
    }
};

```

这个方法很直观，并且将信任法体现的淋漓尽致，不过代码还是复杂了点。

方法二：用栈模拟递归

上面的方法还是有局限性的，比如如果字符串过长会爆栈。此时，我们如果自己维护一个 `std::stack`，理论上只要你的内存够大，多长的串都不会爆栈，随便处理。

那么怎么写呢？其实就是模拟递归的过程。在函数体内进行递归调用时，我们相当于是把当前正在拼接的字符串先放一边去，然后新开一个字符串去处理被切出来的那部分 `s`，如此往复，直到到底了，处理完

了，再一个个拼起来。

C++

```
class Solution {
public:
    string decodeString(string s) {
        stack<pair<string,int>> st;
        string temp;
        int num=0;
        for (auto c:s){
            if (c>='a' && c<='z'){
                temp+=c;
            }else if (c>='0' && c<='9'){
                num=num*10+(c-'0');
            }else if (c=='['){ //如果遍历到左括号
                st.emplace(temp,num); //暂存当前工作状态，入栈
                temp=""; //相当于进入了新的递归函数，此时这两个变量都是初始化状态
                num=0;
            }else if (c==']'){ //如果遍历到右括号
                auto [str,n] = st.top(); //从栈里取出上一层的工作状态
                st.pop();
                for (int i=0;i<n;i++){ //拼接当前处理好的字符串到上一层的字符串
                    str+=temp;
                }
                temp=str; //此时，这个字符串就成为了这一层的工作状态
            }
        }
        return temp;
    }
};
```

哈基米是这样拆解我的代码的，我自己写完都有点迷糊，看完了之后加深了许多理解：

遇到字符 **a-z**：在当前工作台上画画（**temp += c**）。

遇到数字 **0-9**：记下接下来要重复几次（**num**）。

遇到 **[**（开始递归/进深层）：

1. 保存现场：把“目前为止已经画好的画”（**temp**）和“接下来要画几份”（**num**）锁进储物柜。
2. 清空工作台：拿一张全新的白纸（**temp = ""**）开始处理括号里的内容。

遇到 **]**（递归返回/回浅层）：

1. 取回现场：从储物柜拿出“之前的画”（**str**）和“重复次数”（**n**）。
2. 合并成果：把当前工作台上的成果重复 **n** 次，贴到“之前的画”后面。
3. 更新工作台：现在的成果变成了合并后的长画。

翻译一下我的代码和递归的关系：

```

// 你的栈操作                                // 对应的递归逻辑
st.emplace(temp, num);  =>  result = num * decodeString(内部子串);
temp = "";              =>  // 进入新函数, res 初始为 ""
num = 0;                 =>  // 准备记录新函数里的数字

// 遇到 ']'
auto [str, n] = st.top(); =>  // 函数 return 结果到上一层
str += repeat(temp, n);  =>  // 上层函数把返回的结果拼接到自己的前缀上
temp = str;              =>  // 更新上层函数的当前进度

```

完事。

69-739 每日温度

题目链接: <https://leetcode.cn/problems/daily-temperatures/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.19

标签: 栈 数组 单调栈

推荐题解: [这个视频刚开始看不懂, 看到后面茅塞顿开](#)

这道题确实比较新奇。我们需要引入一个新的概念——单调栈。

上面推荐的视频里面讲的挺好的。我们从后往前遍历, 想象我们站在数字对应的高度上, 把不同的数字当成绵延的山, 如果我们要寻找下一个最近的比我大的数, 直接从左往右看就行了。看不见的数, 我们自然就可以忽略。给定一个列表: `[2,3,5,2,3,7,8]`, 我站在3往右看, 肯定只能看到5,7,8, 对吧? 2和3就自动的被忽略了。这就是视频里面辅助我们分析的群山概念。

因此, 我们需要一个数据结构, 来快捷的维护一个最有潜力的候选数列表。其中最有力指的是: 相对位置更靠前, 并且数字本身更大。那我们就需要把靠前并且大的数留下来, 把靠后并且小的数给去掉。

那么为啥选择栈呢? 因为我们维持从栈底到栈顶的数为递减, 此时越靠近栈底的数就越大, 越不可能被拿掉, 如果它也被拿掉了, 那它前面的数一定更小, 更应该被拿掉。换句话说, 如果要从候选名单踢数, 第一个踢的肯定是栈顶的数, 因此数据的进出天然更符合"后进先出"的特性, 栈就很适合来做这个事情。

铺垫了这么多, 具体该如何做? 讲讲思路:

1. 首先新建一个栈, 存放数字本身及其下标(只存下标也行, 要用的时候从数组里面现场拿就行, 更省空间。当然我这样做比较直观);
2. 然后初始化一个值全都是0的, 长度和temps数组一样长的 `ans` 数组, 用于填充答案;
3. 接着我们从后往前遍历。咱们分类讨论:
4. 如果栈空, 那就要加元素进去, 毋庸置疑;
5. 如果栈非空, 就要看看栈顶的数和当前的数的大小了。如果栈顶的数小于当前的数, 说明它就是上面用例中, 咱们往左移动再也看不到的山谷, 属于是干扰项, 我们直接踢掉, 踢到栈顶的数比当前的数大为止, 也就是留下山顶。此时, 如果栈还是非空, 此时的栈顶元素就是下一个最近的比我们大的数;
6. 如果栈顶的数比当前的数大, 说明它就是下一个最近的比我们大的数;
7. 到这一步就搞定了, 直接取栈顶下标和当前数下标一减, 就是距离。

思路讲完了，上代码：

```
C++
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        stack<pair<int, int>> st;
        int n = temperatures.size();
        vector<int> ans(n, 0);
        for (int i = n - 1; i ≥ 0; i--) {
            while (!st.empty() && st.top().first ≤ temperatures[i]) {
                st.pop();
            }
            if (!st.empty()) {
                ans[i] = st.top().second - i;
            }
            st.emplace(temperatures[i], i);
        }
        return ans;
    }
};
```

思路依然是上面的思路，只不过我观察了逻辑，整合优化了一下，减少了不少代码量。

那就讲到这里吧！

70-84 柱状图中最大的矩形

题目链接：<https://leetcode.cn/problems/largest-rectangle-in-histogram/description/?envType=study-plan-v2&envId=top-100-liked>

难度：困难

解题日期：2026.2.20

标签：栈 数组 单调栈

推荐题解：[灵感来源](#)

来到我们hot100笔记的 第8个困难题！其实就是 **69-739 每日温度** 这道题的变种。

在上面这道题，我们学习到了如何用 单调栈 找从左往右看的 山峰。而这道题，我们要找 山谷。

为啥呢？先看看如何计算面积。我们从左到右遍历这个数组，枚举当前高度。那么此时，我们就需要寻找合法的宽度。合法的宽度怎么找？左顾右盼：

1. 先看左边，找到最近的比当前柱子低的柱子，此时说明从这个柱子的右边那一根柱子开始到当前枚举高度的柱子这一段，柱子的高度全都 $\geq$ 当前柱子，所以我们完全可以用当前柱子当高度，然后左边找到的低柱子右边那个柱子作为这一片区域的左边界；
2. 再看右边，同理。不过，我们需要将找到的低柱子的左边那根柱子作为这一片区域的右边界；
3. 左右边界都找到了，计算出宽度，面积就出来了。

找山谷和找山峰同理，把上一题代码里面的 $\geq$ 改成 $\leq$ 就完事了。我们可以遍历3次，职责清晰：

1. 第一次倒序遍历，找从左往右看每个数第一个比自己小的数的位置；

2. 第二次正序遍历，找从右往左看每个数第一个比自己小的数的位置；
3. 上述两次遍历的下标全都存进数组中，便于下一次遍历使用；
4. 第三次正序遍历或者倒序遍历都行，枚举高度，然后从数组中取离当前数左边或者右边最近的小于当前数的数的下标，即可计算出宽度。然后算面积维护最大值就完事了。

还要注意一点：如果一个数本身比较小，向左或者向右单个方向看可能存在找不到小于它的最近的数的情况，此时，如果选它作为高度，就可以把面积边界一直向左或者向右延伸到数组的边界。我们可以初始化存储 向左看小于当前数的数的下标的数组 的初始值为-1，向右的为n，这样就能在右偏或者左偏一位的时候自然参与计算，无需复杂的条件判断。

上代码：

```
C++
class Solution {
public:
    int largestRectangleArea(vector<int>& heights) {
        //1.先从右到左遍历，找从左往右看的山峰
        stack<int> stRL, stLR;
        int n = heights.size();
        vector<int> right(n, n), left(n, -1);
        for (int i = n - 1; i >= 0; i--) {
            while (!stRL.empty() && heights[stRL.top()] >= heights[i]) {
                stRL.pop();
            }
            if (!stRL.empty()) {
                right[i] = stRL.top();
            }
            stRL.emplace(i);
        }
        //2.再从左到右遍历，找从右往左看的山峰
        for (int i = 0; i < n; i++) {
            while (!stLR.empty() && heights[stLR.top()] >= heights[i]) {
                stLR.pop();
            }
            if (!stLR.empty()) {
                left[i] = stLR.top();
            }
            stLR.emplace(i);
        }
        //3.最后枚举高度，左顾右盼找宽度，维护面积最大值
        int maxSize = -1;
        for (int i = 0; i < n; i++) {
            //先找左右两边的合法下标
            auto l = left[i] + 1; //0+1自然变成1
            auto r = right[i] - 1; //n-1自然变成数组最后一位n-1
            auto width = r - l + 1; //因此无需判断
            maxSize = max(maxSize, width * heights[i]);
        }
        return maxSize;
    }
};
```

就讲到这里。当然还有二次甚至一次遍历的做法，这个等我后续来刷第二遍的时候如果还有雅兴，再来拓展。

71-215 数组中的第K个最大元素

题目链接: <https://leetcode.cn/problems/kth-largest-element-in-an-array/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.20

标签: 数组 分治 快速选择 排序 堆 (优先队列)

推荐题解: [快速选择方法的灵感来源](#)

这道题居然用堆做, 还要求 $O(n)$? 相信我, 在你使用优先队列的那一刻, 你就输了(虽说不会超时, 但是时间复杂度为 $O(n\log k)$)。

这题做法其实有挺多的:

1. 直接调用 `std::sort()` 排序, 然后取 `nums[n-k]` 即可;
2. 使用 `priority_queue<int>`, 大根堆。先塞进去全部数, 再弹出堆顶 `k-1` 次, 此时堆顶就是第k大;
3. 使用 `priority_queue<int, vector<int>, greater<int>>`, 小根堆。先塞进去k个数, 然后后面n-k个数一边塞数一边弹堆顶, 这样剩下的k个数就是最大的k个数, 因为小的都被弹走了。完事后的堆顶就是第k大的数;
4. 使用快速选择, 唯一 $O(n)$ 正解。

我们讲讲3和4。

方法一: 使用小根堆(时间复杂度 $O(n\log k)$)

上面思路已经讲了, 上代码:

```
C++
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for (auto x:nums){
            pq.emplace(x);
            if (pq.size()>k){
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

方法二: 快速选择(真正的正解)

其实就是快速排序稍微改一下。快速排序是需要在用基准分隔左右后，分治分别处理左边和右边。而快速选择只需要在确定基准的位置后，判断它和我们要找的k的关系：

1. 如果比 $n-k$ 小，说明第k大的数在它右边，咱们递归分治右边即可；
2. 反之递归分治左边。

所以效率会比快速排序高很多。

而快速排序，或者说快速选择的逻辑，就和数据结构学到的是一样的：

1. 定基准。和平时做题模拟不一样，我们写程序最好别带着基准跑来跑去，可以把它备份了然后挪到第一位；
2. 接着，我们在第一位挖个坑，从右往左遍历，如果找到了比基准小的数，就填进这个坑里面，此时，坑来到了这个右指针停下来的位置；
3. 然后，再从左往右遍历，如果找到了比基准大的数，就填进新的坑里面；
4. 如此往复，直到左指针和右指针相等，此时的坑就是基准应该去到的位置。
5. 然后我们再用上面的逻辑，判断基准的位置和k大元素的位置，再去分治。有点像二分查找，大大降低了问题的规模。

思路讲完了，上代码：

```

class Solution {
public:
    void quickChoose(vector<int>& nums, int l, int r) {
        if (l > r)
            return;
        int pivotPos = rand() % (r - l + 1) + l; // 随机选择一个数作为基准
        int pivot = nums[pivotPos];
        swap(nums[l], nums[pivotPos]); // 我们写程序最好别带着基准跑来跑去，可以把它备份
了然后挪到第一位
        int left = l, right = r;
        while (left < right) {
            while (left < right && nums[right] ≥ pivot)
                right--;
            if (left < right)
                nums[left] = nums[right];
            while (left < right && nums[left] ≤ pivot)
                left++;
            if (left < right)
                nums[right] = nums[left];
        }
        nums[left] = pivot;
        // 分析位置关系，然后分治
        if (left < n - k2) {
            quickChoose(nums, left + 1, r);
        } else if (left == n - k2) {
            ans = pivot;
            return;
        } else {
            quickChoose(nums, l, left - 1);
        }
    }

    int findKthLargest(vector<int>& nums, int k) {
        nums2 = nums;
        k2 = k;
        n = nums.size();
        srand(time(0)); // 初始化随机种子
        quickChoose(nums, 0, n - 1);
        return ans;
    }

private:
    vector<int> nums2;
    int ans = 0;
    int n, k2;
};

```

那么，这里为啥要引入随机数呢？请看哈基米的解释：

1. 致命的 $O(n^2)$ ：为什么“固定选第一个”会死？

在你之前的逻辑中，如果不加随机数，通常会默认选 `nums[l]`（区间第一个数）作为基准。

如果输入数组是已经排好序的（例如 `[1, 2, 3, 4, 5]`），而你偏偏要找最大的元素：

1. 第一轮：选 `1` 做 Pivot。Partition 后发现 `1` 就在原地。剩下 `[2, 3, 4, 5]`。
2. 第二轮：选 `2` 做 Pivot。剩下 `[3, 4, 5]`。
3. ...以此类推。

每次你只能排除一个元素，而不是像理想状态那样排除一半。

- 比较次数： $n + (n - 1) + (n - 2) + \dots + 1 \approx \frac{n^2}{2}$ 。
- 复杂度：从理想的 $O(n)$ 瞬间退化到了 $O(n^2)$ 。

对于 $n = 10^5$ 的数据量， 10^{10} 的运算量会让你的程序直接超时（TLE）。

2. 随机数的作用：打破“有序”的诅咒

随机数就像是在抽奖前“洗牌”。无论输入数据是升序、降序，还是某种特殊的规律分布，随机选择基准值都能保证：

- 在概率学上，我们大概率能选到一个“中位数”附近的数。
- 即使偶尔选到了最小值或最大值，也不可能次次都选到。

结论：引入随机化后，快速选择的期望时间复杂度是 $O(n)$ 。没有任何一组固定的测试用例能稳定地让你的程序跑出 $O(n^2)$ ，因为每次运行时的 Pivot 选择都是不一样的。

完事！就讲到这里。

72-347 前 K 个高频元素

题目链接：<https://leetcode.cn/problems/top-k-frequent-elements/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.21

标签：数组 哈希表 分治 桶排序 计数 快速选择 排序 堆（优先队列）

推荐题解：[我的灵感来源](#)

这道题，做法非常多样。我第一反应想到的是哈希表计数+小根堆一个个弹出答案，和上面这道题类似。

但是我忽略了：这道题其实有一个直接能把时间复杂度压到 $O(n)$ 的神级做法：哈希表计数+桶排序。我们来讲讲这个。

基本思路是这样的：

1. 一切方法的开头：用 `unordered_map<int,int> cnt` 来统计频率，同时统计最高频率 `maxCnt`，作为下面桶的容量；
2. 接着，我们新建一个桶的二维数组 `vector<vector<int>> bucket(maxCnt+1,vector<int>())`，然后遍历 `cnt`，把对应频率的数丢进对应的桶里面；
3. 最后倒序遍历桶，时刻检查 `ans` 数组大小，达到k就直接返回。

上代码：

```
class Solution {  
public:  
    vector<int> topKFrequent(vector<int>& nums, int k) {  
        //1.哈希表先计数  
        unordered_map<int,int> cnt;  
        int n = nums.size();  
        int maxCnt = 0;  
        for (auto x:nums){  
            cnt[x]++;  
            maxCnt=max(maxCnt,cnt[x]);  
        }  
        //2.桶排序  
        vector<vector<int>> bucket(maxCnt+1,vector<int>());  
        for (auto& [k,v]:cnt){  
            bucket[v].emplace_back(k);  
        }  
        //3.输出答案即可  
        vector<int> ans;  
        for (int i=maxCnt;i>0;i--){  
            for (auto &x:bucket[i]){  
                ans.emplace_back(x);  
                if (ans.size()==k) return ans;  
            }  
        }  
        return ans;  
    }  
};
```

对比一下主流方案：

方案	处理流程	时间复杂度	理由
全排序	哈希统计 + 排序所有频率	$O(n\log n)$	没利用频率分布的特性。
小顶堆	哈希统计 + 维护大小为 k 的堆	$O(n\log k)$	进阶要求优于 $O(n\log n)$ ，这算过关。
桶排序	哈希统计 + 扔进桶里 + 倒序取桶	$O(n)$	频率范围固定，利用空间换取了极速。

这就是极致的空间换时间战略。

73-295 数据流的中位数

题目链接: <https://leetcode.cn/problems/find-median-from-data-stream/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难

解题日期: 2026.2.21

标签: 设计 双指针 数据流 排序 堆 (优先队列)

推荐题解: [灵神还得是灵神啊, 很简洁的思路!](#)

来到了我们hot100系列的 第9个困难题! 这题说实话, 不是很难, 设定为中等比较好。

不过也说是设计题, 要想到题解这么简洁的思路也是有难度。好在题解本身比较好理解, 做过一次之后, 下一次应该就会了。

环境设定 是这样的:

1. 利用中位数能够均匀将数组分隔成两半的特性, 维护小于中位数的部分为大根堆 `left`, 大于中位数的部分为小根堆 `right`, 然后分类讨论;
2. 如果总数量是奇数, 我们就规定大根堆比小根堆多一个数——换句话说, 中位数此时规定在 `left` 大根堆的堆顶;
3. 如果是偶数, 就正常相等即可;

接下来我们来处理 插入操作:

1. 先分类讨论。如果当前总数量是奇数, 那我们就得 保证 `left` 总数不变, `right` 总数加一, 否则会打破这个规定。那么如何做到呢?
2. 题解做了一个 很巧妙的操作: 把这个数丢进 `left` 里面, 然后再把堆顶弹出(也就是左区域最大的数), 放入 `right` 中。
3. 这样 完美兼顾了两种情况: 如果这个新数本来就属于左边, 那就放进去了, 然后把大的 挤 到右边; 如果这个数本来就属于右边, 那就放进去这个数, 再弹出来这个数, 也能达到目的。这个做法真的是非常巧妙。
4. 同理, 如果当前总数是偶数, 也就是左右堆的数量相等, 那就反过来, 我们需要保证 `left` 比 `right` 多一个数, 那就把数丢进 `right`, 然后再弹出其堆顶加入 `left`。

插入操作都处理完了, 通过上面的逻辑, 我们能非常方便的保持两个堆为约定俗成的状态(见上方环境设定)。

那么 获取中位数 的操作就好说了:

1. 由于题目确保在插入数后才会获取中位数, 这样连判空逻辑都省了, 简单粗暴;
2. 如果左堆比右堆多一个数, 那就直接以左堆顶为中位数;
3. 如果左右堆数量相等, 那就直接取左右堆顶然后取平均为中位数。

上代码:

```

class MedianFinder {
public:
    MedianFinder() {}

    void addNum(int num) {
        if (right.size()==left.size()){
            right.emplace(num);
            left.emplace(right.top());
            right.pop();
        }else if (right.size()+1==left.size()){
            left.emplace(num);
            right.emplace(left.top());
            left.pop();
        }
    }

    double findMedian() {
        int l=0,r=0;
        if (right.size()==left.size()){
            return (double)(right.top()+left.top())/2;
        }else{
            return left.top();
        }
    }
private:
    priority_queue<int> left; //大根堆
    priority_queue<int,vector<int>,greater<int>> right; //小根堆
};

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder* obj = new MedianFinder();
 * obj->addNum(num);
 * double param_2 = obj->findMedian();
 */

```

甚至都用不着构造函数，没绷住。

OK，就讲到这里。

74-121 买卖股票的最佳时机

题目链接：<https://leetcode.cn/problems/best-time-to-buy-and-sell-stock/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.22

标签：数组 动态规划

推荐题解：[思路来源](#)

不是，我真服了，被这个简单题反将一军了。

贪心算法的精髓，就是追求局部最优解。虽然这题我没怎么看出哪里在追求局部最优解了。

基本思路：低买高卖。同时，为了防止"先卖后买"，我们维护一个价格最低值和一个价格最高值，其中最低值是截至当前价格的前一个的最低值，此时一边遍历一边计算最大值即可。

```
class Solution {  
public:  
    int maxProfit(vector<int>& prices) {  
        int minPrice=prices[0],maxProfit=0;  
        for (auto x:prices){  
            maxProfit=max(maxProfit,x-minPrice); //细节先计算，防止同天买同天卖(其实也  
            不会影响结果哈哈，这样写比较规范)  
            minPrice=min(minPrice,x);  
        }  
        return maxProfit;  
    }  
};
```

就讲到这里。

75-55 跳跃游戏

题目链接：<https://leetcode.cn/problems/jump-game/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.22

标签：贪心 数组 动态规划

推荐题解：[思路来源，偷看了一眼标题秒懂](#)

这题的做法，简单粗暴：如果能到达这里，就基于当前位置和能跳跃的最远距离，维护最远可达的下标。

这就很能体现贪心算法了——追求局部最优解。

```
class Solution {  
public:  
    bool canJump(vector<int>& nums) {  
        int maxDist=0;  
        for (int i=0;i<nums.size();i++){  
            if (maxDist>=i) maxDist=max(maxDist,i+nums[i]); //如果可达，维护最远可到  
            达右侧下标  
        }  
        return maxDist>=nums.size()-1;  
    }  
};
```

就讲到这里。

76-45 跳跃游戏 II

题目链接: <https://leetcode.cn/problems/jump-game-ii/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.22

标签: 贪心 数组 动态规划

推荐题解: [思路来源](#)

我去, 这题确实是有点卡住我了。好在最后结合哈基米的帮助, 还是看懂了。

基本思路就这样: 在我被迫要做决定时(到达当前必须要跳的边界), 利用之前收集到的信息(不断更新维护最远距离), 做出局部最优解(选这个最远距离, 跳的最远, 然后更新新的边界为这个距离)。

```
class Solution {  
public:  
    int jump(vector<int>& nums) {  
        int maxDist=nums[0], curDist=0, cnt=0;  
        for (int i=0; i<nums.size()-1; i++){  
            maxDist=max(maxDist, nums[i]+i); // 不断收集信息: 能跳到的最远距离  
            if (i==curDist){ // 到达边界了, 必须要做选择了  
                cnt++; // 计数器加一, 代表我跳了一次  
                curDist=maxDist; // 更新新的边界  
            }  
        }  
        return cnt;  
    }  
};
```

C++

顺带一提, 我其实一开始写错了顺序, 这是哈基米帮我纠正的文本, 以示警醒:

简单来说: 你应该先“看远方”, 再“决定跳”。而你的代码顺序是“先决定跳, 再看远方”。

1. 致命的“顺序”问题

在贪心算法里, 当你走到当前边界 `curDist` 时, 你应该已经观察过了这一路所有的潜力点, 然后挑一个最远的跳过去。

你的代码逻辑:

1. 走到 `i`。
2. 如果 `i` 刚好是边界, 立刻跳, 把边界更新为 旧的 `maxDist`。
3. 然后 才更新 `maxDist` (把当前位置 `i` 的潜力算进去)。

这会导致什么问题?

如果当前位置 `i` 本身就是那个能带你飞得最远的点, 你的代码会先用“旧的、比较近的”潜力值更新边界, 从而错过了 `i` 这个最强潜力点。

这下记住了！希望下次不要出错。

77-763 划分字母区间

题目链接：<https://leetcode.cn/problems/partition-labels/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.23

标签：贪心 哈希表 双指针 字符串

推荐题解：[我的题解，船新做法！](#)

这道题还是挺惊喜的，居然让我意外发掘出新做法(见上面推荐题解)，感兴趣的同学可以前往力扣看我发的题解，这里就不再赘述了。

当然，这道题作为跳跃游戏2的变种，还是得学会用老方法解题：第一遍遍历统计每个字母最后出现的下标，然后第二遍遍历根据遍历到的字母的最远下标不断更新当前最远下标(也就是边界)，直到到达边界，说明当前区域内全部字母都已经没了，直接切断。

```
class Solution {
public:
    vector<int> partitionLabels(string s) {
        //1.先记录每个字母出现的最远下标
        vector<int> maxPos0
;
        for (int i=0;i<s.size();i++){
            maxPos[s[i]-'a']=max(maxPos[s[i]-'a'],i);
        }
        //2.再次遍历，不断维护最远下标
        int curMax=0,count=0;
        vector<int> ans;
        for (int i=0;i<s.size();i++){
            curMax=max(curMax,maxPos[s[i]-'a']);
            count++;
            if (i==curMax){
                ans.emplace_back(count);
                count=0;
            }
        }
        return ans;
    }
};
```

C++

顺带一提我的做法(忍不住提一嘴)，更倾向于"全收集"思路，有点像滑动窗口那边的做法。

对比一下：

比较维度	我的“全收集”解法	“跳跃游戏”式解法
核心比喻	装箱核对：每种零件必须配齐总数才发货。	修桥延申：不断推高当前片段必须到达的最远终点。
核心逻辑	关注 数量 (Quantity)：判断当前片段内字符的“已见频率”是否等于“全局频率”。	关注 位置 (Position)：判断当前指针是否走到了当前所有字符的“最后出现位置”。
判定切分时机	<code>if (check(cnt2, cnt))</code> 即：所有已出现的字符都拿够了。	<code>if (i == end)</code> 即：指针走到了预设的最远边界。
预处理内容	统计每个字符的 出现总次数 (Frequency)。	记录每个字符 最后一次出现的下标 (Last Index)。
关键变量	两个哈希表/数组 (全局总数 vs 当前累积)。	一个数组 (最后下标) + 一个变量 (当前最远边界 <code>end</code>)。
代码直观度	极高：逻辑非常符合人类的第一直觉 (拿全即走)。	中等：需要对“区间”和“边界”有较强的空间想象力。
时间复杂度	$O(n)$ ：两遍扫描， <code>check</code> 复杂度为 $O(26)$ 。	$O(n)$ ：两遍扫描。
性能瓶颈	在每个位置都要做 $O(26)$ 的状态对比 (虽然是常数，但比 <code>max</code> 重)。	极快：每个位置只做一次 <code>max</code> 比较。

就讲到这里。

78-70 爬楼梯

题目链接：<https://leetcode.cn/problems/climbing-stairs/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.23

标签：记忆化 数学 动态规划

推荐题解：无

基本思路就不讲了，主要是明确一下 `dp[0]` 的定义：

1. 为了满足状态转移方程 (数学意义)

我们看你的递推公式：

$$dp[i] = dp[i - 1] + dp[i - 2]$$

当你计算 $dp[2]$ (爬到 2 阶的方案) 时：

- 实际方案有两种：1阶+1阶 和 直接跨2阶。所以 $dp[2]$ 应该等于 2。
- 根据公式： $dp[2] = dp[1] + dp[0]$ 。
- 既然 $dp[1] = 1$ ，那么为了让 $dp[2] = 2$ 成立， $dp[0]$ 必须定义为 1。

2. “不走也是一种走” (逻辑意义)

在组合数学中，这是一种常见的定义方式，类似于 $0! = 1$ （0的阶乘是1）或 $x^0 = 1$ ：

- $dp[i]$ 的定义是“爬到第 i 阶的方案数”。
- $dp[0]$ 表示“爬到 0 阶”或者说“站在原点不动”的方案数。
- 不动也是一种方案，且只有这一种方案（就是老实实在原地待着）。

3. 支撑“直接跨 2 步”的操作 (物理意义)

从原点（0阶）出发，你有两种选择：

1. 迈 1 步到 1 阶：这对应从 $dp[0]$ 转移到 $dp[1]$ 。
2. 直接迈 2 步到 2 阶：这对应从 $dp[0]$ 直接跳到 $dp[2]$ 。

为了让这次“跨 2 步”被计入总数，起点 $dp[0]$ 必须提供一个基数 1。如果 $dp[0] = 0$ ，那么“直接迈 2 步”这个方案就被抹杀了。

所以，有的代码会考虑直接从 $dp[1]$ 开始，因为这样更符合直觉，但是需要多写判断。现在来看，我还是更喜欢“不走也是一种走”的理解方式。

```
class Solution {
public:
    int climbStairs(int n) {
        vector<int> dp0
;
        dp[0]=1;
        dp[1]=1;
        for (int i=2;i<=n;i++){
            dp[i]=dp[i-1]+dp[i-2];
        }
        return dp[n];
    }
};
```

C++

就讲到这里。

79-118 杨辉三角

题目链接：<https://leetcode.cn/problems/pascals-triangle/description/?envType=study-plan-v2&envId=top-100-liked>

难度：简单

解题日期：2026.2.23

标签：数组 动态规划

推荐题解：无

这道题就是简单题简单做法了：把三角靠左拉平，你会发现其实当前数就等于左上方的数+上方的数，如果没有就不加。

C++

```

class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> dp(numRows, vector<int>());
        dp[0]=vector<int>{1};
        for (int i=1; i<numRows; i++){
            for (int j=0; j<=i; j++){
                int temp=0;
                if (j-1>=0) temp+=dp[i-1][j-1];
                if (j<=i-1) temp+=dp[i-1][j];
                dp[i].emplace_back(temp);
            }
        }
        return dp;
    }
};

```

哈基米会推荐我通过初始化每一行为指定大小，并且全部填写1，来避开边界讨论，确实更优雅：

C++

```

class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> res(numRows);
        for (int i = 0; i < numRows; ++i) {
            // 关键点: 直接 resize 并初始化为 1
            res[i].resize(i + 1, 1);

            // 只需要计算中间的元素, 天然避开了边界 (0 和 i)
            for (int j = 1; j < i; ++j) {
                res[i][j] = res[i - 1][j - 1] + res[i - 1][j];
            }
        }
        return res;
    }
};

```

就讲到这里。

80-198 打家劫舍

题目链接: <https://leetcode.cn/problems/house-robber/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.23

标签: 数组 动态规划

推荐题解: 无

非常经典的基本题型！2分钟一遍过带注释bug-free代码：

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int n=nums.size();
        vector<vector<int>> dp(n,vector<int>{0,0});
        dp[0][1]=nums[0];
        for (int i=1;i<n;i++){
            //如果偷这家, 那上一家不能偷
            dp[i][1]=dp[i-1][0]+nums[i];
            //如果不偷这家, 上一家可偷可不偷
            dp[i][0]=max(dp[i-1][0],dp[i-1][1]);
        }
        return max(dp[n-1][0],dp[n-1][1]);
    }
};
```

就讲到这里。

81-279 完全平方数

题目链接: <https://leetcode.cn/problems/perfect-squares/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.23

标签: 广度优先搜索 数学 动态规划

推荐题解: 无

这道题还是非常经典的, 我自己都忘记怎么做了。hot100这个安排题目的顺序也有问题, 怎么连01背包都没接触就直接上完全背包了?

这题主要的马甲是: 商品一开始没初始化。只要你把 商品预处理好, 就完全是完全背包的逻辑。

至于展开逻辑, 让哈基米来吧! 我看了觉得不错:

我们可以把题目转化一下逻辑：

- 物品 (Items) : 所有的完全平方数 (1, 4, 9, 16, ...) 。
- 背包容量 (Capacity) : 整数 n 。
- 物品价值/代价 : 每个物品的“重量”是它的数值，而“价值”是 1 (因为我们计算的是数量) 。
- 目标 : 装满容量为 n 的背包，所用的 物品数量最少 。

1. 深度拆解 DP 表的含义

在你的代码中，`dp[i+1][j]` 的定义是：使用前 i 个完全平方数，凑出和为 j 所需的最少个数。

为什么是 `dp[m+1][c+1]` ？

- m 是可用平方数的个数。
- c 是目标和 n 。
- 多出来的 $+1$ 是为了处理“不选任何数”和“和为 0”的边界情况。

初始化 `INT_MAX` 的意义

- 因为我们要找的是 最小值，所以先把所有方案初始化为无穷大。
- `dp[0][0] = 0` 是唯一的起点：凑出和为 0 不需要任何数字，数量为 0 。

2. 状态转移方程的逻辑

这是你代码中最核心的部分：

- 情况 A：当前平方数 x 太大了。
如果当前的平方数 x 比我们要凑的和 j 还要大，那我们肯定没法用它。只能沿用“不用这个数”时的结果，即 `dp[i][j]` (上一行) 。
- 情况 B：可以选择用或者不用 x 。
我们要从两个策略里选个最小的：
 1. 不用它 : `dp[i][j]` 。
 2. 至少用它一次 : `dp[i+1][j-x] + 1` 。

注意点：这里是 `dp[i+1]` 而不是 `dp[i]` 。这正是 完全背包 的精髓——因为你可以无限次使用同一个平方数。所以当你用了一次 x 后，你依然可以在当前这一行 (包含 x 的集合里) 继续寻找剩余 $j-x$ 的最优解。

上代码：

```
class Solution {
public:
    int numSquares(int n) {
        //1.先制造商品
        vector<int> nums;
        for (int i=1;i*i≤n;i++){
            nums.emplace_back(i*i);
        }
        //2.再dp
        int m=nums.size(),c=n;
        vector<vector<int>> dp(m+1,vector<int>(c+1,INT_MAX));
        dp[0][0]=0;
        for (int i=0;i<m;i++){
            auto x=nums[i];
            for (int j=0;j≤c;j++){
                if (j-x<0) dp[i+1][j]=dp[i][j];
                else dp[i+1][j]=min(dp[i][j],dp[i+1][j-x]+1);
            }
        }
        return dp[m][n];
    }
};
```

就讲到这里。

82-322 零钱兑换

题目链接: <https://leetcode.cn/problems/coin-change/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.23

标签: 广度优先搜索 数组 动态规划

推荐题解: 无

是上面这道题的变种。思路几乎一致,但是:因为这道题可能出现凑不出来的情况,需要在继承上一层状态之前判断一下是否为 `INT_MAX`, 或者说是判断一样到底能不能到达上一层,不然我上一层的这个状态我都到不了,我肯定不能继承啊。

别的逻辑都是一样的,不多说。

```

class Solution {
public:
    int coinChange(vector<int>& coins, int amount) {
        int n=coins.size(),c=amount;
        vector<vector<int>> dp(n+1,vector<int>(c+1,INT_MAX));
        dp[0][0]=0;
        for (int i=0;i<n;i++){
            auto x=coins[i];
            for (int j=0;j<=c;j++){
                dp[i+1][j]=dp[i][j]; //这一行是无条件执行的，把上一层的数据直接搬过来，
                否则会丢状态。
                if (j-x>=0 && dp[i+1][j-x]<INT_MAX) dp[i+1][j]=min(dp[i]
[j],dp[i+1][j-x]+1);
            }
        }
        if (dp[n][c]==INT_MAX) return -1;
        return dp[n][c];
    }
};

```

讲讲这个"兜底"机制，挺有意思的。结论：都会兜底，只是上一题的写法和这一题不一样。

1. 逻辑等价性对比

你在《完全平方数》里的写法（`if-else` 派）：

```
C++
if (j - x < 0) {
    // 情况 A: 数太大, 装不下
    dp[i+1][j] = dp[i][j]; // <--- 这里的“显式赋值”就是兜底
} else {
    // 情况 B: 能装下, 就在“不装”和“装”之间选个小的
    dp[i+1][j] = min(dp[i][j], dp[i+1][j-x] + 1); // <--- min 里的第一个参数 dp[i][j] 也是兜底
}
```

你在《零钱兑换》里的写法（“先抄再改”派）：

```
C++
// 1. 先不管三七二十一, 把上一行的状态“抄”下来
dp[i+1][j] = dp[i][j]; // <--- 这里的“预赋值”就是兜底

// 2. 如果能装下, 再拿刚才抄的值和新方案比
if (j - x ≥ 0 && dp[i+1][j-x] < INT_MAX) {
    dp[i+1][j] = min(dp[i+1][j], dp[i+1][j-x] + 1);
    // 注意: 这里的 dp[i+1][j] 已经是第一步抄过来的 dp[i][j] 了
    // 所以这样写也行:
    // dp[i+1][j] = min(dp[i][j], dp[i+1][j-x] + 1);
}
```

结论：在《完全平方数》的代码里，不管走 `if` 还是 `else`，`dp[i][j]`（上一行的状态）都必然参与了计算。所以，它并没有漏掉兜底，只是把兜底逻辑拆到了两个分支里。

2. 为什么你会觉得它“没有兜底”？

可能是因为在 `else` 分支里，`dp[i][j]` 隐藏在了 `min` 函数的参数中。

- 当 $j - x \geq 0$ 时，你并没有直接写 `dp[i+1][j] = dp[i][j]`。
- 但 `min(dp[i][j], ...)` 的逻辑本质上是：“如果用这个新数的方案还没我上一行的方案好，那我就继续用上一行的方案。”

这其实是一种更紧凑的写法。

就讲到这里。

83-139 单词拆分

题目链接: <https://leetcode.cn/problems/word-break/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.24

标签: 字典树 记忆化 数组 哈希表 字符串 动态规划

推荐题解: 无

这道题, 我第一反应是用递归+记忆化搜索——感谢前一阵子练习的回溯题目, 让我对递归有了新的理解。虽是dp题目, 但是我一时半会没想到递推dp怎么做。

先讲讲递归的思路:

1. 将 `wordDict` 丢进无序集合中, 用于判定子串是否存在于其中;
2. 在dfs函数内开循环, 从字符串头开始往后找"落刀点"。如果在这里切, 切出来的子串能和集合里面的字符串匹配上, 那就可以做选择了: 切或者不切。如果切, 就正常递归即可; 不切, 就啥也不用干, 继续往后就行。

上代码:

```
C++
class Solution {
public:
    bool dfs(string s){
        if (mem.count(s)) return mem[s];
        if (s.size()==0) return true;
        for (int i=0;i<s.size();i++){
            auto sub=s.substr(0,i+1);
            if (hash.count(sub)){
                //可以选择切
                if (dfs(s.substr(i+1))){
                    mem[s]=true;
                    return true;
                }
                //往下就是选择不切
            }
        }
        mem[s]=false;
        return false;
    }

    bool wordBreak(string s, vector<string>& wordDict) {
        for (auto x:wordDict) hash.emplace(x);
        return dfs(s);
    }
private:
    unordered_set<string> hash;
    unordered_map<string,bool> mem;
};
```

我这个代码也算是极限过测了：虽说字符串作为 `mem` 的key开销挺大，但是直观啊！反正都是倒数的速度，这方面要不要优化我倒是无所谓了。

接着来看看如何优化成递推。一句话：定义 `dp[i]` 为0到i这个子串是否能被拆分。

如何遍历？继续拆：再来一个j指针，在0到i之间扫描，如果0到j已经能被拆分，并且j+1到i这个子串存在于 `hash` 中，就说明0到i可以被拆分，标记 `dp[i]=true` 即可。

上代码：

```
C++
class Solution {
public:
    bool wordBreak(string s, vector<string>& wordDict) {
        unordered_set<string> hash;
        for (auto x:wordDict){
            hash.emplace(x);
        }
        int n=s.size();
        vector<bool> dpfalse;

        ;

        dp[0]=true;
        for (int i=0;i<n;i++){
            for (int j=0;j<=i;j++){
                if (dp[j] && hash.count(s.substr(j,i-j+1))){ //如果0到j已经能被拆
                    分，并且j+1到i这个子串存在于hash中，就说明0到i可以被拆分
                    dp[i+1]=true;
                    break;
                }
            }
        }
        return dp[n];
    }
};
```

就讲到这里。

84-300 最长递增子序列

题目链接：<https://leetcode.cn/problems/longest-increasing-subsequence/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.24

标签：数组 二分查找 动态规划

推荐题解：无

我没招了。现在做dp题目，第一反应都是想到递归+记忆化。和之前截然不同。

递归加记忆化，或者说是二维dp的做法是这样的：记录 上一步选了啥，以及 当前的下标 在哪里，如果 满足严格递增条件，就可以选择选或者不选当前数字；否则就直接往后然后不选。

特别要注意，如果想要记忆化，就必须写有返回值的递归函数，我第一版把答案直接维护在参数里面了，虽然很直观，但是没法记忆化会超时。

下面是我第二版的代码：

```
C++  
class Solution {  
public:  
    int dfs(vector<int>& nums, int pos, int lastPos) {  
        if (pos == nums.size()) {  
            return 0;  
        }  
        if (mem[pos][lastPos+1] != -1) return mem[pos][lastPos+1];  
        // 选这个  
        int l1=0, l2=0;  
        if (lastPos == -1 || nums[lastPos] < nums[pos]) {  
            l1 = dfs(nums, pos + 1, pos)+1;  
        }  
        // 不选这个  
        l2 = dfs(nums, pos + 1, lastPos);  
        mem[pos][lastPos+1]=max(l1, l2);  
        return max(l1, l2);  
    }  
  
    int lengthOfLIS(vector<int>& nums) {  
        int n = nums.size();  
        mem.assign(n+1, vector<int>(n+1, -1));  
        return dfs(nums, 0, -1); // 由于lastPos初始值为-1，所以要么特判，要么就把  
        // lastPos整体+1偏移，否则会越界。我选择了后者  
    }  
  
private:  
    int maxLength = 0;  
    vector<vector<int>> mem;  
};
```

接着，我们就可以很轻松的知道如何转换成二维递推dp：整一个正方形的二维dp数组，然后外层遍历当前数字，内层遍历可能的上一个位置，然后选符合条件的并且最大的接上来+1——有没有发现这样有点多余？直接整个一维数组 `dp[i]` 记录以下标i处数字为结尾的最长递增子序列长度就行了。

第三版代码如下：

```

class Solution {
public:
    int lengthOfLIS(vector<int>& nums) {
        int n=nums.size();
        vector<int> dp(n,0);
        int maxAns=1; //由于数组最短为1，所以答案最小为1
        dp[0]=1;
        for (int i=1;i<n;i++){ //往前遍历
            int maxLength=0;
            for (int j=0;j<i;j++){ //回头去找潜在的能接上的结尾
                if (nums[j]<nums[i]){ //必须保持递增
                    maxLength=max(maxLength,dp[j]); //才会纳入最大值的考虑范围
                }
            }
            dp[i]=maxLength+1; //找到了最大值，接上当前数字，长度加一即为：当前数字结尾的
            //递增子序列长度
            maxAns=max(maxAns,dp[i]); //同时维护最长的长度作为答案
        }
        return maxAns;
    }
};

```

就讲到这里。

85-152 乘积最大子数组

题目链接：<https://leetcode.cn/problems/maximum-product-subarray/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.24

标签：数组 动态规划

推荐题解：[我的思路来源](#)

dp真是变幻莫测。。。之前还说这部分题目属于是我领域展开，现在看来是展开了个寂寞。

回到正题。这道题我甚至连递归的方法都没想出来。看了一下题解，豁然开朗：与其去处理复杂的负负得正、正负变负等逻辑，不如直接硬性维护最大和最小值，其中最小值的作用是维护绝对值最大的负数，这样如果需要负负得正，就能实现完美跃迁，得到最佳结果。

我们每一个数字的最大最小乘积需要分如下三种情况计算：

1. 当前数字 和 前一位数字为结尾的连续子数组总乘积 的 最大值 相乘，可能正正得正，也可能正负得负；
2. 当前数字 和 前一位数字为结尾的连续子数组总乘积 的 最小值 相乘，可能负负得正，也可能正负得负；
3. 直接掐断，从当前数字新开始。

上述三种情况，由于正负号的作用，都有可能是最大和最小。因此，简单粗暴：当前位最大值就是上面三者取max，最小值则取min。

```

class Solution {
public:
    int maxProduct(vector<int>& nums) {
        int n=nums.size();
        vector<int> dpMax(n,0),dpMin(n,0);
        dpMax[0]=nums[0];
        dpMin[0]=nums[0];
        for (int i=1;i<n;i++){
            dpMax[i]=max({dpMax[i-1]*nums[i],dpMin[i-1]*nums[i],nums[i]});
            dpMin[i]=min({dpMax[i-1]*nums[i],dpMin[i-1]*nums[i],nums[i]});
        }
        int ans=-INT_MAX;
        for (auto& x:dpMax) ans=max(ans,x);
        return ans;
    }
};

```

最后，由于最大值位置的不确定性，我们不知道以哪个数字结尾的子数组乘积总和最大，因此不要无脑取最后一位返回，需要再遍历一遍(或者再上面连带着把这个逻辑做了也行，反正时间复杂度一样)找最大值。

就讲到这里。

86-416 分割等和子集

题目链接：<https://leetcode.cn/problems/partition-equal-subset-sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.25

标签：数组 动态规划

推荐题解：无

这道题最重要的是要识别马甲：看似问你 能不能分隔成等和的两个子集，其实是——你先把这个数组求和记为sum，然后看看 能不能01背包凑出和为 $\text{sum}/2$ 的子序列。当然，如果和是奇数，可以直接排除了。

把这个马甲去掉，就很简单了，就是很经典的能不能凑成的01背包问题。我们定义商品为 `nums` 中的数字，背包容量为 $\text{sum}/2$ ，背包的单格物品类型为 `bool`，即讨论能不能在我选择到了这个数字的时候装这么多东西。

```

class Solution {
public:
    bool canPartition(vector<int>& nums) {
        //1.先计算总和
        int sum=reduce(nums.begin(),nums.end());
        if (sum%2!=0) return false;
        sum/=2;
        //2.开始凑
        int n = nums.size();
        vector<vector<bool>> dp(n+1,vector(sum+1,false));
        dp[0][0]=true;
        for (int i=0;i<n;i++){
            auto x=nums[i];
            for (int j=0;j<=sum;j++){
                //这里是状态继承/转移的简洁写法: 要么是不选当前数字, 直接继承上一层状态;
                //要么是在能选当前数字的情况下选当前数字, 并且得保证上一层能凑到j-x, 才能叠
                //上当前数字, 否则就是空中楼阁
                dp[i+1][j]= dp[i][j] || (j-x>=0 && dp[i][j-x]);
            }
        }
        return dp[n][sum];
    }
};

```

就讲到这里。

87-32 最长有效括号

题目链接: <https://leetcode.cn/problems/longest-valid-parentheses/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 困难

解题日期: 2026.2.25

标签: 栈 字符串 动态规划

推荐题解: [这次不是灵神, 但是这个题解也简直是神](#)

来到了本次hot100笔记的 第10道 Hard题!

看到这道题, 我的第一反应是一头雾水。

看了题解, 有用栈做的, 有用动态规划做的, 看了依然一头雾水。

直到我看到了这个神级题解——原来Hard题也可以如此简单??

不绕弯子, 讲讲基本思路: 先用栈左右匹配括号, 利用栈天然处理嵌套的优势, 把对得上的一对对左右括号所在的下标处都打上1 (新建一个和原字符串等长的数组)。然后, 下一步要做的事情简单到可怕: 数这个数组最长的连续1, 就是答案。

```

class Solution {
public:
    int longestValidParentheses(string s) {
        // 1.先匹配括号, 并将匹配的标记为1
        int n = s.size();
        vector<int> sIndex(n, 0);
        stack<int> left;
        for (int i = 0; i < n; i++) {
            if (s[i] == '(') {
                left.emplace(i);
            } else {
                if (!left.empty()) {
                    sIndex[left.top()] = 1;
                    left.pop();
                    sIndex[i] = 1;
                }
            }
        }
        // 2.找最长连续1
        int ans = 0, temp = 0;
        for (int i = 0; i < n; i++) {
            if (sIndex[i] == 1) {
                temp++;
                ans = max(ans, temp);
            } else {
                temp = 0;
            }
        }
        return ans;
    }
};

```

我感觉这个做法已经比很多中等题要简单了。。。.

对比一下这道题的三种主流解法:

维度	DP 递推 (正统派)	单栈一次遍历 (技巧派)	标记法 (工程派/邪修)
核心逻辑	定义 $dp[i]$ 为以 i 结尾的最长有效长度, 通过 $s[i-1]$ 寻找匹配。	栈中预存 -1 , 遇到 $)$ 弹出并计算 $i - stack.top()$ 。	第一步匹配标记, 第二步找连续 1 。
转移方程/规则	$dp[i] = dp[i-1] + 2 + dp[i - dp[i-1] - 2]$ (极其绕脑)。	弹出后空栈需压入当前索引作为新锚点。	只要栈不空, 弹出 $left.top()$ 并将两处设为 1 。
代码可读性	差。充满了下标减法, 极易写错 $+1$ 或 -2 。	中。逻辑紧凑但“预存 -1 ”的初衷不直观。	极佳。逻辑解耦, 每一步的功能清清楚楚。
空间复杂度	$O(n)$ (DP 数组)	$O(n)$ (栈)	$O(n)$ (栈 + 标记数组)
时间复杂度	$O(n)$ (一次遍历)	$O(n)$ (一次遍历)	$O(n)$ (两次线性遍历)

维度	DP 递推（正统派）	单栈一次遍历（技巧派）	标记法（工程派/邪修）
面试容错率	低。稍微忘一个下标偏移就全盘皆错。	中。需要记住“空栈压入索引”的特殊处理。	极高。两个子模块都是基础逻辑，很难写错。

我还是记住这个邪修方法吧，毕竟面试能写出来才是王道。

88-62 不同路径

题目链接： <https://leetcode.cn/problems/longest-valid-parentheses/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.25

标签：数学 动态规划 组合数学

推荐题解：无

这题其实就是二维版本的爬楼梯，核心就一句话：因为我只能向下或者向右，因此，我到达这一格的方法总数=我到上面这一格的方法总数+我到左边这一格的方法总数。

至于如何初始化？依旧是老生常谈的起点设置为1，因为站着不动也是一种方法。你也可以理解成这是dp一整个累加过程的万物起源。

```

class Solution {
public:
    int uniquePaths(int m, int n) {
        vector<vector<int>> dp(m+1, vector(n+1, 0));
        for (int i=0; i<m; i++){
            for (int j=0; j<n; j++){
                if (i==0 && j==0) dp[1][1]=1;
                else dp[i+1][j+1]=dp[i][j+1]+dp[i+1][j];
            }
        }
        return dp[m][n];
    }
};

```

C++

就讲到这里。

89-64 最小路径和

题目链接: <https://leetcode.cn/problems/minimum-path-sum/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.25

标签: 数组 动态规划 矩阵

推荐题解: 无

和上一题有异曲同工之妙, 并且有点贪心的味道: 每一步都选取总和最少的上一步(左边或者上面)接上, 然后累计到终点, 就是最优解。

```
class Solution {  
public:  
    int minPathSum(vector<vector<int>>& grid) {  
        int m=grid.size(),n=grid[0].size();  
        vector<vector<int>> dp(m+1,vector(n+1,INT_MAX));  
        for (int i=0;i<m;i++){  
            for (int j=0;j<n;j++){  
                if (i==0 && j==0){  
                    dp[1][1]=grid[i][j];  
                }else{  
                    dp[i+1][j+1]=min(dp[i][j+1],dp[i+1][j])+grid[i][j];  
                }  
            }  
        }  
        return dp[m][n];  
    }  
};
```

就讲到这里。

90-5 最长回文子串

题目链接: <https://leetcode.cn/problems/longest-palindromic-substring/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.26

标签: 双指针 字符串 动态规划

推荐题解: 无

没想到啊, 这道题, 放在多维DP的分类里面, 结果DP反而是最慢的?

这道题有三个主流解法: 中心扩散法($O(n^2)$)、DP优化中心扩散法(实则负优化)(也是 $O(n^2)$, 但是比前者慢很多), 以及拉爆全场的Manacher算法($O(n)$)。

我选择了第一个方法：中心扩散法。最后一个方法虽快，但是过于复杂，对于一个中等题来说，掌握这个不值得。

基本思路很简单。我们分类讨论：

1. 先以单个字母为中心，也就是回文串长度为奇数，初始左右指针全部指向同一个字母。只要左右指针指向的字母都相同并且不越界，就一直扩散。
2. 扩散完毕，如果比当前最长长度长，就记录下新的最长长度和起始坐标。
3. 再假设回文串长度为偶数，也就是以两个相同的字母为中心。步骤一样，扩散就完事，然后扩散完毕了记录。
4. 最后通过记录下来的信息，切出子串返回。

上代码：

```
C++
class Solution {
public:
    string longestPalindrome(string s) {
        int n=s.size();
        int maxLength=0,maxL=0,maxR=0;
        for (int i=0;i<n;i++){
            //这个循环相当于分类讨论单/双字母为中心的情况
            for (int j=0;j<=1;j++){
                int l=i,r=i+j;
                if (s[l]!=s[r]) break;
                while (l>=0 && r<n && s[l]==s[r]){
                    l--;
                    r++;
                }
                if (r-l-1>maxLength){
                    maxL=l+1;
                    maxR=r-1;
                    maxLength=maxR-maxL+1;
                }
            }
        }
        return s.substr(maxL,maxLength);
    }
};
```

就讲到这里。

91-1143 最长公共子序列

题目链接：<https://leetcode.cn/problems/longest-common-subsequence/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.2.26

标签：字符串 动态规划

推荐题解：无

这道题的精髓在于字符串错配的情况下的处理方式。

我们先定义dp二维数组，其中 `dp[i][j]` 的定义：`text1[0,i]`和`text2[0,j]`的最长公共子序列。由于我选择 **1base** 来省去分类讨论，因此 `dp[0][0]` 的意义：两个字符串一个都不选的情况下的最长公共子序列，那当然也是0。

讲讲dp思路吧。开好双层循环，分别遍历`text1`和`text2`。咱们分类讨论：

1. 如果 `text1[i]` 和 `text2[j]` 一样，说明匹配上了，这是我们唯一加长度的情况。此时，转移状态：`dp[i+1][j+1]=dp[i][j]+1`，代表直接继承`[0,i-1]`和`[0,j-1]`已经算好的最长公共子序列长度，加上1就是`[0,i]`和`[0,j]`的最长公共子序列。
2. 如果字母不一样，那这就是这道题的精髓了——我们需要看看 把哪边的字母掐掉一位，得到的答案更大。为啥要掐字母呢？我的理解是：为了模拟子序列的"不连续"性。既然不连续，总会有字母被拿掉，导致子串"中空"，变成子序列。字母总得是一个个拿掉的吧？那我们就得讨论一下，把哪个字母拿掉了。也简单，就看看之前咱们dp算出来的结果，拿到哪边的字母结果最大就继承哪边的结果(从别的角度理解我依旧感觉很牵强，只能就题论题的理解了)。转移方程如下：`dp[i+1][j+1]=max(dp[i+1][j],dp[i][j+1])`。

思路讲完了，上代码：

```
C++
class Solution {
public:
    int longestCommonSubsequence(string text1, string text2) {
        int m=text1.size(),n=text2.size();
        vector<vector<int>> dp(m+1,vector(n+1,0));
        for (int i=0;i<m;i++){
            for (int j=0;j<n;j++){
                if (text1[i]==text2[j]){
                    dp[i+1][j+1]=dp[i][j]+1;
                }else{
                    dp[i+1][j+1]=max(dp[i+1][j],dp[i][j+1]);
                }
            }
        }
        return dp[m][n];
    }
};
```

思路弄清楚了，代码是不难的。我感觉我这个解释确实也挺费力，如果下次有了更新的理解，我会补充的。

就讲到这里。

92-72 编辑距离

题目链接: <https://leetcode.cn/problems/edit-distance/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.2.28

标签: 字符串 动态规划

推荐题解: 无

这题还是比较难理解, 因此, 我优先想办法记住怎么写, 二刷的时候说不定就能真正理解这道题了。

当然, 这道题最重要的初始化部分, 还是可以理解的。在上完代码之后再讲。

其中 `dp[i][j]` 的定义和上一题差不多: `text1[0,i]` 转换到 `text2[0,j]` 的最小步数。

这道题其实就是上面这道题的改编版本, 把原本十分局限的 选与不选 的二元操作拓展到了 替换、插入和删除 三个操作。做题思路如下:

1. 如果 `w1[i] == w2[j]`, 说明这一位字母不需要变, 直接继承前面的状态就行: `dp[i+1][j+1] = dp[i][j]` (dp数组为1base, i和j为0base循环, 因此+1);
2. 如果不相等, 就得实行一些操作来变换。操作如下:
 - `dp[i-1][j] + 1` (删除):
你把 `word1` 的当前字符删了, 相当于进度退回了 `i-1`, 但 `j` 的目标还没变。
 - `dp[i][j-1] + 1` (插入):
你在 `word1` 末尾插了一个和 `word2[j-1]` 一样的字符, 这时 `j` 的进度完成了, 但 `i` 的原字符还在。
 - `dp[i-1][j-1] + 1` (替换):
最粗暴的, 直接把 `word1[i-1]` 强行改成 `word2[j-1]`, 两个进度同时推进一步。然后咱们三者取最小即可。

虽说讲述的是做法, 但是也稍微能嗅到一点拆子问题的味道。咱们上代码:

```

class Solution {
public:
    int minDistance(string word1, string word2) {
        int m=word1.size(),n=word2.size();
        vector<vector<int>> dp(m+1,vector(n+1,0));
        //下面三行是初始化
        dp[0][0]=0;
        for (int i=0;i<m;i++) dp[i+1][0]=i+1; //w1的前i位如果想转换成w2的前0位(即空串)需要有几位删几位, 因此操作次数就等于其长度
        for (int j=0;j<n;j++) dp[0][j+1]=j+1; //w1的前0位(即空串)如果想转换成w2的前j位, 需要有几位就插入几位, 因此操作次数就等于其长度
        for (int i=0;i<m;i++){
            for (int j=0;j<n;j++){
                if (word1[i]==word2[j]){
                    dp[i+1][j+1]=dp[i][j];
                }else{
                    dp[i+1][j+1]=min({dp[i+1][j],dp[i][j+1],dp[i][j]})+1;
                }
            }
        }
        return dp[m][n];
    }
};

```

为啥要这样初始化已经在注释里面讲的很清楚了。就讲到这里。

93-136 只出现一次的数字

题目链接: <https://leetcode.cn/problems/single-number/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.28

标签: 位运算 数组

推荐题解: [灵神的题解](#)

这题就是纯技巧题了。说是只允许常数级别额外空间, 我想的是排序然后两个一组遍历配对, 这个思路比较简单, 时间复杂度 $O(n\log n)$, 就不写了。

这题真正的技巧是: 位运算。利用了两个相同的数做异或运算结果为0的特性。

(从题解里面偷的)例如 $\text{nums}=[4,1,2,1,2]$, 把所有元素异或:

$$\begin{aligned}
 &= 4 \oplus 1 \oplus 2 \oplus 1 \oplus 2 \\
 &= 4 \oplus (1 \oplus 1) \oplus (2 \oplus 2) \\
 &= 4 \oplus 0 \oplus 0 \\
 &= 4
 \end{aligned}$$

上代码:

C++

```
class Solution {
public:
    int singleNumber(vector<int>& nums) {
        int ans=nums[0];
        if (nums.size()>1){
            for (int i=1;i<nums.size();i++){
                ans ^= nums[i];
            }
        }
        return ans;
    }
};
```

就讲到这里。

94-169 多数元素

题目链接: <https://leetcode.cn/problems/majority-element/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 简单

解题日期: 2026.2.28

标签: 数组 哈希表 分治 计数 排序

推荐题解: [不错的思路](#)

这道题, 咱们来个一题两吃。第一种方法是经典的哈希表计数+判断, 代码非常简洁, 但是时间复杂度和空间复杂度都是 $O(n)$;第二种方法是来自于灵神题解的"打擂台"做法, 代码比第一种方法稍微复杂一点, 但是时间复杂度 $O(n)$, 额外空间只有 $O(1)$ 。

先看第一种, 没啥好讲的, 计数+判断就完事了。直接上代码:

C++

```
class Solution {
public:
    int majorityElement(vector<int>& nums) {
        unordered_map<int,int> cnt;
        for (auto &x:nums){
            cnt[x]++;
            if (cnt[x]>nums.size()/2) return x;
        }
        return 0;
    }
};
```

接着讲讲此题的重头戏: 方法二。规则如下:

1. 初始化擂主 `master=nums[0]` 和擂主的血量 `masterHP=1` ;
2. 直接从数组的第二位开始遍历(如果有)。如果是同个数字(说明是同一派力量), 血量+1; 如果不是, 说明需要互相斗争, 血量-1;

3. 时刻保持检查：如果血量归零，这个擂主就得换成下一个挑战者了，然后如此往复，笑到最后的就是此题要求的多数元素，即"严格众数"。

代码如下：

```
C++  
class Solution {  
public:  
    int majorityElement(vector<int>& nums) {  
        int master=nums[0],masterHP=1;  
        if (nums.size()==1) return nums[0];  
        for (int i=1;i<nums.size();i++){  
            if (masterHP==0){  
                master=nums[i];  
                masterHP=1;  
            }  
            if (nums[i]==master) masterHP++;  
            else masterHP--;  
        }  
        return master;  
    }  
};
```

至于这个方法为何可行？这个擂台比赛其实比的就是总力量谁最大，无论相同势力是否连续，都不影响。

就讲到这里。

95-75 颜色分类

题目链接：<https://leetcode.cn/problems/sort-colors/description/?envType=study-plan-v2&envId=top-100-liked>

难度：中等

解题日期：2026.3.1

标签：数组 双指针 排序

推荐题解：无

长话短说。这道题有两个主流方法：两次遍历和一次遍历法，都挺好理解。

首先是两次遍历法，时间复杂度 $O(n)$ 。第一遍计数0,1,2的个数，第二遍直接原地填写。

C++

```

class Solution {
public:
    void sortColors(vector<int>& nums) {
        int n=nums.size();
        array<int,3> cnt={0,0,0};
        for (auto &x:nums) cnt[x]++;
        for (int i=0;i<n;i++){
            for (int j=0;j<3;j++){
                if (cnt[j]>0){
                    nums[i]=j;
                    cnt[j]--;
                    break;
                }
            }
        }
    }
};

```

接着是一次遍历法。它有个专属名字叫“荷兰国旗问题”（Dutch National Flag Problem），由计算机科学家 Dijkstra 提出。

它的核心逻辑是：把 0 往左边扔，把 2 往右边扔，1 自然就留在了中间。

C++

```

class Solution {
public:
    void sortColors(vector<int>& nums) {
        int n=nums.size();
        int p0=0,p2=n-1,cur=0;
        while (cur<=p2){
            if (nums[cur]==0){
                swap(nums[cur],nums[p0]);
                p0++;
                cur++;
            }else if (nums[cur]==2){
                swap(nums[cur],nums[p2]);
                p2--;
            }else{
                cur++;
            }
        }
    }
};

```

那么为啥找到0的时候能 `cur++`，而找到2的时候不行？请看Gemini解释：

1. 核心铁律： `curr` 左边没有秘密

在这道题里，我们要维护一个关键的“领土现状”：

- `[0, p0]` 区域：全是 0（已确定）。
 - `[p0, curr]` 区域：全是 1（已确定）。
 - `[p2, n-1]` 区域：全是 2（已确定）。
 - `[curr, p2]` 区域：未知领域（待检查）。
-

2. 为什么交换 2 时， `curr` 不能动？

你的理解是对的：因为 `p2` 指向的是右侧的未知领域。当你把 `nums[curr]` 的 2 扔到后面，从 `p2` 换回来的那个数，我们完全不知道它是什么。

- 它可能是 0，需要扔到最左边。
- 它可能是 1，需要留在原地。
- 它甚至可能是另一个 2（需要继续扔到后面）。

因为换回来的是个“陌生人”，所以 `curr` 必须原地踏步，在下一轮循环里重新审视这个刚换过来的数。

3. 为什么交换 0 时， `curr` 可以放心 `curr++` ？

这里的逻辑和你的理解略有不同：你认为是因为换过来的是 1 或 2，其实真相更简单——换过来的只可能是 1（或者就是 `curr` 自己）。

为什么不可能是 2？因为 `curr` 是从左往右走的，所有的 2 在 `curr` 经过它们的时候，早就已经被交换到数组最后面去了。

- 既然 `p0` 在 `curr` 的左边（或者重合），那么 `p0` 所指向的位置，只可能是在 `curr` 扫过之后留下的 1。
- 所以，当你把 `curr` 位置的 0 和 `p0` 位置的 1 交换后，`curr` 位置现在确定拿到了一个 1。
- 既然 1 本该呆在中间，我们当然可以放心地 `curr++` 去看下一个数了。

这个逻辑确实有点绕，当然，面试可以用二次扫描做法兜底。

96-31 下一个排列

题目链接: <https://leetcode.cn/problems/next-permutation/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.3.1

标签: 数组 双指针

推荐题解: [讲的还行](#)

讲讲这题的思路吧:

1. 如果序列本身就是 从左往右看 的递减状态, 那只能通过反转整个序列来使其字典序加一;
2. 反之, 我们从右往左找第一个比右侧数字小的数, 记下来它的位置为 `pos`。此时, 这个数的右边从右往左看是递增状态;
3. 接下来, 我们需要找最小的比这个数字大的数, 看我上方标粗的特性, 因此, 我们只需要从右往左遍历, 然后第一个比这个数字大的数就是我们想要的数, 记录其位置为 `minPos`;
4. 然后交换 `pos` 和 `minPos` 指向的数;
5. 最后, 继续用这个标粗的特性: 我们不需要排序, 只需要反转, 就能使其从 从右往左看 的递增变成递减, 使 `pos` 数字右边的字典序保持最小, 以此来确保我们的字典序只加了1。

思路就是这么个思路, 上代码:

```

class Solution {
public:
    void nextPermutation(vector<int>& nums) {
        int n = nums.size();
        if (n == 1)
            return;
        int minPos = -1, pos = 0;
        // 1. 先找到第一个比右侧相邻数字小的数
        for (int i = n - 2; i ≥ 0; i--) {
            if (nums[i] < nums[i + 1]) {
                pos = i;
                break;
            }
            // 如果没找到, 就反转整个序列
            if (i == 0) {
                reverse(nums.begin(), nums.end());
                return;
            }
        }
        // 2. 寻找比当前值大的最小值
        for (int i = n - 1; i > pos; i--) {
            if (nums[i] > nums[pos]) {
                minPos = i;
                break;
            }
        }
        swap(nums[pos], nums[minPos]);
        reverse(nums.begin() + pos + 1, nums.end());
    }
};

```

就讲到这里。

97-287 寻找重复数

题目链接: <https://leetcode.cn/problems/find-the-duplicate-number/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.3.1

标签: 位运算 数组 双指针 二分查找

推荐题解: [灵感来源](#)

你能想到, 这玩意能和 **24-142 环形链表 II** 联系在一起???

长话短说, 这道题的思路其实就是链表的环判定算法。利用题目给定的条件:

给定一个包含 $n + 1$ 个整数的数组 `nums`, 其数字都在 $[1, n]$ 范围内 (包括 `1` 和 `n`), 可知至少存在一个重复的整数。

我们就可以通过把数字当成下一位下标来跳转的形式，把它看成链表。

那么为何环入口就是重复的数字呢？举个例子：假设这个数字是2，它出现了多次。我们第一次跳转2，无事发生，咱们继续往后...直到后面我们又遇到了2，有没有发现，按照这个跳转规则，我们又得重新走一遍老路，相当于是进入了链表的环。那么，作为环入口的2，就是我们要找的答案。

代码如下，就和142题形式差不多，只不过少了判环逻辑，以及往下跳一步的逻辑不同：

```
C++
class Solution {
public:
    int findDuplicate(vector<int>& nums) {
        int slow=0,fast=0;
        while (true){
            slow=nums[slow];
            fast=nums[nums[fast]];
            if (slow==fast) break;
        }
        slow=0;
        while (slow!=fast){
            slow=nums[slow];
            fast=nums[fast];
        }
        return slow;
    }
};
```

就讲到这里。

98-42 接雨水

题目链接：<https://leetcode.cn/problems/trapping-rain-water/description/?envType=study-plan-v2&envId=top-100-liked>

难度：困难

解题日期：2026.3.2

标签：栈 数组 双指针 动态规划 单调栈

推荐题解：[这波灵神真神了](#)

我承认，我之前确实被接雨水的鼎鼎大名唬住了，现在看来好像也就那样。

那么欢迎来到我们hot100刷题笔记的 第11道 困难题！

这题我整了个一题两吃：三次遍历(维护最大前后缀数组)和双指针做法。

先讲三次遍历。由于接雨水的核心就是木桶效应，有一个基本公式是： $\text{这个格子能接住水的量} = \min(\text{其左边最高的墙, 其右边最高的墙}) - \text{当前高度}$ 。其中，其左边或者右边最高的墙需要包括它自己，因此，这个式子能够处理接不住水的情况，因为自己减去自己就是0。

```

class Solution {
public:
    int trap(vector<int>& height) {
        //1.先计算正反向前缀最大值
        int n=height.size();
        vector<int> lMax(n,0),rMax(n,0);
        lMax[0]=height[0];
        rMax[n-1]=height[n-1];
        for (int i=1;i<n;i++){
            lMax[i]=max(lMax[i-1],height[i]);
        }
        for (int i=n-2;i>=0;i--){
            rMax[i]=max(rMax[i+1],height[i]);
        }
        //2.一趟扫描求出总值
        int ans=0;
        for (int i=1;i<n;i++){
            ans+=min(lMax[i],rMax[i])-height[i];
        }
        return ans;
    }
};

```

接着，来讲本题的 核心：双指针做法。基本思想也是木桶效应，可以将三次遍历优化为一次遍历， $O(n)$ 的额外空间优化成 $O(1)$ 的额外空间。

为啥可以这样优化呢？因为我们观察上面两次遍历的做法可以发现，其实左边最大值和右边最大值都是遍历到哪里用到哪里，类似于前缀和，随着指针往后遍历，前面的部分就被遗弃了，没啥用。

双指针的思路是这样的：

1. 准备两个指针，左指针和右指针，简称 **l** 和 **r**，分别初始化到数组的开头和结尾；
2. 再准备两个最值，称为 **lMax** 和 **rMax**，前者维护 **l** 指向的数前面数的最大值(包括其自身)，后者维护 **r** 指向的数后面数的最大值(包括其自身)；
3. 接着来到了双指针做法的精髓：谁小动谁。这也将木桶效应体现的淋漓尽致。公式是这样的：当前格水量 = **lMax-height[i]**。为啥不需要看右边呢？请听哈基米解释：

1. 核心逻辑：木桶原理（短板效应）

接雨水的公式你已经记住了： $water = \min(leftMax, rightMax) - height$ 。这意味着：水位高度只取决于左右两边那块比较矮的“短板”。

在双指针移动的过程中，我们维护了两个变量： $lMax$ （左边走过的最高墙）和 $rMax$ （右边走过的最高墙）。

当 $lMax < rMax$ 时：

1. 我们百分之百确定 左边最高的墙是 $lMax$ 。
2. 我们不确定 右边最高的墙是谁。但是！我们确定 右边 至少 有一个 $rMax$ 挡在那儿。
3. 既然 $lMax < rMax$ ，那么对于当前的 $left$ 位置来说，右边的约束（不管它具体多高）一定比左边的 $lMax$ 高。
4. 所以， $\min(\text{左边最高}, \text{右边最高})$ 的结果 一定是 $lMax$ 。

2. 逻辑推演：为什么不需要看“中间”？

你可能会担心：“万一 $left$ 和 $right$ 中间藏着一个超级大深坑，或者一个超级大高墙怎么办？”

- 如果中间有超高墙：那更好，它只会让右边的实际 $rightMax$ 变得更大。但只要 $lMax < rMax$ 成立，水位依然被左边的 $lMax$ 锁死。
- 如果中间有深坑：没关系，我们现在只结算 $left$ 指针指向的这一个格子的水。等 $left$ 指针往后挪到那个深坑时，它自然会被当时的 $lMax$ 重新计算。

换句话说：我们不是在一次性计算一整片湖泊，而是在一个格子一个格子地确认。只要我们确定了当前格子的一侧边界是“全局短板”，这个格子的水位就固定了。

为啥我不自己写呢？感觉哈基米比我讲的好。上代码：

```
class Solution {
public:
    int trap(vector<int>& height) {
        int n=height.size();
        int l=0,r=n-1,lMax=0,rMax=0,ans=0;
        while (l<r){
            lMax=max(lMax,height[l]);
            rMax=max(rMax,height[r]);
            if (lMax<rMax){
                ans+=lMax-height[l];
                l++;
            }else{
                ans+=rMax-height[r];
                r--;
            }
        }
        return ans;
    }
};
```

就讲到这里。小小接雨水，不过如此！哈哈哈。

99-53 最大子数组和

题目链接: <https://leetcode.cn/problems/maximum-subarray/description/?envType=study-plan-v2&envId=top-100-liked>

难度: 中等

解题日期: 2026.3.2

标签: 数组 分治 动态规划

推荐题解: [推荐一下我自己的笔记](#)

这题也算是非常经典了，属于是之前忘记做了，挺莫名其妙的。

基本思路就一句话：看看当前这一位是接上前面的数组好，还是从这一位重新开始好？

```

class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n=nums.size();
        vector<int> dp(n,0);
        dp[0]=nums[0];
        int ans=dp[0];
        for (int i=1;i<n;i++){
            dp[i]=max(nums[i],dp[i-1]+nums[i]); //看看当前这一位是接上前面的数组好，还是从这一位重新开始好
            ans=max(ans,dp[i]);
        }
        return ans;
    }
};

```

PS: 之前一直是写的两次遍历，现在看来一次遍历就够了，虽然时间复杂度没啥差别。

100-4 寻找两个正序数组的中位数

题目链接: <https://leetcode.cn/problems/median-of-two-sorted-arrays/description/>

难度: 困难

解题日期: 2026.3.3

标签: 高级工程师 数组 二分查找 分治

推荐题解: [我的灵感来源](#)

欢迎来到我们Hot100笔记的 第12道 困难题！也是我本次刷题的最后一道题。

作为hot100困难题目的难度天花板，这题属于是实至名归了。

我看了好几个题解，最后发现还是灵神的题解最好理解。双指针做法比较好理解，咱们直接跳过，我们直接欧美上手达到了甚至超越了题目要求的二分做法(时间复杂度 $O(\log(\min(m,n)))$)，比题目要求的 $O(\log(m+n))$ 还要快！)。

前排提示：先理解上方推荐题解的双指针做法，再往下看。

首先，为啥能二分？学习了双指针做法，我们可以发现，除了 第一组的最大值 $\leq$ 第二组的最小值 的找到答案的情况，其它情况要么是第一组在 `nums1` 取多了，要么是取少了。由于我们决定在 `nums1` 二分，因此，只要能判断我是取多了还是取少了，就可以通过二分大大加快速度。

那么如何判断我是取多了还是取少了？请看下图，我们通过例子的形式理解。

1	5	7	8	9
2	3	4	6	10

a 有 1 个数在第一组。
第一组的最大值是 6，
第二组的最小值是 5。
不符合要求。

1	5	7	8	9
2	3	4	6	10

a 有 2 个数在第一组。
第一组的最大值是 5，
第二组的最小值是 6。
符合要求！

1	5	7	8	9
2	3	4	6	10

a 有 3 个数在第一组。
第一组的最大值是 7，
第二组的最小值是 4。
不符合要求。

例子

我们先明确一点：序列递增，然后我们二分的又是上方的 `nums1`，所以，如果 `nums1[mid]` 偏小，才会往右，反之往左。

那我们该把 `nums1[mid]` 和谁比呢？看上图，看上方的不符合条件的情况，此时 `nums1[mid]=1`。

能把它和谁比较？无非三个选项：5(下一位)，6(第一组在`nums2`的最大数)以及10(第二组在`nums2`的最小数)。直接用 排除法：5肯定没有意义，因为`nums1`递增，下一个数肯定一直比这个数大，排除；6的话，是第一组内部的对比，请记住，我们是因为第一组的最大值太大了才会来判断这个东西，我们要和第二组的相关值对比，组内对比没法解决问题。

最后就只剩下10了，也就是第二组在`nums2`的最小值，咱们代入看看，是合法的：在图片上方部分的不合法情况中， $1 < 10$ ，所以能判断出`nums1`取少了，咱们也确实右移了指针，然后中间部分直接合法了；在图片下方部分的不合法情况中， $7 > 4$ ，说明`nums1`取多了，并且也能发现越往下走越不合法，然后也能发现如果回退一下，左移指针，就能回到合法状态。

为啥找第二组在`nums2`的最小值作为准绳是合法的？我目前只能通过排除法证明，但是Gemini的解释有略微不同，可供参考：

1. 为何向右二分是 `nums1[i] < nums2[j+1]` ?

这个逻辑的本质是“交叉比对”。

我们要找的完美切分点必须满足：左边所有数 $\leq$ 右边所有数。

因为 `nums1` 和 `nums2` 内部已经有序，所以我们只需要保证：

1. `nums1[i]` (左半部 1 区最大) $\leq$ `nums2[j+1]` (右半部 2 区最小)
2. `nums2[j]` (左半部 2 区最大) $\leq$ `nums1[i+1]` (右半部 1 区最小)

为什么满足 `nums1[i] < nums2[j+1]` 就要往右走？

当你进入 `else if` 时，说明你还没找到完美切分点（即 `g1Max > g2Min`）。这时候只有两种可能：

- 可能性 A：`nums1[i] > nums2[j+1]`（`nums1` 这一段拿多了，太强了，得左移 `i`）。
- 可能性 B：`nums2[j] > nums1[i+1]`（`nums2` 这一段拿多了，`nums1` 拿少了，得右移 `i`）。

你的代码里写的是 `if (nums1[i] < nums2[j+1]) l = i + 1;`。

这其实是在说：“虽然 `nums1[i]` 还没越界，但既然还没满足最终条件，那肯定是因为 `nums1` 拿得还不够多，导致 `nums2[j]` 太大了（可能性 B），所以我们要多拿点 `nums1`。”

好的，解决了最核心的二分逻辑，接下来就得处理找到答案的情况了。这和双指针做法是一致的：

1. 如果总长度为偶数，那就取第一组最大值和第二组最小值的平均数；
2. 如果总长度为奇数，那么我们就规定在短一些的数组上面二分(如果在长一些的数组上面二分，算出的在短数组上的下标会越界)，并且规定第一组的总数要长一个。那这样答案就很明显了：取第一组最大值作为答案即可。

基本思路讲完了，上代码：

```

class Solution {
public:
    // 虚拟索引函数，用于优雅处理边界情况
    int getNum(vector<int>& nums, int index){
        if (index==0) return -INT_MAX;
        if (index==nums.size()+1) return INT_MAX;
        return nums[index-1];
    }
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        if (nums1.size()>nums2.size()) swap(nums1,nums2); // 确保在短数组上二分
        // 在nums1二分找合适的点
        int n1=nums1.size(), n2=nums2.size();
        int l=0, r=n1, k=(n1+n2+1)/2; // 这里的k的算法可以保证总数是奇数的时候，第一组比第二组多一个
        while (l<=r){
            int i = l+(r-l)/2;
            int j = k-i;
            int g1Max=max(getNum(nums1,i),getNum(nums2,j)); // 第一组最大值
            int g2Min=min(getNum(nums1,i+1),getNum(nums2,j+1)); // 第二组最小值
            if (g1Max<=g2Min){ // 找到了
                if ((n1+n2)%2==0){ // 总数是偶数个
                    return (double)(g1Max+g2Min)/2;
                }else{ // 奇数个
                    return (double)g1Max;
                }
            }else if (getNum(nums1,i)<getNum(nums2,j+1)){ // nums1取少了
                l = i+1;
            }else{ // nums1取多了
                r = i-1;
            }
        }
        return 0.0; // 无效代码，仅仅为了保证编译不出错
    }
};

```

最后来讲讲为啥要通过虚拟索引引入哨兵：为了优雅处理第一组取0个和全部取的情况。前者我们返回 `-INT_MAX`，不干扰取max；后者返回 `INT_MAX`，不干扰取min。值得一提的是，灵神的做法要么是直接头插尾差这两个哨兵(时间复杂度 $O(m+n)$)，要么就直接偏移下标(这样时间复杂度才和我一样)，感觉还是不够优雅，可能我这个虚拟索引的做法更偏后端思维吧。

OK，就讲到这里！hot100笔记，就此完结！（后续二刷甚至三刷会更新的，keep watching!）